



Sistema Conversacional Baseado em LLM para Transparência e Explicabilidade em Modelos de Energia

GECAD

2025 / 2026

1230420 – Rui Gabriel Ribeiro Margarido

ISEP INSTITUTO SUPERIOR
DE ENGENHARIA DO PORTO

Sistema Conversacional Baseado em LLM para Transparência e Explicabilidade em Modelos de Energia

GECAD

2025 / 2026

1230420 – Rui Gabriel Ribeiro Margarido



Licenciatura em Engenharia Informática

Julho de 2026

Orientador ISEP: **Brígida Constança Teixeira**

Supervisor: **Filipe Ferreira Sousa**

*«À minha família, pelo apoio constante e pela confiança demonstrada ao longo deste
percurso. »*

Agradecimentos

Agradeço ao GECAD, por me ter permitido desenvolver este projeto num contexto real de investigação aplicada na área da inteligência artificial e dos sistemas de energia.

Agradeço à Professora Brígida Teixeira, orientadora do ISEP, pela disponibilidade, acompanhamento e feedback ao longo do desenvolvimento do trabalho. Agradeço igualmente ao meu supervisor, Filipe Sousa, pelo apoio técnico, pelas sugestões e pelo acompanhamento prestado durante a implementação da solução.

Agradeço ao Instituto Superior de Engenharia do Porto e aos docentes da Licenciatura em Engenharia Informática, pela formação e pelos conhecimentos que tornaram possível a concretização deste projeto.

Por fim, agradeço à minha família e aos meus amigos pelo apoio constante e pela motivação transmitida ao longo deste percurso.

Resumo

A utilização de modelos de *machine learning* na previsão de energia permite apoiar decisões operacionais em edifícios, mas levanta desafios de transparência e compreensibilidade. Estes desafios decorrem do facto de as previsões, métricas e explicações produzidas por estes sistemas serem frequentemente difíceis de interpretar por utilizadores sem conhecimento técnico especializado. Este trabalho propõe o *Explainable Artificial Intelligence Energy Chatbot*, um sistema conversacional baseado em *Large Language Models* que permite consultar, em linguagem natural, informação relacionada com modelos de previsão energética e respetivos artefactos de explicabilidade.

A solução foi desenvolvida segundo os princípios da *Clean Architecture*, colocando o *backend* como único orquestrador da interação com o *Large Language Model*, o *Retrieval-Augmented Generation*, o *Machine Learning Explainability & Forecasting API* e a persistência aplicacional. O sistema integra inferência local através do *LM Studio*, consulta de documentação validada com *Retrieval-Augmented Generation*, acesso a previsões e explicações de *Explainable Artificial Intelligence* através da *Machine Learning Explainability & Forecasting API*, autenticação de utilizadores, gestão de conversas, adaptação das respostas ao perfil do utilizador e registo de evidências. São disponibilizadas uma experiência de *chat* para utilizadores comuns e uma área administrativa para gestão de utilizadores, *corpus* documental e diagnóstico operacional.

A validação foi realizada através de testes unitários, de integração e *end-to-end*, demonstrando o cumprimento dos principais requisitos funcionais e não funcionais definidos. O trabalho evidencia que a combinação de *Large Language Models*, *Retrieval-Augmented Generation* e ferramentas externas pode aproximar modelos energéticos complexos de utilizadores com diferentes níveis de conhecimento, desde que as respostas sejam fundamentadas em fontes verificáveis e apresentadas de forma adequada ao respetivo perfil.

Palavras-chave (Tema):	Sistemas conversacionais, <i>Explainable Artificial Intelligence</i> , Transparência, Modelos de energia, <i>Retrieval-Augmented Generation</i> .
Palavras-chave (Tecnologias):	<i>Python</i> , <i>FastAPI</i> , <i>Next.js</i> , <i>PostgreSQL</i> , <i>Qdrant</i> , <i>Docker</i> , <i>LM Studio</i> .

Índice

1	<i>Introdução</i>	1
1.1	Enquadramento/Contexto	1
1.2	Descrição do Problema	2
1.2.1	Objetivos	3
1.2.2	Abordagem	4
1.2.3	Contributos	4
1.2.4	Planeamento do trabalho	5
1.3	Estrutura do relatório	5
2	<i>Estado da arte</i>	7
2.1	Fundamentos de IA e Explicabilidade	7
2.1.1	Large Language Models e arquitetura Transformer	7
2.1.2	Sistemas Conversacionais	8
2.1.3	<i>Explainable Artificial Intelligence</i>	9
2.1.4	Integração de LLMs com sistemas externos	10
2.2	Trabalhos relacionados	12
2.2.1	LLMs e agentes conversacionais em sistemas de energia	12
2.2.2	Agentes conversacionais explicáveis (<i>conversational XAI</i>)	13
2.2.3	XAI em modelos de previsão de energia	14
2.2.4	Síntese crítica e lacunas	14
2.3	Tecnologias existentes	15
2.3.1	Linguagem de programação e <i>framework</i> de <i>backend</i>	15
2.3.2	Modelo de linguagem	17
2.3.3	Mecanismos de integração com fontes externas	17
2.3.4	Serviço de explicabilidade - ForeXAI	19
2.3.5	Interface e implementação	19
3	<i>Análise e desenho da solução</i>	19
3.1	Domínio do problema	20
3.2	Requisitos funcionais	24
3.3	Requisitos não funcionais	26
3.4	Pressupostos	27

3.5	Desenho	29
3.5.1	Nível 1 - Contexto do Sistema.....	29
3.5.2	Nível 2 - Vista de Contentores	33
3.5.3	Nível 3 - Vista de Componentes.....	39
3.5.4	Design alternativo.....	49
4	Implementação da Solução	50
4.1	Escolha tecnológica.....	50
4.2	Descrição da implementação.....	50
4.2.1	Configuração do sistema	50
4.2.2	Orquestração de ferramentas	52
4.2.3	Integração RAG	54
4.2.4	Integração ForeXAI	55
4.2.5	Autenticação.....	55
4.2.6	Interface gráfica.....	56
4.2.7	Implementação.....	58
4.3	Testes.....	59
4.3.1	Testes unitários.....	59
4.3.2	Testes de integração.....	61
4.3.3	Testes end-to-end.....	62
4.4	Avaliação da solução.....	64
4.4.1	Requisitos funcionais	64
4.4.2	Requisitos não funcionais	65
4.4.3	Validação qualitativa	66
5	Conclusões.....	67
5.1	Objetivos concretizados	67
5.2	Limitações e trabalho futuro	68
5.3	Apreciação final	69
	Referências.....	71
Anexo A	Cronograma.....	77
Anexo B	SHAP - LIME	78
Anexo C	tools.json.....	79
Anexo D	profiles.json.....	80

Anexo E	<i>chat_prompts.json</i>	81
Anexo F	<i>rag_retrieval.json</i>	82
Anexo G	<i>_build_planner_prompt</i>	83
Anexo H	<i>_enforce_tool_usage</i>	84
Anexo I	<i>execute</i>	85
Anexo J	<i>_filter_tool_execution_results</i>	87
Anexo K	<i>_compose_assistant_reply</i>	88
Anexo L	<i>_ingest_selected_documents</i>	89
Anexo M	<i>_retrieve_context</i>	92
Anexo N	<i>Interface</i>	94

Índice de Figuras

Figura 1: Diagrama adaptado do Modelo de domínio	20
Figura 2: User aggregate.....	21
Figura 3: Conversation Agreggate	22
Figura 4: Assistant response aggregate.....	23
Figura 5: Vista de cenários do USER	30
Figura 6: Vista de cenários do master e do administrador	31
Figura 7: Vista lógica nível 1	31
Figura 8: Vista de processo genérica de nível 1	32
Figura 9: Vista lógica de nível 2	33
Figura 10: Vista de implementação de nível 2	34
Figura 11: Vista física de nível 2	35
Figura 12: Vista de processo genérica de nível 2	36
Figura 13: Vista de processo de nível 2 de início de conversaço.....	37
Figura 14: Vista de processo de nível 2 de envio de mensagem.....	37
Figura 15: Vista de processo nível 2 de gestão do corpus do RAG.....	38
Figura 16: Vista de processo nível 2 da autenticação	39
Figura 17: Representação da Clean Architecture e do seu fluxo	40
Figura 18: Vista lógica de nível 3	41
Figura 19: Vista de implementação de nível 3	42
Figura 20: Vista de processo genérica de nível 3	43
Figura 21: Vista de processo nível 3 de início de conversaço.....	44
Figura 22: Vista de processo de nível 3 de envio de mensagem.....	45
Figura 23: Vista de processo nível 3 de reindexação do corpus do RAG.....	47
Figura 24: Vista de processo nível 3 do envio de mensagem no frontend	48
Figura 25: Interface gráfica da autenticação.....	56

Figura 26: Vista do Sistema conversacional	57
Figura 27: Painel de administração	58
Figura 28: Execução dos testes unitários	61
Figura 29: Execução dos testes de integração.....	62
Figura 30: Relatório dos testes E2E	64
Figura 31: Diagrama de Gantt do planeamento do trabalho	77
Figura 32:Excerto do ficheiro tools.json.....	79
Figura 33: Excerto do ficheiro profiles.json.....	80
Figura 34: Excerto do ficheiro chat_prompts.json	81
Figura 35: Excerto do ficheiro rag_retrieval.json	82
Figura 36: Construção do prompt para o planner.....	83
Figura 37: Grounding de no_tool	84
Figura 38:Figura 38: Execução das tools(parte 1/2)	85
Figura 39: Execução das tools (parte 2/2)	86
Figura 40: Grounding pós execução	87
Figura 41: Construção da resposta do assistente.....	88
Figura 42: Fluxo RAG de ingestão documental (parte 1/3)	89
Figura 43: : Fluxo RAG de ingestão documental (parte 2/3)	90
Figura 44: Fluxo RAG de ingestão documental (parte 3/3)	91
Figura 45: Fluxo RAG de recuperação (parte 1/2)	92
Figura 46: Fluxo RAG de recuperação (parte 2/2)	93
Figura 47: Vista geral do chat	94
Figura 48: Vista especifica do painel de conversação	95
Figura 49: Painel de gestão de utilizadores.....	96
Figura 50: Painel de avaliação do estado das principais ligações	96
Figura 51: Painel de administração RAG	97
Figura 52: Painel de consulta de modelos ForeXAI	97

Figura 53: Painel de comparação da explicabilidade da resposta para diferentes perfis de utilizador.....	98
--	----

Índice de Tabelas

Tabela 1: Comparação de linguagens para o backend	16
Tabela 2: Comparação de frameworks web Python	16
Tabela 3: Comparação de abordagens de inferência LLM	17
Tabela 4: Comparação de abordagens RAG	18
Tabela 5: Comparação de vector stores	18
Tabela 6: Comparação de frameworks de frontend	19
Tabela 7: Requisitos funcionais	24
Tabela 8: Requisitos não funcionais	26
Tabela 9: Pressupostos	27
Tabela 10: Workflows predefinidos	51
Tabela 11: Perfis de utilizador e orientações de resposta	52
Tabela 12: Serviços docker compose	58
Tabela 13: Grupos de testes unitários por camada.....	59
Tabela 14: Testes de integração	61
Tabela 15: Cenários de teste E2E	63
Tabela 16: Estado de implementação dos requisitos funcionais	64
Tabela 17: Avaliação dos requisitos não funcionais.....	65

Notação e Glossário

ACID	Atomicidade, Consistência, Isolamento, Durabilidade
ALE	<i>Accumulated Local Effects</i>
API	<i>Application Programming Interface</i>
DIP	<i>Dependency Inversion Principle</i>
DTO	<i>Data Transfer Object</i>
E2E	<i>End-To-End</i>
ForeXAI	<i>Machine Learning Explainability & Forecasting API</i>
GECAD	<i>Research Group on Intelligent Engineering and Computing for Advanced Innovation and Development</i>
GGUF	<i>GGML Universal File</i>
HVAC	<i>Heat, Ventilation and Air Conditioning</i>
IA	Inteligência Artificial
ISEP	Instituto Superior de Engenharia do Porto
ISP	<i>Interface Segregation Principle</i>
LDAP	<i>Lightweight Directory Access Protocol</i>
LIME	<i>Local Interpretable Model Agnostic Explanations</i>
LLM	<i>Large Language Model</i>
LSP	<i>Liskov Substitution Principle</i>
ML	<i>Machine Learning</i>
OCP	<i>Open/Closed Principle</i>
pdp	<i>Partial Dependence Plots</i>
RAG	<i>Retrieval-Augmented Generation</i>
SD	<i>System Diagram</i>
SHAP	<i>Shapley Additive Explanations</i>

SRP	<i>Single Responsibility Principle</i>
SSD	<i>System Sequence Diagram</i>
UUID	<i>Universally Unique Identifier</i>
VOC	Compostos Orgânicos Voláteis
XAI	<i>Explainable Artificial Intelligence</i>

1 Introdução

A crescente integração de modelos de *machine learning* (ML) em sistemas de previsão de energia coloca desafios de transparência que vão além da qualidade preditiva (Ali, Akhlaq, et al., 2023), os diferentes utilizadores que dependem destes sistemas necessitam de aceder à informação produzida em formatos compreensíveis e adequados ao seu perfil. Esta necessidade motiva o desenvolvimento do *XAI Energy Chatbot*, um sistema conversacional baseado em *Large Language Models* (LLM) (Zhao et al., 2026) concebido no contexto do *Research Group on Intelligent Engineering and Computing for Advanced Innovation and Development* (GECAD) (GECAD, 2026), que permite interagir em linguagem natural com modelos de previsão e os respetivos artefactos de explicabilidade, garantindo que as respostas são sempre fundamentadas em dados reais e rastreáveis (Lewis et al., 2021a). As secções seguintes apresentam o enquadramento do projeto e a motivação que lhe está subjacente, descrevem o problema identificado, os objetivos definidos, a abordagem adotada e os principais contributos do trabalho.

1.1 Enquadramento/Contexto

A gestão energética eficiente de edifícios está, cada vez mais, ligada à capacidade de prever padrões de consumo e geração. A variabilidade introduzida pelos diferentes comportamentos dos utilizadores e a influência das condições meteorológicas associadas à integração de fontes renováveis tornam a previsão um elemento essencial para apoiar decisões operacionais, como o planeamento de cargas, a otimização de equipamentos e a identificação de situações anómalas (Prabhu et al., 2023).

O GECAD dispõe de modelos avançados de previsão de consumo e geração de energia, assim como de previsão de variáveis ambientais (temperatura, humidade, concentração de CO₂ ou compostos orgânicos voláteis (VOC)) em determinados espaços do edifício de investigação do Instituto Superior de Engenharia do Porto (ISEP). Estas previsões são produzidas com aproximação a tempo real, resultando da aplicação de modelos de ML treinados com dados recolhidos pelos sensores instalados no edifício.

O fluxo da obtenção destas previsões respeita a seguinte sequência: aquisição e integração de dados, pré-processamento, geração de previsões através de modelos de ML e posterior análise de explicabilidade. A explicabilidade é a propriedade de um modelo que torna as suas previsões e decisões compreensíveis e auditáveis, permitindo identificar de que

forma cada variável de entrada contribui para o resultado produzido. Técnicas de *Explainable Artificial Intelligence* (XAI), como *Shapley Additive Explanations* (SHAP) (Lundberg et al., 2026) ou *Local Interpretable Model-Agnostic Explanations* (LIME) (Ribeiro et al., 2016), são instrumentos concretos para alcançar esta propriedade. A importância da XAI tem crescido à medida que sistemas de apoio à decisão baseados em ML se tornam mais complexos, tornando a transparência e a rastreabilidade dos resultados uma necessidade reconhecida (Ali, Akhlaq, et al., 2023).

Apesar das previsões e dos artefactos explicativos estarem disponíveis, a sua compreensão exige, em muitos casos, formação técnica específica, dificultando assim a utilização dos resultados por diferentes perfis de utilizador. O verdadeiro valor da explicabilidade só se concretiza quando a informação produzida pode ser efetivamente compreendida pelo utilizador final e não apenas gerada pelo sistema (Dinis Castelhana, 2026).

Os LLMs têm-se afirmado como uma tecnologia de relevo na interação em linguagem natural e na transformação de informação técnica em explicações acessíveis e adaptadas ao utilizador (Singh & Namin, 2025; Zhao et al., 2023). A integração desta camada conversacional com o sistema de previsão e explicabilidade representa uma abordagem promissora para o aumento da transparência e compreensibilidade dos resultados bem como uma redução da barreira no acesso a métricas e explicações avançadas (Ghani et al., 2025; Michelon et al., 2025). A principal motivação para aceitar este desafio prende-se com oportunidade de aprofundar o conhecimento na área da inteligência artificial (IA), através da exploração detalhada de vários componentes como modelos de linguagem de grande escala, técnicas de explicabilidade e sistemas conversacionais. A integração destas tecnologias com sistemas reais de previsão e análise de dados permite compreender os desafios associados à aplicação prática de soluções baseadas em IA e, ao mesmo tempo, consolidar competências no desenvolvimento e integração de sistemas inteligentes.

1.2 Descrição do Problema

O GECAD possui modelos avançados de previsão de consumo e de geração de energia, complementados por componentes de XAI que permitem interpretar os fatores que influenciam os resultados. No entanto, a análise e interpretação destes resultados requerem conhecimento técnico especializado quer na leitura de métricas de desempenho, quer na interpretação de importâncias de variáveis ou na avaliação de artefactos explicativos como gráficos SHAP ou LIME. Dado que o edifício é utilizado por uma diversidade de perfis, esta exigência técnica limita a utilização efetiva da informação disponível pela maioria dos

utilizadores. Esta limitação traduz-se numa lacuna entre a disponibilidade de informação (previsões, métricas e explicações) e a capacidade dos utilizadores de a compreenderem e utilizarem para apoiarem decisões, como, por exemplo, identificar as razões de uma previsão de consumo elevada, comparar o desempenho de dois modelos ou entender quais as variáveis ambientais que mais influenciam os resultados (Singh & Namin, 2025). Para os utilizadores não técnicos, os *outputs* são muitas vezes apresentados com terminologia e detalhe excessivos, o que dificulta a interpretação correta e reduz a confiança no sistema. Os utilizadores especialistas, por seu lado, necessitam de explicações mais completas, com rigor técnico e transparência sobre limitações, pressupostos e validade dos resultados.

Portanto, o problema consiste na necessidade de adaptar os resultados produzidos pelo sistema de previsão e pelos componentes de XAI aos diferentes perfis de utilizador tornando a informação acessível, de fácil compreensão e útil. É também essencial garantir que qualquer informação comunicada tenha como base, exclusivamente, *outputs* reais do sistema e documentação validada, para reduzir o risco de respostas não fundamentadas.

1.2.1 Objetivos

O principal objetivo do projeto é conceber e implementar um sistema conversacional baseado em LLM que promova a transparência e a explicabilidade dos modelos de previsão de energia do GECAD, permitindo a interação em linguagem natural e explicações adaptadas ao perfil do utilizador. Mais especificamente, a solução deverá:

- Integrar um LLM, através de API compatível, responsável pela interpretação das questões em linguagem natural e pela geração de respostas contextualizadas;
- Integrar-se com o motor de previsão e com os componentes de XAI existentes, através de *web services*, garantindo acesso estruturado às previsões, métricas e artefactos explicativos;
- Adaptar automaticamente o nível de detalhe, a linguagem e o conteúdo das respostas ao perfil do utilizador, distinguindo entre utilizador comum do edifício, especialista em energia e especialista em ML;
- Disponibilizar uma interface conversacional *web* que permita aos utilizadores colocar questões em linguagem natural sobre previsões, comportamento do modelo, métricas de desempenho e resultados de explicabilidade;
- Assegurar que as respostas sejam geradas exclusivamente com base nos *outputs* reais do sistema e na documentação validada, evitando a geração de informação não fundamentada.

1.2.2 Abordagem

A solução foi concebida segundo os princípios da *Clean Architecture* (Medium, 2026) , organizada em cinco camadas com responsabilidades estritamente separadas, fronteira HTTP, serviços de aplicação, domínio, infraestrutura e composição de dependências. O *backend* atua como único orquestrador: o *frontend* comunica exclusivamente com o *backend* e nunca invoca diretamente o LLM, Retrieval-Augmented Generation (RAG) ou o ForeXAI. A injeção de dependências é gerida centralmente, tornando os adaptadores substituíveis sem alteração da lógica de negócio.

A integração do LLM com fontes de informação externas segue um *pipeline* de orquestração de ferramentas em quatro etapas (*planner*, *validator*, *executor* e *responder*), combinando RAG para incorporar documentação validada no contexto (Lewis et al., 2021b) com *tool calling* para consulta de serviços externos. As ferramentas disponíveis e os *workflows* predefinidos são declarados em ficheiro de configuração. As respostas são sempre fundamentadas por uma política de *grounding* baseada em limiares de qualidade, que assegura que a informação comunicada está ancorada em evidências verificáveis, reduzindo o risco de *hallucination* (Ali, Akhlaq, et al., 2023; Singh & Namin, 2025).

A adaptação das respostas ao utilizador é configurada declarativamente através de quatro perfis (utilizador comum, especialista em energia, especialista em ML e administrador). A inferência é realizada localmente através do LM Studio, com um modelo *open-source*, assegurando controlo sobre os dados e independência de APIs comerciais (Zhao et al., 2026).

1.2.3 Contributos

O trabalho desenvolvido disponibiliza ao GECAD um sistema conversacional baseado em LLM que permite interagir em linguagem natural com os modelos de previsão de energia e os respetivos artefactos de explicabilidade, tornando acessível a informação técnica produzida pelo grupo a utilizadores com diferentes perfis. A solução inova ao combinar RAG sobre documentação validada com *tool calling* para consulta de serviços XAI externos, garantindo que as respostas são sempre fundamentadas em evidências verificáveis e não em conhecimento parametrizado do modelo de linguagem.

Propõe uma arquitetura de referência para sistemas conversacionais XAI em que o *backend* atua como único orquestrador, estruturada segundo os princípios da *Clean Architecture* e operacionalizada por um pipeline de orquestração de ferramentas com ciclo controlado de planeamento, validação e execução. Esta separação de responsabilidades

assegura que o *frontend* nunca invoca diretamente o LLM, o RAG nem serviços externos, simplificando a evolução do sistema e o controlo da qualidade das respostas.

Contribui para a redução da barreira de acesso à informação de modelos energéticos, através de um mecanismo declarativo de adaptação de respostas a quatro perfis de utilizador, permitindo que a mesma plataforma sirva utilizadores com níveis de conhecimento distintos.

1.2.4 Planeamento do trabalho

O planeamento do trabalho encontra-se organizado nas fases de Pesquisa, Análise e desenho da solução, Implementação e Entrega final. Prevê atividades paralelas na fase de implementação, com o desenvolvimento e a avaliação da solução a decorrer em simultâneo com a escrita do relatório, de modo que a documentação acompanhe progressivamente o estado do sistema. O diagrama de *Gantt* com a calendarização semanal detalhada encontra-se na Figura 31 (ANEXO A).

1.3 Estrutura do relatório

Para além da introdução, este relatório contém mais quatro capítulos. O segundo capítulo apresenta o estado da arte relevante para o projeto, organizado em fundamentos teóricos, trabalhos relacionados e análise das tecnologias existentes que fundamenta as escolhas de implementação adotadas. O terceiro capítulo descreve a análise e o desenho da solução, incluindo o modelo de domínio, os requisitos, os pressupostos do sistema e o desenho arquitetural. O quarto capítulo documenta a implementação da solução, detalhando as opções técnicas em cada subsistema da arquitetura, os testes e a avaliação da solução face aos requisitos identificados. O quinto capítulo apresenta as conclusões, incluindo os objetivos concretizados, as limitações atuais e as oportunidades de melhoria identificadas para trabalho futuro, e uma apreciação final do processo de desenvolvimento.

2 Estado da arte

Neste capítulo é apresentado o estado da arte relevante para a realização do projeto, onde se começam por rever os fundamentos teóricos associados a LLMs, sistemas conversacionais, técnicas de XAI e as principais abordagens para integração de modelos de linguagem com sistemas externos. De seguida, discutem-se trabalhos relacionados que combinam previsão e gestão de energia com agentes conversacionais e métodos de explicabilidade, demonstrando as lacunas existentes e o enquadramento do sistema proposto. Para finalizar, apresentam-se as tecnologias existentes de modo a fundamentar as escolhas adotadas na implementação da solução.

2.1 Fundamentos de IA e Explicabilidade

Esta secção apresenta os conceitos teóricos que sustentam o presente trabalho, cobrindo quatro áreas: os modelos de linguagem de grande escala e a arquitetura que os suporta, os sistemas conversacionais, as técnicas de IA explicável e as abordagens para integrar modelos de linguagem com fontes de informação externas.

2.1.1 Large Language Models e arquitetura Transformer

Os LLMs são modelos de ML baseados em redes neuronais profundas, treinados com grandes volumes de texto de múltiplas fontes para aprenderem padrões linguísticos, relações semânticas e estruturas sintáticas complexas (Zhao et al., 2023). Contrastando com abordagens tradicionais baseadas em regras ou modelos estatísticos de menor dimensão, os LLMs apresentam uma elevada capacidade de generalização e desempenho consistente em diversas tarefas de processamento de linguagem natural, onde se incluem geração de texto, tradução, sumarização e resposta a perguntas (Minaee et al., 2025a; Zhao et al., 2023).

A maioria dos LLMs modernos baseia-se na arquitetura *Transformer*, introduzida por Vaswani et al. (2017), que substitui as redes recorrentes por mecanismos de atenção capazes de modelar dependências de longo alcance e processar sequências em paralelo. O texto é transformado em *tokens* e representado em vetores (*embeddings*), que são processados por camadas com mecanismos de *self-attention* e *multi-head attention*, que permitem ao modelo avaliar a relevância relativa de cada *token* no contexto da conversa e capturar diferentes tipos de relações entre palavras. Para preservar a ordem na sequência, são adicionadas codificações posicionais, e camadas *feed-forward* transformam as representações intermédias,

contribuindo para a capacidade do modelo aprender padrões complexos (Chitty-Venkata et al., 2023; Liu et al., 2023).

O aumento da dimensão dos modelos e dos dados de treino, aliado a técnicas como *fine-tuning* (ajuste supervisionado do modelo com dados específicos do domínio-alvo), *instruction tuning* (treino com pares instrução–resposta que aumenta a responsividade do modelo a comandos em linguagem natural) e alinhamento com *feedback* humano (orientação do comportamento do modelo de acordo com preferências humanas), possibilitou que os LLMs adquirissem capacidades de *zero-shot* e *few-shot learning*, ou seja, passaram a conseguir resolver tarefas para as quais não foram especificamente treinados, a partir de apenas uma descrição da tarefa ou de alguns exemplos. Simultaneamente, o crescimento de modelos *open-source*, como LLaMA (Llama, 2026), Falcon (Falcon, 2026) e Mistral (Mistral AI, 2026), facilitou a utilização de LLMs em contextos académicos e organizacionais, incluindo cenários de execução local que oferecem maior controlo sobre os dados (Minaee et al., 2025b; Zhao et al., 2026).

2.1.2 Sistemas Conversacionais

Os sistemas conversacionais (*chatbots*, assistentes virtuais ou agentes conversacionais) permitem aos utilizadores interagir em linguagem natural para obter informação, executar tarefas ou receber apoio em diversas áreas como serviço ao cliente, saúde ou educação. Exemplos amplamente utilizados incluem assistentes virtuais como a Siri (Apple, 2026), a Alexa (Amazon, 2026) ou o Google Assistant (Google, 2026), bem como sistemas de conversação de propósito geral como o ChatGPT (OpenAI, 2026a) (Casheekar et al., 2024). As abordagens tradicionais usadas na construção dos mesmos eram baseadas em regras que seguiam fluxos de diálogo pré-definidos e ofereciam elevado controlo sobre as respostas, mas apresentavam dificuldade em lidar com variações linguísticas e questões fora dos cenários previstos (Kaiyrbekov et al., 2025). O progresso das técnicas de ML conduziu ao desenvolvimento de *chatbots* baseados em LLMs, que interpretam o contexto das perguntas e geram respostas coerentes e contextualizadas. Podem distinguir-se três tipos principais de sistemas: *rule-based* que utilizam regras linguísticas e padrões fixos, *retrieval-based* que selecionam respostas com base em documentos ou bases de conhecimento existentes, e *generative*, que recorrem aos modelos de linguagem para gerar respostas novas. Os sistemas *rule-based* são adequados para domínios muito restritos e bem definidos, onde o conjunto de perguntas possíveis é limitado e as respostas podem ser completamente especificadas por regras, mas revelam-se pouco escaláveis perante a variabilidade da linguagem natural. Os sistemas *retrieval-based* são adequados quando existe uma base de conhecimento

estruturada e se pretende seguir estritamente o conteúdo dessas fontes, enquanto os sistemas *generative* oferecem maior flexibilidade para lidar com perguntas complexas ou formuladas de forma inesperada (Kovari, 2025a; Lin & Chen, 2023).

2.1.3 Explainable Artificial Intelligence

A utilização crescente de modelos de ML em sistemas de apoio à decisão aumenta a necessidade de compreender o seu comportamento, sobretudo quando envolvem modelos de caixa-negra como redes neurais profundas, cujo funcionamento interno é difícil de interpretar diretamente devido à sua complexidade. A XAI procura desenvolver métodos que justifiquem as previsões destes modelos, tornando-os mais transparentes por meio da identificação dos fatores que influenciam os resultados, facilitando processos de validação e a deteção de erros ou enviesamentos. Em domínios sensíveis, como energia, saúde ou finanças, a explicabilidade assume um papel fundamental, pois permite aumentar a confiança dos utilizadores ajudando-os a tomar decisões mais informadas (Ali, Abuhmed, et al., 2023; Saranya & Subhashini, 2023). Esta necessidade é igualmente reconhecida no quadro regulatório europeu, o AI Act (Regulamento UE 2024/1689) estabelece requisitos de transparência e explicabilidade para sistemas de IA de alto risco, incluindo aplicações no setor energético (Parlamento Europeu e Conselho da UE, 2024).

Os métodos de XAI podem fornecer explicações globais, que descrevem o comportamento médio de um modelo e a importância relativa das variáveis ao longo de todo o conjunto de dados, ou explicações locais, que se focam na interpretação de previsões individuais. Esta distinção permite analisar simultaneamente o comportamento global do modelo e as razões específicas associadas a uma decisão concreta, contribuindo para uma compreensão mais completa do sistema (Santos et al., 2023).

Entre os métodos mais utilizados em XAI destacam-se SHAP e LIME, ambos aplicáveis a uma grande variedade de modelos. O SHAP baseia-se na teoria de jogos cooperativos e utiliza valores de *Shapley* para quantificar a contribuição marginal de cada variável para o resultado de uma previsão, permitindo obter tanto explicações locais como globais. O LIME, por sua vez, cria um modelo simples e interpretável que aproxima localmente o comportamento do modelo original, analisando o efeito de pequenas perturbações nas variáveis de entrada sobre a previsão obtida (Ali, Akhlaq, et al., 2023).

De forma geral, o SHAP oferece explicações mais consistentes e teoricamente fundamentadas, embora possa ser computacionalmente exigente em modelos complexos, enquanto o LIME apresenta uma implementação conceptual mais simples e intuitiva para

explicações locais (Ali, Akhlaq, et al., 2023; Lundberg et al., 2026; Ribeiro et al., 2016). O Anexo B resume as principais diferenças entre os dois métodos.

Um dos principais desafios da XAI está relacionado com a forma como as explicações são apresentadas a diferentes perfis de utilizador, que necessitam de níveis distintos de detalhe e de respostas mais técnicas ou mais intuitivas. A combinação de técnicas de explicabilidade com sistemas conversacionais tem sido apresentada como uma abordagem promissora de forma a tornar os resultados de modelos complexos mais acessíveis a utilizadores não especialistas e, simultaneamente, mantendo informação relevante para especialistas. A persistência do equilíbrio entre precisão preditiva e interpretabilidade continua a ser um dos desafios centrais da área, dado que modelos mais complexos tendem a apresentar melhor desempenho preditivo, mas são normalmente mais difíceis de interpretar (Ali, Akhlaq, et al., 2023; Santos et al., 2023).

2.1.4 Integração de LLMs com sistemas externos

Os LLMs demonstram uma elevada capacidade para compreender e gerar linguagem natural, mas, quando utilizados de forma isolada, apresentam várias limitações. Da informação desatualizada, à impossibilidade de consultar diretamente fontes externas de dados e à tendência a gerar respostas plausíveis, mas incorretas (*hallucination*), são alguns exemplos. Para ultrapassar estas limitações, propõem-se abordagens de integração dos LLMs com mecanismos de recuperação de informação, bases de dados, serviços *web* ou motores analíticos, permitindo combinar a capacidade de interpretação em linguagem natural com o acesso ao conhecimento externo estruturado ou em tempo real. Neste contexto, as principais estratégias são o RAG, o *tool calling* e os sistemas baseados em agentes autónomos e a comparação destas abordagens fundamenta as decisões de implementação adotadas, detalhadas na secção 2.3 (Huang et al., 2025a; Ji et al., 2024a).

2.1.4.1 Retrieval-Augmented Generation

A abordagem RAG combina modelos de linguagem com mecanismos de recuperação de informação, de forma a incorporar no contexto de geração (instruções fornecidas ao modelo antes de gerar a resposta) partes de documentos ou conhecimento provenientes de fontes externas. Normalmente, a pergunta do utilizador é usada para consultar fontes externas, como documentos ou bases de conhecimento, e a informação mais relevante é incluída no *prompt* fornecido ao LLM, que gera então uma resposta fundamentada nessa informação. Esta estratégia permite reduzir o risco de respostas incorretas ao ancorar a geração em

conteúdo previamente existente, e facilita a atualização do conhecimento sem necessidade de re-treinar o modelo de linguagem (Lewis et al., 2021a).

2.1.4.2 *Tool Calling*

Outra abordagem relevante consiste na utilização de *tool calling*, que permite aos LLMs invocar diretamente ferramentas ou serviços externos durante uma interação. Nestes cenários, o modelo interpreta a intenção do utilizador e decide, com base no *prompt* e nas especificações das ferramentas disponíveis, se deve solicitar a execução de uma função externa, como uma API, uma base de dados ou um motor de previsão. Os resultados devolvidos por essas ferramentas são então incorporados no contexto da conversa, permitindo ao LLM sintetizar uma resposta final baseada em dados concretos, em vez de depender exclusivamente do conhecimento adquirido no treino (Meta Intelligence, 2026; OpenAI, 2026b). Os *sistemas baseados em agentes autónomos* representam uma evolução das abordagens anteriores, em que o LLM atua como um agente capaz de planejar e executar sequências de ações para resolver tarefas complexas, dividindo o problema em múltiplos passos de raciocínio. Embora estas abordagens ofereçam maior flexibilidade, colocam também desafios adicionais de controlo, previsibilidade e segurança, com maior risco de *hallucination* em cenários onde múltiplos modelos interagem entre si (Meta Intelligence, 2026; Wang et al., 2025). No presente trabalho, optou-se por um pipeline de orquestração controlado em alternativa à autonomia de agentes, garantindo maior rastreabilidade das respostas.

2.1.4.3 Estratégias para reduzir alucinações

Apesar do seu excelente desempenho em diversas tarefas de processamento de linguagem natural, os LLMs estão sujeitos ao fenómeno de *hallucination* onde geram respostas convincentes, mas factualmente incorretas. Em contextos técnicos como a previsão de energia e a explicação de modelos, a presença de alucinações pode comprometer a confiança dos utilizadores. Uma das estratégias mais eficazes consiste em limitar a geração de respostas à informação proveniente de fontes externas verificáveis (*grounding*), implementado através de RAG ou *tool calling* com políticas explícitas que instruem o modelo a utilizar apenas a informação fornecida no contexto. Em muitos sistemas, o *grounding* é complementado por técnicas de *prompt engineering* que definem cuidadosamente as instruções fornecidas ao LLM para reduzir a probabilidade de respostas incorretas e incluem passos de raciocínio estruturado (Ji et al., 2024b; Lewis et al., 2021b; Ronanki et al., 2025).

Adicionalmente, a utilização de *tool-based retrieval*, em que valores numéricos e métricas são obtidos diretamente de serviços externos especializados, e não gerados pelo LLM, permite eliminar inconsistências factuais nas respostas apresentadas. Mecanismos de verificação de respostas podem ainda ser aplicados para comparar a informação produzida pelo LLM com as fontes utilizadas no processo de *grounding*, detetando potenciais inconsistências antes da apresentação ao utilizador (Huang et al., 2025b; Ji et al., 2024c).

2.2 Trabalhos relacionados

Nesta secção são analisados trabalhos relacionados com o desenvolvimento de sistemas conversacionais baseados em LLMs em contextos de energia, agentes conversacionais orientados à explicabilidade e a aplicação de métodos de XAI em modelos de previsão de energia. O objetivo consiste em enquadrar o sistema proposto face ao estado da arte, identificando contribuições relevantes e lacunas que justificam o desenvolvimento de uma solução focada na transparência e adaptada a diferentes perfis de utilizador.

2.2.1 LLMs e agentes conversacionais em sistemas de energia

A utilização de LLMs como interface entre utilizadores e sistemas de energia tem sido explorada em diferentes contextos, em particular em sistemas de gestão energética em edifícios. Michelon et al. (2025) propõem uma interface baseada em LLM que permite a utilizadores domésticos especificar preferências e restrições em linguagem natural, as quais são convertidas em parâmetros estruturados para um sistema de gestão de energia. A solução interpreta comandos pouco estruturados, extrai intenções e gera configurações formais, com o objetivo de reduzir a complexidade da interação com o sistema e diminuir o esforço de configuração por parte dos utilizadores.

Num contexto diferente, mas ainda no domínio da gestão energética em edifícios, Ghani et al. (2025) apresentam o sistema SPARA como um agente conversacional que apoia cooperativas de habitação na identificação de medidas de eficiência energética. A abordagem combina um modelo de linguagem com uma arquitetura RAG sobre uma base de conhecimento construída a partir de comunicações reais entre consultores de energia e cooperativas, permitindo responder a perguntas em linguagem natural com recomendações fundamentadas em documentação existente.

Estes trabalhos demonstram que LLMs podem atuar como camada intermédia entre utilizadores e sistemas de energia, facilitando a interação em linguagem natural e a parametrização de sistemas de gestão de energia. No entanto, centram-se sobretudo em

tarefas de configuração e recomendação, sem abordar de forma aprofundada a explicação do comportamento dos modelos de previsão e das métricas associadas.

2.2.2 Agentes conversacionais explicáveis (*conversational XAI*)

Tem sido explorada a utilização de agentes conversacionais focados na explicação de modelos de ML, isto é, *conversational XAI*. Nguyen et al. (2024) propõem um agente conversacional que combina métodos de XAI com LLMs para explicar previsões de modelos de ML em diferentes domínios. A arquitetura inclui um módulo que seleciona e executa métodos de explicabilidade (por exemplo, SHAP), produz artefactos explicativos (tabelas, gráficos, métricas) e um LLM baseado no LLaMA-2 responsável por transformar esses artefactos em explicações textuais em linguagem natural, acessíveis a utilizadores com diferentes níveis de literacia técnica.

Kovari (2025b) analisa diversas abordagens em que *chatbots* incorporam mecanismos de explicabilidade, incluindo a apresentação de justificações, a indicação das fontes utilizadas para gerar respostas e técnicas de *grounding* para reduzir alucinações. Os autores concluem que a confiança dos utilizadores aumenta quando os sistemas conversacionais fornecem explicações claras sobre as razões subjacentes às respostas e quando estas estão explicitamente e transparentemente ligadas a dados ou documentos de suporte. A revisão enfatiza ainda a necessidade de integrar XAI de forma nativa em agentes conversacionais, mas a maioria dos estudos analisados incide sobre domínios como serviços financeiros, saúde ou apoio ao cliente, não existindo foco específico na previsão de energia.

No domínio de edifícios, Zhang & Chen (2024) propõem uma abordagem que combina valores SHAP e LLMs para explicar decisões de controlo em sistemas HVAC. A abordagem consiste em utilizar SHAP para quantificar a contribuição de variáveis (por exemplo, temperatura exterior, ocupação, horário) para ações de controlo e, em seguida, recorrer a um LLM para gerar descrições narrativas dessas contribuições, destinadas a operadores de edifícios. Embora o sistema não seja um *chatbot* generalista, demonstra a viabilidade de usar LLMs como camada de explicação em linguagem natural sobre artefactos de XAI em contextos energéticos.

Em conjunto, estes trabalhos mostram que agentes conversacionais e LLMs podem ser usados para tornar as explicações de modelos de ML mais acessíveis. Contudo, não integram, de forma completa, a explicação de modelos de previsão de energia com interação em linguagem natural adaptada a diferentes perfis de utilizador, nem abordam explicitamente

a restrição das respostas a outputs reais de sistemas específicos, como o motor de previsão do GECAD.

2.2.3 XAI em modelos de previsão de energia

A aplicação de métodos de XAI a modelos de previsão de energia tem sido explorada em diversos contextos, com foco na interpretação de modelos complexos e na identificação de padrões de consumo. Clement et al. (2024) utilizam valores SHAP para analisar a importância de variáveis e combinam essa informação com técnicas de *clustering* para identificar grupos de comportamentos energéticos. A análise das contribuições das variáveis em diferentes *clusters* permite detetar mudanças nos padrões de consumo e ajustar os modelos de previsão de forma adaptativa, demonstrando a utilidade da XAI não apenas na interpretação, mas também na melhoria contínua dos modelos.

Prabhu et al. (2023) aplicam técnicas de XAI, nomeadamente SHAP, a modelos de previsão de consumo e de deteção de anomalias em edifícios. Os autores utilizam modelos de gradiente reforçado (LightGBM) e recorrem a visualizações baseadas em SHAP para explicar, tanto a nível global como local, a contribuição de variáveis para as previsões de consumo e para a classificação de observações como anómalas. As explicações são apresentadas sob a forma de gráficos de importância das variáveis, de dependência parcial e de contributos individuais por observação, direcionadas sobretudo a analistas e especialistas técnicos.

Estes trabalhos evidenciam que métodos como SHAP são adequados para explicar modelos de previsão de energia, permitindo analisar importâncias de variáveis, erros e padrões de comportamento ao longo do tempo. No entanto, a interação com estes artefactos explicativos requer conhecimentos técnicos e acesso a ferramentas específicas, o que limita a acessibilidade da informação para utilizadores comuns de edifícios ou para decisores não especializados. Além disso, não é fornecida uma camada conversacional que traduza estas explicações em linguagem natural adaptada ao perfil do utilizador.

2.2.4 Síntese crítica e lacunas

A análise dos trabalhos relacionados permite identificar três dimensões principais. Em primeiro lugar, LLMs e agentes conversacionais começam a ser utilizados como interface entre utilizadores e sistemas de energia, em particular em sistemas de gestão de energia doméstica e agentes de eficiência energética, demonstrando que a interação em linguagem natural pode simplificar a configuração e o acesso à informação. Contudo, estes sistemas não se focam na explicação detalhada de modelos de previsão ou de métricas de desempenho, mas

principalmente, na tradução de preferências e na apresentação de recomendações. Em segundo lugar, agentes conversacionais explicáveis e revisões sobre XAI em *chatbots* mostram que é possível combinar artefactos de XAI com LLMs para gerar explicações textuais mais acessíveis, embora o foco seja, muitas vezes, genérico ou sem ligação direta a motores de previsão de energia concretos. Por fim, os trabalhos de XAI em modelos de previsão de energia evidenciam que já existem métodos consolidados para analisar as importâncias de variáveis, os erros e os padrões de consumo e geração de energia, mas as explicações são tipicamente apresentadas em formatos técnicos, o que dificulta o seu uso por parte de utilizadores não especializados.

Até onde foi possível apurar, não foi identificado um sistema que integre, simultaneamente, (i) um LLM ligado por *Application Programming Interface* (API) a um motor de previsão de energia e a componentes de XAI de um sistema real, (ii) associado à geração de respostas conversacionais baseadas exclusivamente em outputs reais e documentação validada e (iii) à adaptação automática do nível de detalhe, da linguagem e do conteúdo das explicações a perfis distintos de utilizador. Esta lacuna no estado da arte reforça a relevância do presente trabalho, que procura combinar estas dimensões numa solução única, orientada para a transparência e a acessibilidade das previsões e explicações produzidas pelos modelos de energia do GECAD.

2.3 Tecnologias existentes

Nesta secção são apresentadas as principais tecnologias avaliadas para a implementação do sistema conversacional proposto, abrangendo linguagem e *framework* de *backend*, modelo de linguagem, mecanismos de integração com fontes externas, serviço de explicabilidade e soluções de interface e Implementação.

2.3.1 Linguagem de programação e *framework* de *backend*

Para o *backend*, foram avaliados *Python*, *Node.js* e *Java* como linguagem de programação, e *FastAPI*, *Flask* e *Django* como *framework* web, como se pode observar na Tabela 1 e Tabela 2 respetivamente.

Tabela 1: Comparação de linguagens para o backend

Critério	Python ¹	Node.js ²	Java ³
Ecosistema IA/ML	Dominante	Limitado	Moderado
Interoperabilidade com ForeXAI (Python)	Nativa	Requer adaptação	Requer adaptação
Velocidade de desenvolvimento	Alta	Alta	Moderada
Frameworks web maduras disponíveis	FastAPI, Django, Flask	Express, NestJS	Spring

Tabela 2: Comparação de frameworks web Python

Critério	FastAPI ⁴	Flask ⁵	Django ⁶
Operações assíncronas (ASGI)	Nativo	Limitado	Limitado
Documentação <i>OpenAPI</i> automática	Sim	Não	Parcial
Validação integrada via <i>Pydantic</i>	Sim	Manual	Manual
Adequação a microserviços	Alta	Alta	Baixa (monolítico)
Construção de APIs REST	Nativa	Nativa	Nativa

Como se pode observar na Tabela 1, *Python* destaca-se das alternativas avaliadas pelo ecossistema dominante em IA e ML e pela interoperabilidade nativa com o ForeXAI, eliminando qualquer camada de adaptação que *Node.js* ou *Java* exigiriam. A Tabela 2 evidencia que, entre as *frameworks Python* avaliadas, o *FastAPI* se distingue pelo suporte nativo de operações assíncronas, geração automática de documentação *OpenAPI* e validação integrada via *Pydantic*, características ausentes ou limitadas no *Flask* e no *Django*.

¹ *Python Documentation, (2026)*² *Node.js V26.3.1 Documentation (2026)*³ *Learn Java - Dev.Java (2026)*⁴ *FastAPI (2026)*⁵ *Flask Documentation (2026)*⁶ *Django (2026)*

2.3.2 Modelo de linguagem

A inferência LLM pode ser realizada através de APIs comerciais (OpenAI⁷, Anthropic⁸) ou de modelos *open-source* executados localmente com o LM Studio (LM Studio, 2026), como se pode observar na Tabela 3.

Tabela 3: Comparação de abordagens de inferência LLM

Critério	APIs comerciais	LM Studio + modelo <i>open-source</i>
Controlo de dados	Dados enviados ao fornecedor	Processamento totalmente local
Custo por utilização	Por <i>token</i>	Zero (infraestrutura própria)
Substituição do modelo	Dependente do fornecedor	Flexível (qualquer modelo GGUF)
Suporte a <i>vision</i>	Disponível	Dependente do modelo instalado
Gestão de infraestrutura	Mínima	Requer configuração local

Como ilustrado na Tabela 3, o LM Studio com modelo *open-source* contrasta com as APIs comerciais em dois critérios decisivos: o processamento totalmente local elimina a transmissão de dados para fornecedores externos, e o custo zero de utilização por *token* é determinante para um contexto académico-institucional. A escolha do modelo concreto resulta de dois condicionantes adicionais, (1) as limitações de hardware disponível favorecem modelos de dimensão reduzida e (2) o sistema processa artefactos explicativos visuais (gráficos SHAP), exigindo capacidade de visão.

2.3.3 Mecanismos de integração com fontes externas

Para a implementação do mecanismo RAG foram avaliadas as *frameworks* LangChain e LlamaIndex e uma implementação própria, apresentadas na Tabela 4. Para *vector store*, conforme apresentado na Tabela 5, foram comparados Qdrant, Pinecone, Weaviate e ChromaDB.

⁷ OpenAI Developers (2026)

⁸ (Documentation - Claude API Docs, 2026)

Tabela 4: Comparação de abordagens RAG

Critério	LangChain ⁹	LlamaIndex ¹⁰	Implementação própria
Abstração adicional	Elevada	Moderada	Nula
Adequação a corpus pequeno e bem definido	Excessiva	Moderada	Total
Controlo sobre <i>tool calling</i>	Parcial	Parcial	Total
Dependências externas	Muitas	Moderadas	Mínimas

Tabela 5: Comparação de vector stores

Critério	Qdrant ¹¹	Pinecone ¹²	Weaviate ¹³	ChromaDB ¹⁴
Modelo de acesso	<i>Open-source</i>	<i>Cloud (comercial)</i>	<i>Open-source</i>	<i>Open-source</i>
REST + gRPC	Sim	Sim	Sim	Limitado
Docker nativo	Sim	Não	Sim	Sim
Garantias de produção	Altas	Altas	Altas	Moderadas
Complexidade de configuração	Baixa	Baixa	Alta	Baixa

Conforme evidenciado na Tabela 4, as *frameworks* *LangChain* e *LlamaIndex* introduzem abstrações excessivas ou moderadas para um *corpus* limitado e bem definido, e limitam o controlo sobre *tool calling*, aspeto central da arquitetura do sistema. Optou-se, por isso, por implementação RAG própria.

A Tabela 5 mostra que, entre os *vector stores* avaliados, o *Qdrant* combina modelo *open-source*, suporte *REST+gRPC*, *Docker* (*Docker*, 2026) nativo e baixa complexidade de

⁹ *LangChain* (2026)

¹⁰ *LlamaIndex Developer Documentation* (2026)

¹¹ *Documentation - Qdrant* (2026)

¹² *Pinecone Documentation - Pinecone Docs* (2026)

¹³ *Retrieval Augmented Generation | Weaviate* (2026)

¹⁴ *Chroma Docs* (2026)

configuração, sendo o único com garantias de produção equiparáveis ao *Pinecone* sem depender de infraestrutura *cloud* comercial.

2.3.4 Serviço de explicabilidade - ForeXAI

A produção de artefactos de explicabilidade é da responsabilidade do ForeXAI, serviço externo de utilização obrigatória, desenvolvido e disponibilizado pelo GECAD, acedido exclusivamente via HTTP. O ForeXAI disponibiliza métodos como SHAP global/local, LIME, importância por permutação, PDP, ALE, Anchors e Counterfactuals.

2.3.5 Interface e implementação

Para o frontend foram avaliadas frameworks orientadas a ML *Gradio* (*Gradio*, 2026) e *Streamlit* (*Streamlit Documentation*, 2026) e soluções web generalistas como *React* (*React*, 2026), conforme apresentado na Tabela 6.

Tabela 6: Comparação de frameworks de frontend

Critério	React	Gradio	Streamlit
Autenticação e gestão de perfis	Completa	Limitada	Limitada
Layouts e componentes customizados	Total	Pré-definidos	Pré-definidos
Separação <i>frontend/backend</i>	Explícita	Acoplada	Acoplada
Adequação a produção	Alta	Prototipagem	Prototipagem

Como se pode observar na Tabela 6, *React* é a única opção avaliada que oferece autenticação completa, suporte a *layouts* customizados e separação explícita entre *frontend* e *backend*, requisitos indispensáveis para um sistema com múltiplos perfis de utilizador e painel administrativo. A implementação baseia-se em *Docker* e *Docker Compose* (*Docker*, 2026) para containerização de todos os serviços, a persistência de utilizadores, conversas e mensagens é assegurada por *PostgreSQL* (*PostgreSQL Global Development Group*, 2026), cuja conformidade das transações que respeitam a atomicidade, consistência, isolamento, durabilidade (ACID) e o modelo relacional se adequam à estrutura do sistema.

3 Análise e desenho da solução

Esta secção apresenta a análise e o desenho da solução proposta, partindo da compreensão do domínio do problema para a identificação dos requisitos, pressupostos e decisões arquiteturais. A exposição segue uma progressão do geral para o particular, modelo de domínio, requisitos funcionais e não funcionais, pressupostos de integração e desenho da

objects, e é caracterizado por dois conceitos distintos: o *ProfileType*, que determina a base de conhecimento usada para adaptar o conteúdo e o nível de detalhe das respostas (utilizador comum, especialista em energia, especialista em ML ou administrador), e o *Role*, que determina o nível de acesso às funcionalidades administrativas do sistema (utilizador comum, administrador ou *master*). Esta distinção é necessária porque a adaptação da resposta e o controlo de acesso ao *backoffice* são preocupações independentes, um especialista em energia, por exemplo, não tem necessariamente acesso às funcionalidades de gestão do sistema. Relativamente à autenticação esta é gerida via LDAP, à exceção do *master* cuja autenticação é gerida internamente pela aplicação utilizando uma *password* local.

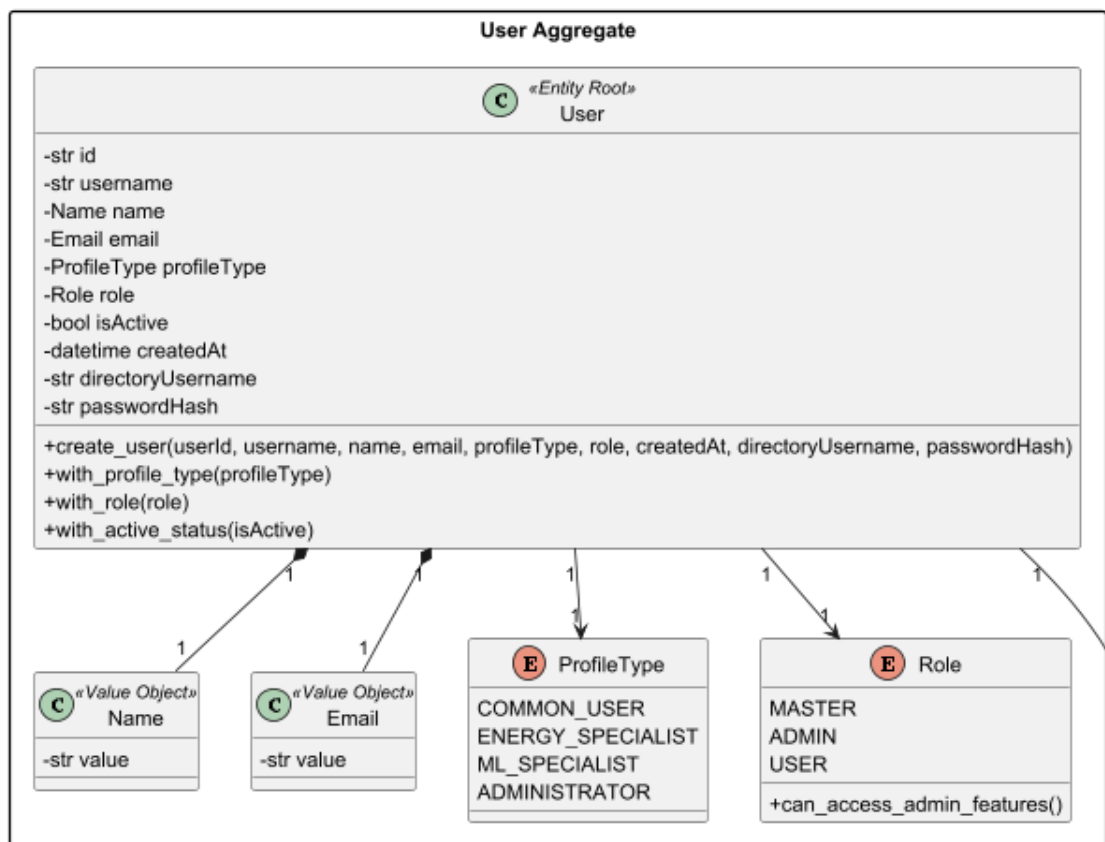


Figura 2: User aggregate

A entidade *Conversation*, Figura 3, representa uma interação conversacional entre um utilizador e o sistema. Cada conversa pertence a um único utilizador e encontra-se associada a um estado, podendo estar ativa ou encerrada. Para além disso, a conversa regista o instante de início e, quando aplicável, o instante de término. Este conceito permite modelar o histórico de interação e preservar o contexto ao longo do diálogo.

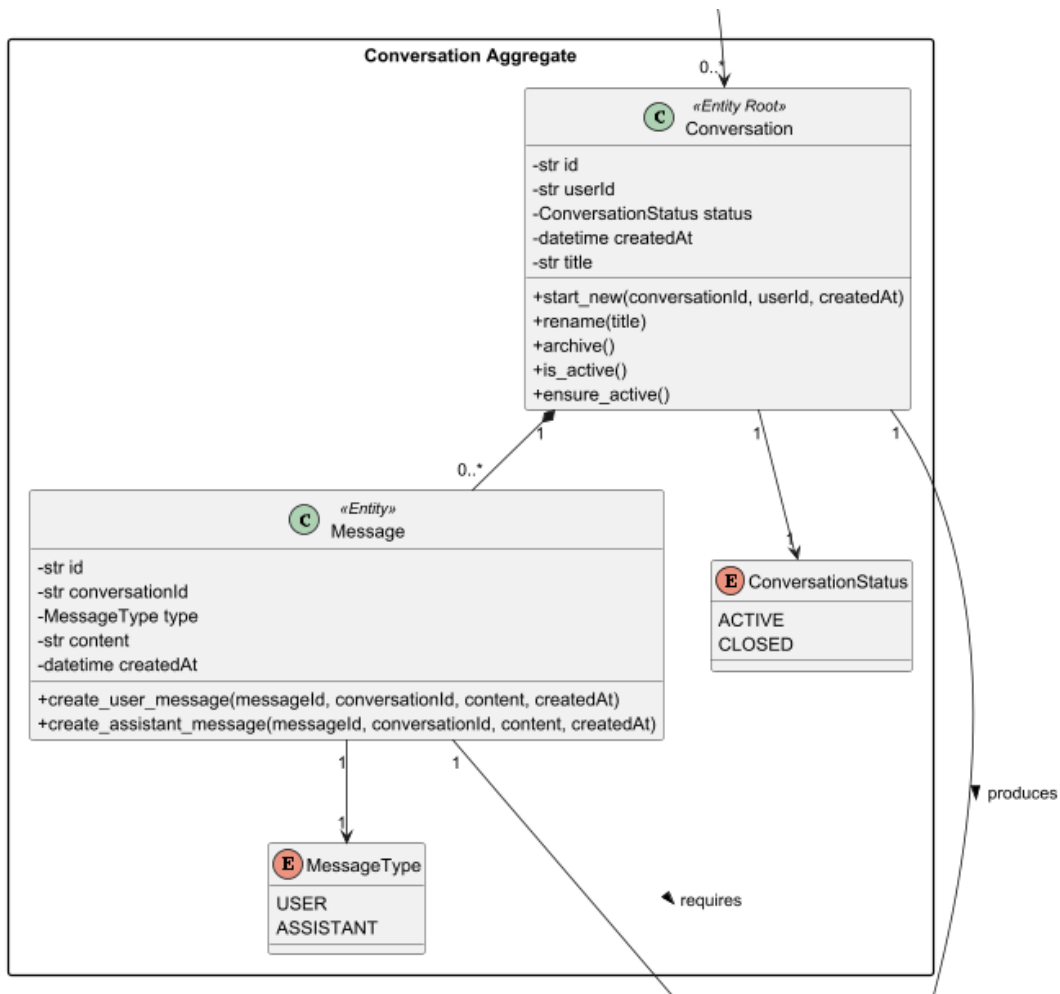


Figura 3: Conversation Aggregate

A entidade *Message* representa cada mensagem trocada no âmbito de uma conversa. Cada mensagem possui uma ordem sequencial, conteúdo textual, instante de criação e um tipo associado, permitindo distinguir mensagens do utilizador de mensagens do assistente. Desta forma, a conversa passa a ser composta por uma sequência ordenada de mensagens que refletem o diálogo estabelecido entre ambas as partes.

A entidade *AssistantResponse* representa a resposta produzida pelo sistema para uma mensagem do tipo assistente. Cada resposta está associada à conversa e à mensagem do assistente que lhe deu origem, regista o nome do LLM que a gerou e possui um estado de conclusão (*ResponseStatus*). É composta por um conjunto de *ToolInvocation*, que documentam cada ferramenta efetivamente invocada, e por um conjunto de *ResponseEvidence*, que representam os elementos concretos usados para fundamentar a resposta (excerto, *score* de relevância, *scores* de similaridade vetorial e de *overlap* lexical, e tipo de evidência, distinguindo contexto recuperado por RAG de resultados devolvidos pelo ForeXAI).

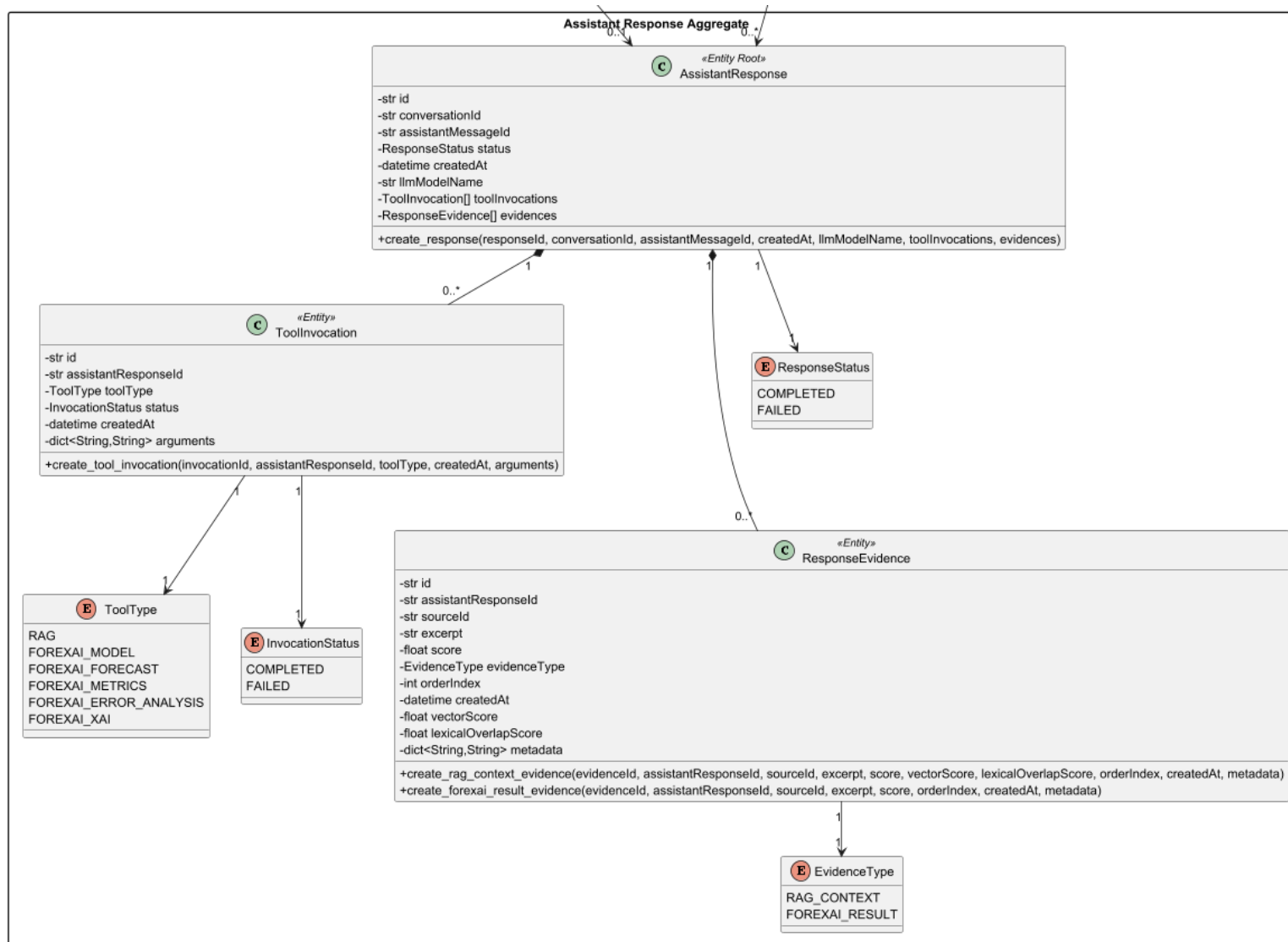


Figura 4: Assistant response aggregate

As relações entre as entidades refletem o fluxo do sistema: um *User* possui várias *Conversation*, cada *Conversation* é composta por várias *Message* e pode originar várias *AssistantResponse*, cada *Message* do tipo assistente está associada a, no máximo, uma *AssistantResponse*, e cada *AssistantResponse* agrega as suas *ToolInvocation* e *ResponseEvidence*.

Desta forma, o modelo de domínio concentra-se nos conceitos essenciais do problema, a interação entre utilizador e sistema, a adaptação ao perfil de conhecimento e ao nível de acesso de cada utilizador, e a necessidade de produzir respostas fundamentadas e rastreáveis às fontes que as suportam.

3.2 Requisitos funcionais

Os requisitos funcionais detalham as funcionalidades que o sistema deve disponibilizar, isto é, as tarefas e comportamentos que deve executar para responder às necessidades identificadas no domínio do problema. A Tabela 7 apresenta os requisitos funcionais identificados para o sistema proposto, bem como os casos de uso que lhes dão resposta.

De acordo com o modelo (F)URPS+¹⁶, todos os requisitos apresentados nesta secção pertencem à categoria F (*Functionality*), uma vez que descrevem as funcionalidades que o sistema deve assegurar. As restantes categorias do modelo serão consideradas na especificação dos requisitos não funcionais.

Tabela 7: Requisitos funcionais

Requisito	Caso de Uso
1: Iniciar e gerir conversas	UC1: Como utilizador, quero iniciar uma nova conversa. UC2: Como utilizador, quero consultar o histórico de uma conversa. UC3: Como utilizador, quero encerrar uma conversa.
2: Submeter perguntas em linguagem natural	UC4: Como utilizador, quero submeter perguntas em linguagem natural ao sistema.

¹⁶ <https://medium.com/practical-software-testing/test-cases-taxonomy-a32900e1d994>

3: Gerar respostas adaptadas ao perfil do utilizador	UC5: Como utilizador, quero obter respostas adaptadas ao meu perfil.
4: Consultar resultados de previsão energética	UC6: Como utilizador, quero consultar resultados de previsão energética através da interface conversacional.
5: Consultar explicações de modelos e resultados de XAI	UC7: Como utilizador, quero obter explicações sobre os resultados dos modelos de previsão.
6: Consultar métricas e análise de erros dos modelos	UC8: Como utilizador, quero consultar métricas de desempenho e análise de erros dos modelos de previsão.
7: Consultar documentação validada	UC9: Como utilizador, quero obter informação documental validada para suportar as respostas do sistema.
8: Selecionar e orquestrar a invocação de <i>tools</i>	UC10: UC10: Como sistema, quero planear, validar e executar as <i>tools</i> adequadas para responder ao pedido do utilizador.
9: Registrar evidências da resposta	UC11: Como utilizador, quero que a resposta apresentada inclua suporte explícito através de evidências.
10: Identificar o estado de conclusão da resposta	UC12: Como utilizador, quero saber se a resposta foi concluída com sucesso ou se falhou.
11: Gerir utilizadores e perfis	UC13: Como administrador, quero gerir utilizadores do sistema (ativar/desativar, consultar lista). UC14: Como administrador, quero atribuir ou alterar perfis e <i>roles</i> de utilizador.
12: Gerir o <i>corpus</i> documental do RAG	UC15: Como administrador, quero adicionar, atualizar, remover e reindexar documentos da base de conhecimento RAG. UC16: Como administrador, quero consultar o estado do <i>corpus</i> e o histórico de reindexações.

13: Consultar o catálogo de modelos ForeXAI	UC17: Como administrador, quero consultar os modelos ForeXAI disponíveis, os respectivos <i>datasets</i> e metadados, fora do fluxo conversacional.
--	--

3.3 Requisitos não funcionais

Os requisitos não funcionais caracterizam propriedades e restrições do sistema, não descrevendo funcionalidades específicas, mas sim qualidades que o produto deve assegurar durante a sua utilização, operação e manutenção. A Tabela 8 apresenta os requisitos não funcionais identificados, distribuídos pelas categorias do modelo (F)URPS+: usabilidade (*Usability*), confiabilidade (*Reliability*), desempenho (*Performance*), suportabilidade (*Supportability*) e a categoria adicional (+), usada para restrições de implementação, integração, operação ou segurança.

Tabela 8: Requisitos não funcionais

Requisito	Categoria
1: O <i>backend</i> deverá disponibilizar os seus serviços através de uma API, sendo o único orquestrador do LLM, do RAG e do ForeXAI.	Usabilidade; + Restrições de Implementação
2: O sistema deverá apresentar uma arquitetura que facilite a integração com serviços externos de previsão (ForeXAI), explicabilidade e documentação (RAG).	Suportabilidade; + Restrições de Implementação
3: O sistema deverá suportar a execução local do modelo de linguagem através do LM Studio, reduzindo a dependência de serviços externos de inferência.	Confiabilidade; + Restrições de Implementação
4: O sistema deverá registar as <i>tools</i> invocadas e as evidências utilizadas em cada resposta gerada.	Confiabilidade
5: O sistema deverá garantir que o histórico das conversas é preservado de forma consistente.	Confiabilidade
6: O sistema deverá permitir a adaptação e evolução das <i>tools</i> e fontes de informação sem necessidade de reestruturação profunda da arquitetura.	Suportabilidade

7: O sistema deverá ser passível de configuração e Implementação em ambiente containerizado (<i>Docker Compose</i>)	Suportabilidade; + Restrições de Implementação
8: O sistema deverá apresentar tempos de resposta adequados para utilização interativa em contexto conversacional.	Desempenho
9: O sistema deverá assegurar que apenas utilizadores autenticados podem aceder às funcionalidades conversacionais e administrativas, de acordo com o seu <i>role</i> .	Confiabilidade; + Segurança
10: O sistema e a sua documentação técnica deverão ser produzidos em inglês sempre que tal seja necessário para integração com componentes, bibliotecas ou serviços externos.	Usabilidade; Suportabilidade

3.4 Pressupostos

O desenvolvimento da solução proposta assentou num conjunto de pressupostos que delimitam o seu contexto de aplicação, integração e evolução, apresentados na Tabela 9.

Tabela 9: Pressupostos

Pressuposto	Descrição
1: Integração no contexto tecnológico do GECAD	A solução será desenvolvida para integração no ecossistema tecnológico do GECAD, considerando os serviços, infraestruturas e tecnologias já existentes nesse contexto.
2: Implementação futura em servidor do GECAD	A solução será posteriormente implantada num servidor do GECAD, sendo esse o ambiente previsto para a sua execução em contexto real.
3: Utilização do LM Studio para inferência local	A geração de respostas em linguagem natural recorrerá a um modelo de linguagem executado localmente através do LM Studio.
4: Interface conversacional construída de raiz	A interface de <i>chat</i> foi construída especificamente para este projeto, centrando-se o desenvolvimento na lógica conversacional, na orquestração e na integração com as fontes de informação.

5: Integração de RAG	É requisito de solução a integração de um mecanismo de RAG, com o objetivo de suportar as respostas geradas com base em documentação e informação validada.
6: Disponibilização prévia das fontes de dados	Os dados de previsão energética, métricas de desempenho, artefactos de explicabilidade e documentação relevante estarão disponíveis para consumo pela solução, através de serviços, ficheiros ou outras fontes acessíveis.
7: Documentação validada e preparada para recuperação	Assume-se que a documentação a utilizar no processo de recuperação de informação se encontra validada, organizada e adequada ao seu processamento no mecanismo de RAG.
8: Consumo de resultados já produzidos	O sistema não terá como objetivo treinar ou recalcular modelos de previsão, mas sim consumir resultados produzidos por sistemas existentes.
9: Camada conversacional sobre serviços existentes	A solução atua como uma camada de interação e interpretação sobre resultados de previsão, explicabilidade e documentação, não substituindo os mecanismos analíticos já existentes.
10: Evolução incremental da solução	Assume-se que a solução será evoluída de forma incremental, podendo incorporar futuramente novos requisitos, novas <i>tools</i> e novas fontes de informação.
11: Utilizadores autenticados e perfis válidos	Assume-se que os utilizadores que interagem com o sistema estarão autenticados e associados a um perfil válido, permitindo adaptar o conteúdo e o nível de detalhe das respostas.
12: Acesso administrativo restrito	Assume-se que apenas utilizadores com permissões administrativas terão acesso às funcionalidades de gestão de utilizadores, perfis e configuração do sistema.
13: Preparação prévia dos dados e conteúdos	Assume-se que os dados e conteúdos fornecidos ao sistema já se encontram preparados, estruturados ou normalizados sempre que tal seja necessário para o seu processamento.

14: Segurança base assegurada pelo ambiente de integração	Assume-se que os mecanismos base de segurança de infraestrutura (rede, LDAP) são assegurados pelo ambiente tecnológico do GECAD, não constituindo o foco principal deste trabalho.
--	--

3.5 Desenho

O desenho do sistema é apresentado segundo o Modelo C4¹⁷, organizado em quatro níveis de abstração crescente, e complementado com as vistas do modelo arquitetural 4+1 (vista lógica, vista de implementação, vista física, vista de processos e vista de cenários).

O desenho da solução foi realizado tendo em consideração que o *backend* constitui o único orquestrador de toda a lógica de negócio, integrando o LLM, o RAG e o ForeXAI. O *frontend* atua exclusivamente como camada de apresentação, nunca contactando diretamente nenhum dos serviços de inferência ou de dados. Esta decisão arquitetural determina a estrutura de todos os diagramas de processos apresentados nesta secção.

3.5.1 Nível 1 - Contexto do Sistema

A vista de cenários de nível 1 identifica os atores e os comportamentos que o sistema suporta, numa perspetiva de caixa preta. Estão definidos três atores no sistema (USER, ADMIN e MASTER) diferenciados pelo seu nível de acesso que é controlado pela *role*. A Figura 5 demonstra os cenários disponíveis ao utilizador com *role* USER, que é utilizador apenas do chat do sistema. Os casos de uso UC01 a UC03 dizem respeito à gestão do ciclo de vida da conversa (Iniciar, Consultar Histórico e Encerrar), enquanto UC04 representa o fluxo principal de envio de mensagem, que incorpora obrigatoriamente UC05 (adaptação ao perfil), UC10 (orquestração de *tools*), UC11 (registo de evidências) e UC12 (identificação do estado de conclusão) através de relações de *include*. Os UC06 a UC09 (consulta de previsões, explicações XAI, métricas e documentação RAG) estendem UC04 em função do conteúdo do pedido.

¹⁷ <https://c4model.com/>
(C4 Model, 2026)

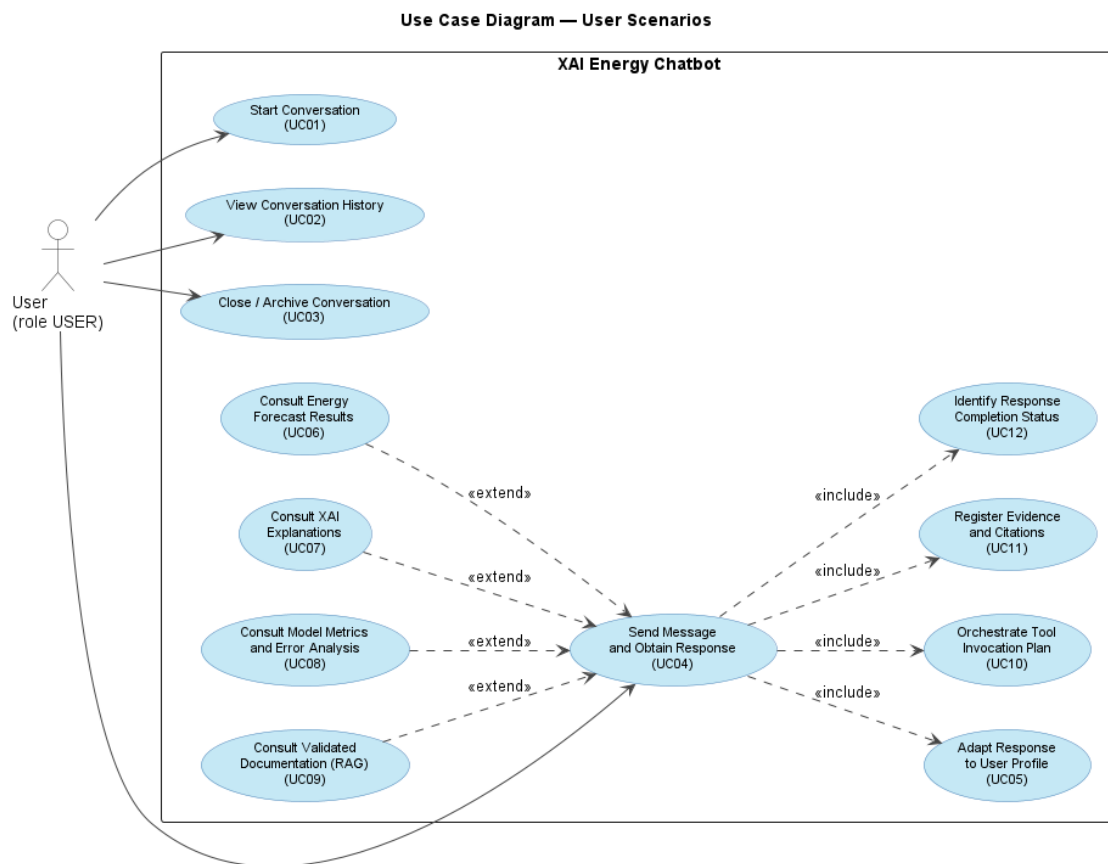


Figura 5: Vista de cenários do USER

A Figura 6 demonstra os cenários disponíveis ao administrador (*role ADMIN*) e ao *master* (*role MASTER*) através do painel de administração. O *master* herda todos os cenários do administrador e dispõe adicionalmente da capacidade de alterar *roles* de utilizador (UC14 na variante *MASTER*). Os cenários UC13 e UC14 cobrem a gestão de utilizadores e a atribuição de perfis, UC15 e UC16 cobrem a gestão e monitorização do corpus RAG e UC17 cobre a consulta do catálogo de modelos ForeXAI.

Use Case Diagram — Admin / Master Scenarios

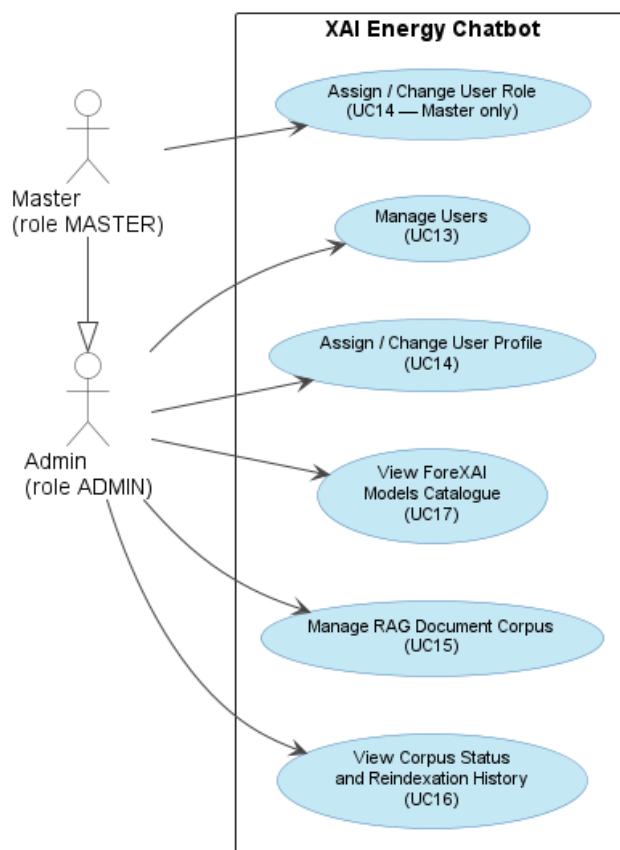


Figura 6: Vista de cenários do master e do administrador

A Figura 7 apresenta a vista lógica de nível 1 do sistema com o *XAI Energy Chatbot* como unidade única, as interfaces que disponibiliza aos seus atores (utilizadores USER e ADMIN/MASTER) e os sistemas externos com que interage, o LM Studio (servidor de inferência LLM local), a plataforma ForeXAI (API REST) e o serviço LDAP.

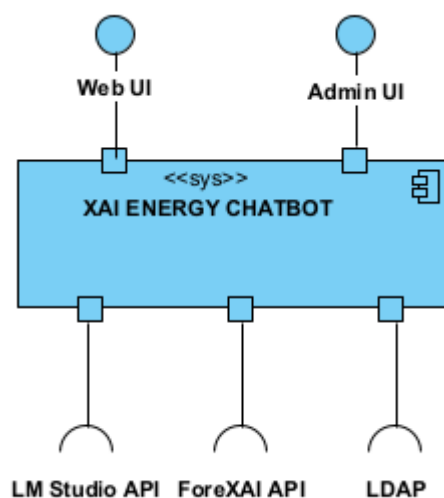


Figura 7: Vista lógica nível 1

O sistema atua como única fonte de orquestração em que nenhum componente externo (incluindo o *frontend*) contacta diretamente o LM Studio, o Qdrant ou o ForeXAI. Toda a comunicação passa obrigatoriamente pelo *backend*, garantindo um ponto único de controlo, auditabilidade e aplicação da política de *grounding*.

A Figura 8 apresenta um *System sequence diagram* (SSD) genérico de nível 1, que ilustra o fluxo de comunicação primário entre um ator externo e o sistema como caixa preta.

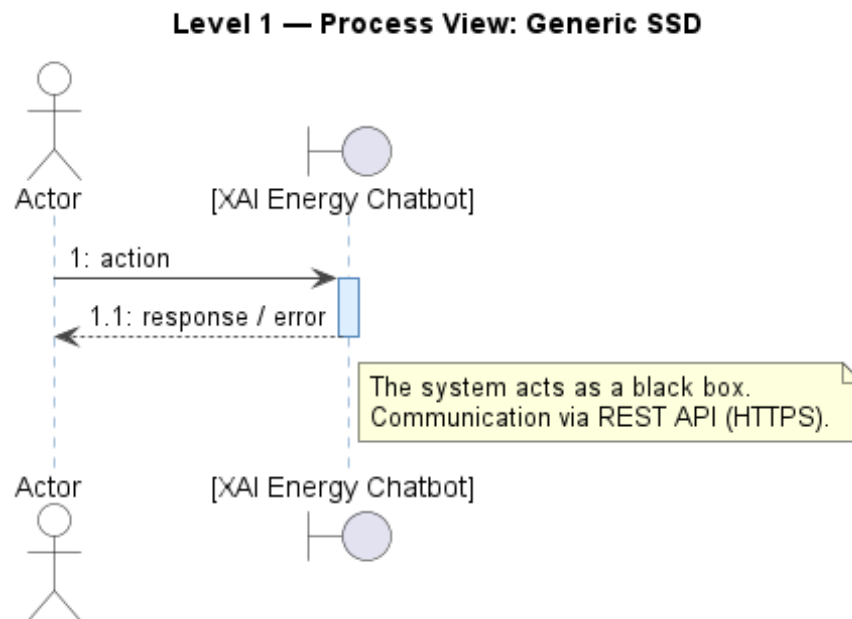


Figura 8: Vista de processo genérica de nível 1

3.5.2 Nível 2 - Vista de Contentores

A Figura 9 apresenta a vista lógica ao nível dos contentores, revelando a decomposição do sistema nos seus constituintes principais e as interligações entre eles. O *frontend* (Next.js) comunica com o *backend* (FastAPI) exclusivamente via REST HTTP com token Bearer. O *backend* integra dois servidores *PostgreSQL* distintos, a base de dados principal (utilizadores, conversas, mensagens, respostas e evidências) e o catálogo RAG (metadados de documentos e histórico de reindexações), bem como o *Qdrant* (*vector store* para *embeddings* RAG). Externamente, o *backend* comunica com o LM Studio (inferência LLM e *embeddings*), o ForeXAI (previsões e artefactos XAI via HTTP REST) e o fornecedor de identidade (LDAP).

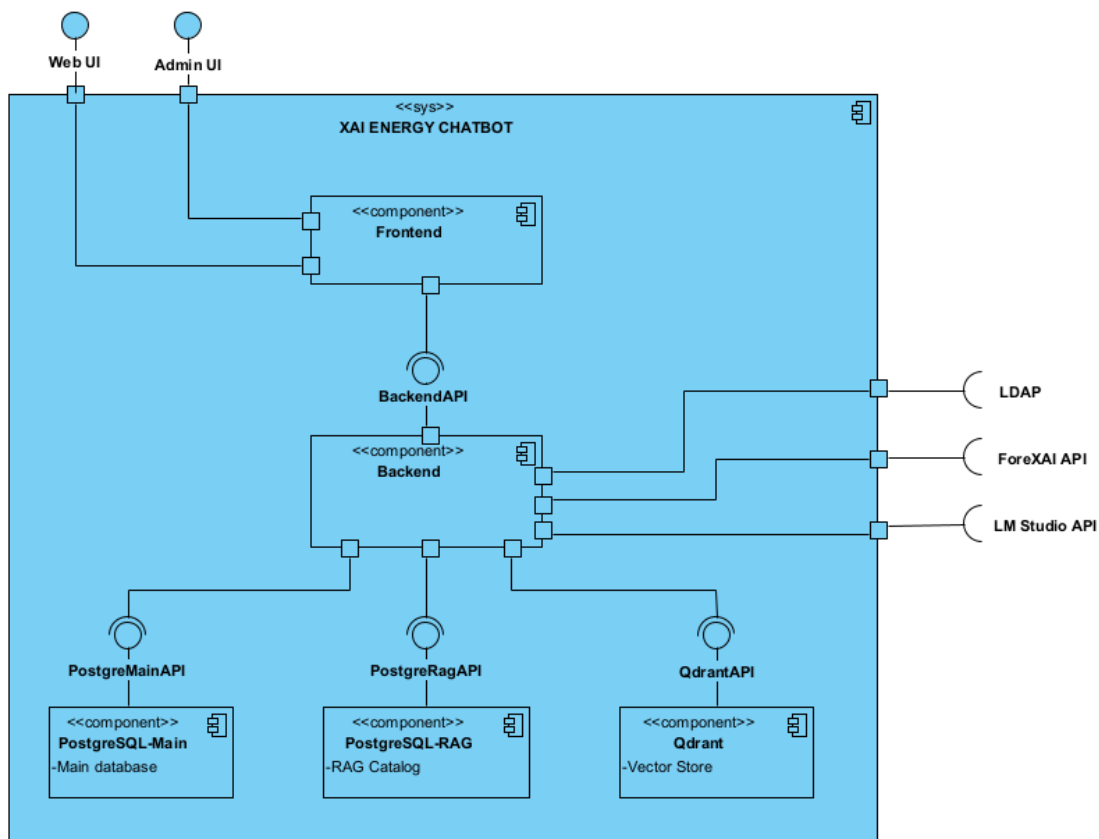


Figura 9: Vista lógica de nível 2

A separação entre a base de dados de catálogo RAG e o *Qdrant* face à base de dados principal determina que a persistência associada ao RAG se mantenha isolada dos dados conversacionais. Esta separação facilita a evolução independente do *corpus* documental sem afetar a integridade das sessões conversacionais.

A Figura 10 apresenta a vista de implementação dos contentores. As tecnologias adotadas foram *Next.js* no *frontend*, *FastAPI/Uvicorn* no *backend*, *PostgreSQL* nas duas bases de dados (principal e catálogo RAG) e *Qdrant* na base de dados vetorial.

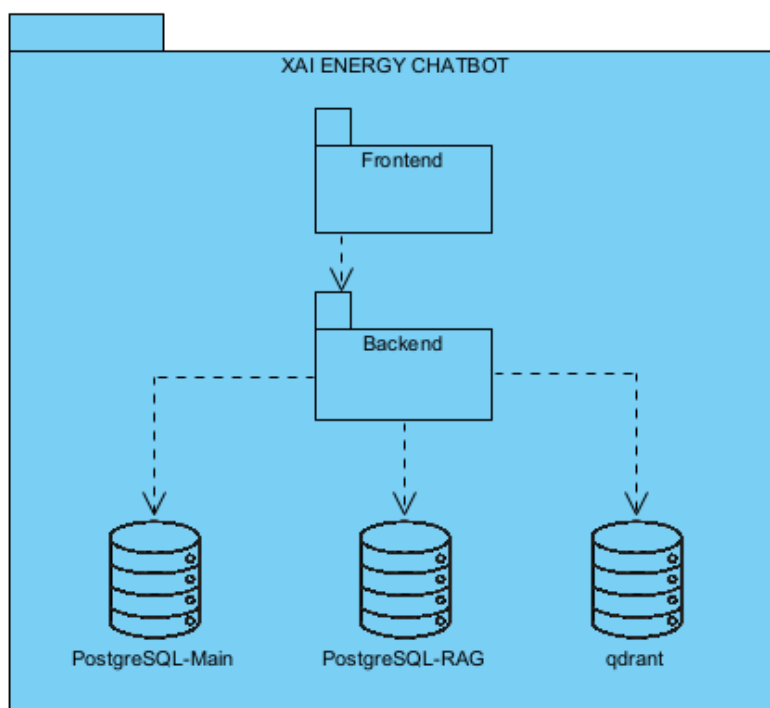


Figura 10: Vista de implementação de nível 2

A Figura 11 descreve a vista física, detalhando a topologia de *Docker Compose* e as portas expostas com *Next.js* na porta 3000, *FastAPI/Uvicorn* na porta 8005, *PostgreSQL (main)* na porta 5432, *PostgreSQL (RAG catalog)* na porta 5435, *Qdrant* na porta 6333 e *LM Studio* na porta 1234. Os contentores *Docker* são orquestrados por *Docker Compose* num único *host*, o *LM Studio* corre como processo nativo no mesmo *host*, externo ao *Compose*. O *ForeXAI* e o *LDAP* são servidores externos, acessíveis via rede.

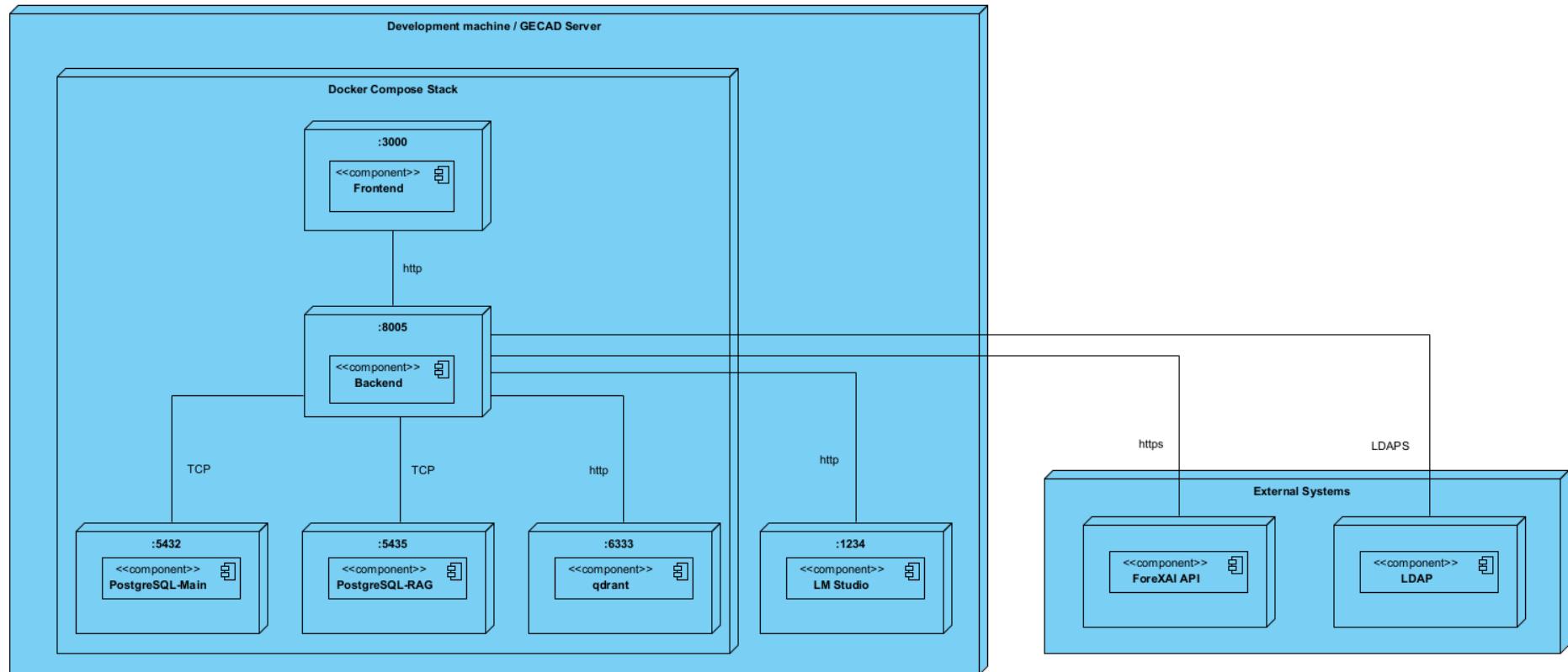


Figura 11: Vista física de nível 2

A Figura 12 apresenta o SSD genérico de nível 2, que ilustra o padrão de interação entre os contentores em que o ator interage com o *frontend*, que por sua vez contacta o *backend* via *REST HTTP* com *token Bearer*, e o *backend* acede ao armazenamento. Este padrão é transversal a todos os casos de uso.

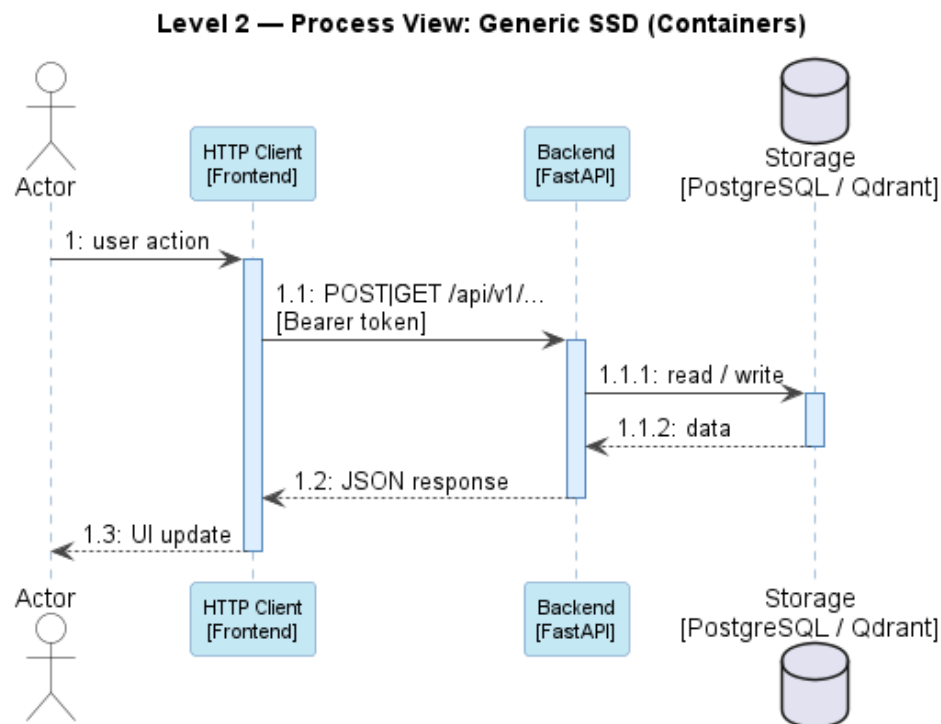


Figura 12: Vista de processo genérica de nível 2

A Figura 13 apresenta o SSD do fluxo do início de uma conversação e a Figura 14 apresenta envio de uma mensagem e obtenção da respetiva resposta, o mais representativo da complexidade da solução. Como se pode observar, o *backend* consulta sequencialmente o LM Studio (para o planeamento de *tools* e para a composição da resposta final), o *Qdrant* e o ForeXAI, antes de persistir a resposta e as evidências no *PostgreSQL*, evidenciando que o *backend* é o único contentor que coordena todos os subsistemas externos.

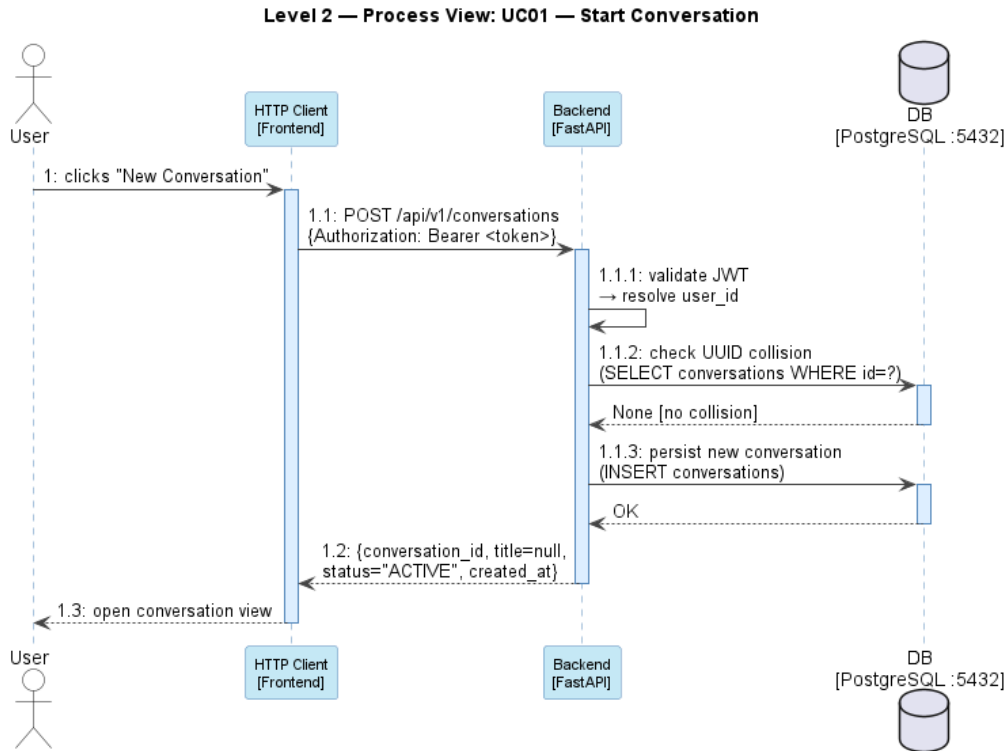


Figura 13: Vista de processo de nível 2 de início de conversação

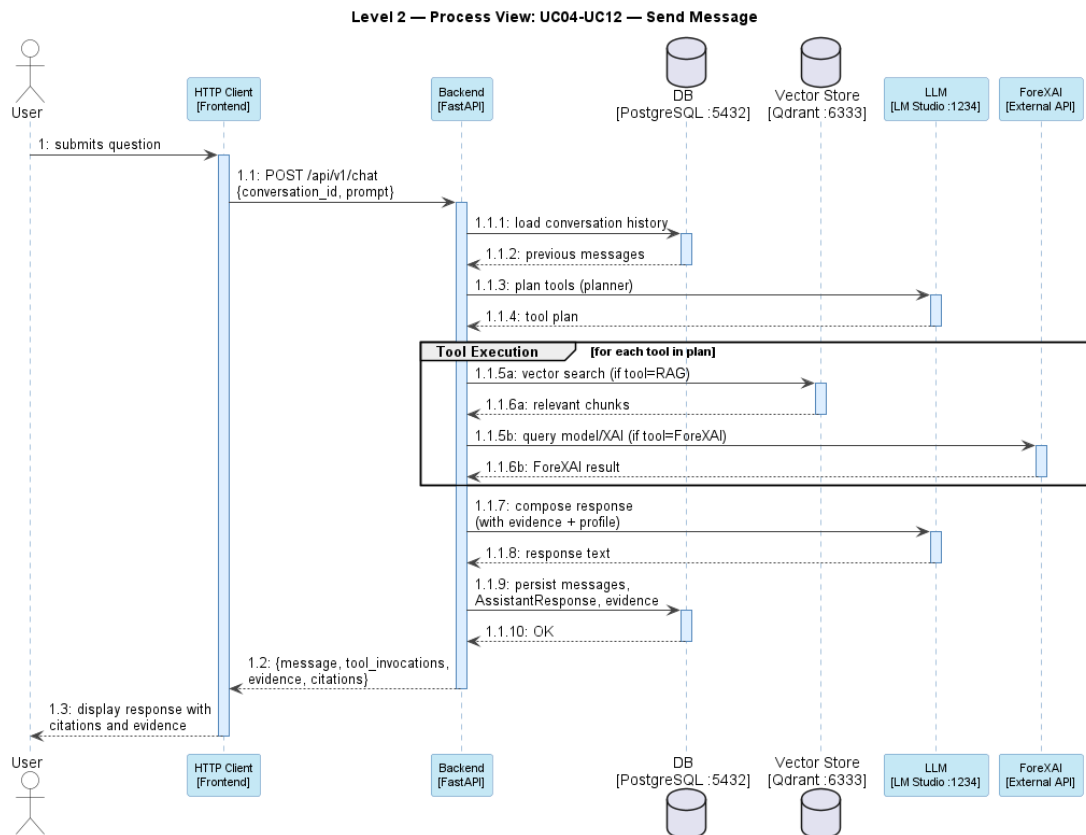


Figura 14: Vista de processo de nível 2 de envio de mensagem

Os SSDs de gestão do *corpus RAG* (Figura 15) e de autenticação (Figura 16) documentam os restantes fluxos relevantes neste nível. O fluxo de gestão *RAG* evidencia a coordenação entre o *backend*, o *Qdrant* e o catálogo *RAG* durante as operações de ingestão e reindexação documental. O fluxo de autenticação distingue dois modos, credenciais locais (consulta directa à base de dados principal apenas para o *Master*) e credenciais externas (validação via *LDAP*).

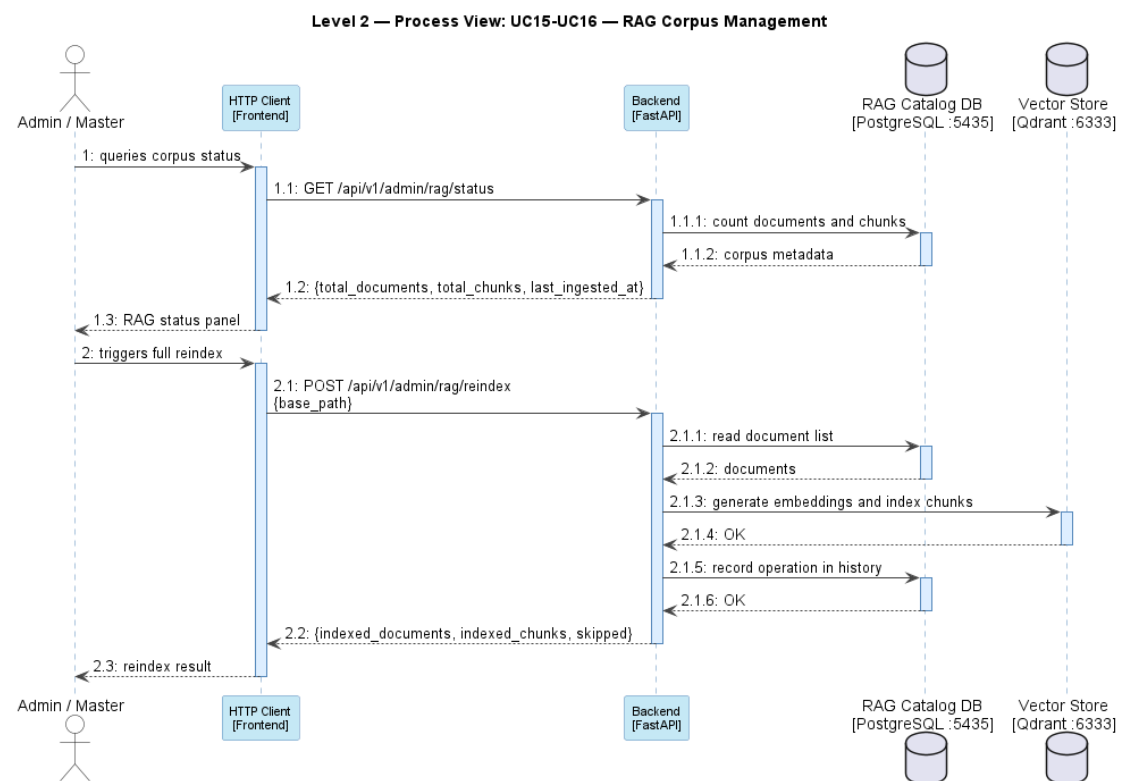


Figura 15: Vista de processo nível 2 de gestão do corpus do RAG

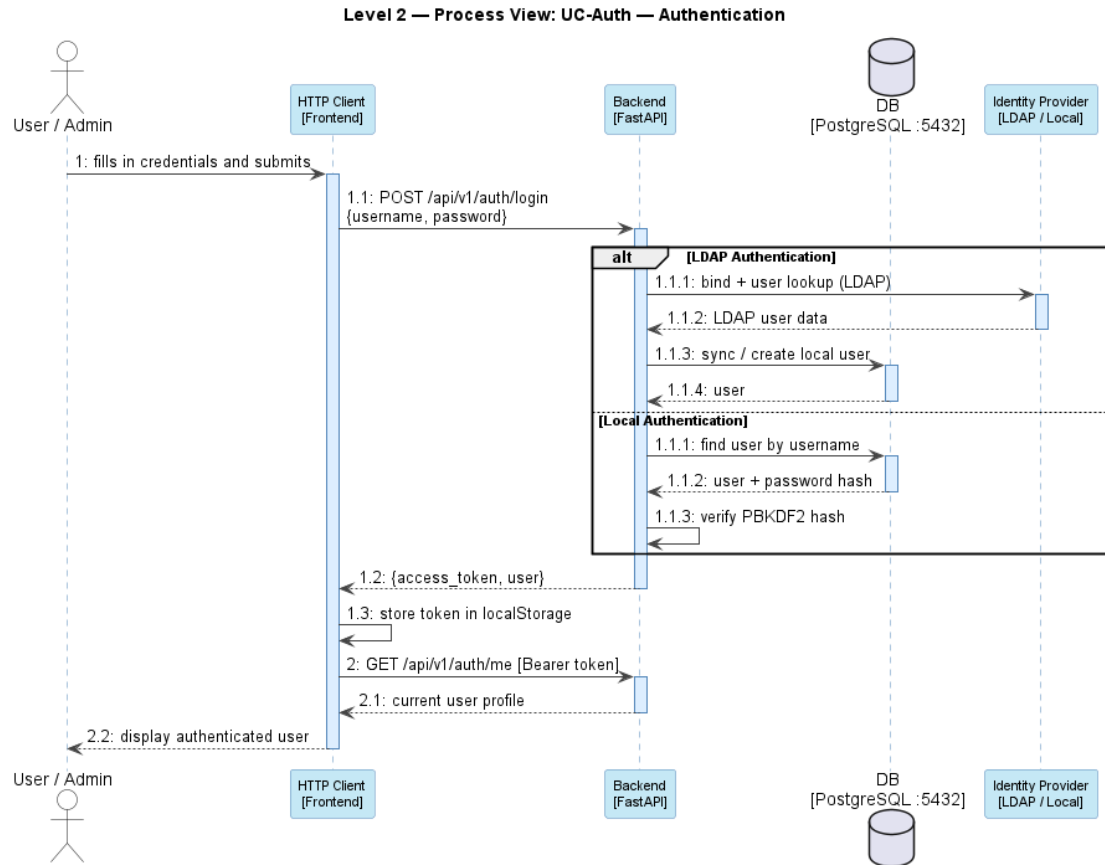


Figura 16: Vista de processo nível 2 da autenticação

3.5.3 Nível 3 - Vista de Componentes

O nível 3 desagrega o *backend* nos seus componentes internos, tornando explícita a arquitetura de código que suporta todos os casos de uso descritos. O sistema recorre a um conjunto de padrões de desenvolvimento complementares. O padrão *Controller* (Larman, 2004) delimita a fronteira HTTP, os *controllers* recebem os pedidos REST, validam os *schemas* de entrada e delegam imediatamente na camada de aplicação, sem conterem lógica de negócio. O padrão *Service* (Fowler, 2003) é aplicado com granularidade de ação, cada *service* encapsula a lógica de um único caso de uso (por exemplo *SendMessageService*, *LoginService*, *RunRAGIngestionService*), assegurando coesão funcional e facilitando os testes unitários. O padrão *Data Transfer Object* (DTO) formaliza os objetos de transferência entre a camada de API e a camada de aplicação, prevenindo a exposição direta das entidades de domínio. O padrão *Repository* (Fowler, 2003) abstrai o acesso à persistência, as *interfaces* residem na camada de domínio, enquanto as implementações concretas (*SQLAlchemy*) residem na infraestrutura, permitindo a substituição ou o teste independente da persistência. O padrão *Factory Method* (Gamma et al., 2016) é aplicado nas entidades de domínio, que expõem métodos de fábrica estáticos, como *Conversation.start_new()*,

`Message.create_user_message()`, `Message.create_assistant_message()` e `AssistantResponse.create_response()`, garantindo que os objetos são sempre criados num estado válido. O padrão *Adapter* (Gamma et al., 2016) isola os serviços externos (LLM, RAG, ForeXAI, fornecedor de identidade) atrás de interfaces de domínio, eliminando qualquer acoplamento direto entre a lógica de negócio e os detalhes técnicos de terceiros.

Estes padrões são organizados segundo uma *Clean Architecture* (Martin, 2018) de cinco camadas, representada na Figura 17, onde as dependências apontam sempre de fora para dentro, da infraestrutura para o domínio, nunca o inverso.

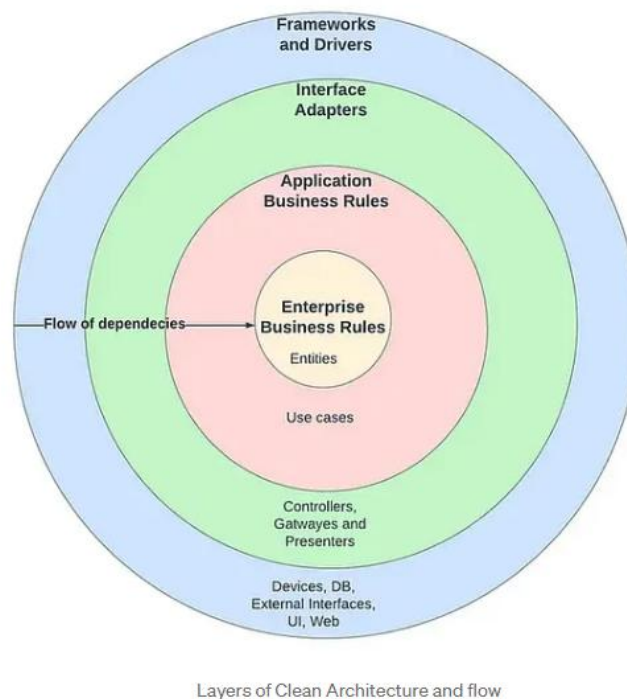


Figura 17: Representação da Clean Architecture e do seu fluxo¹⁸

A Figura 17 evidencia as cinco camadas que constituem o backend: *api/* (fronteira HTTP, *controllers* e *schemas*), *application/* (*services* de aplicação, DTOs e interfaces de integração), *domain/* (entidades, *value objects*, *enums*, exceções e interfaces de repositório - sem qualquer dependência de *frameworks*), *infrastructure/* (adaptadores concretos: *SQLAlchemy*, *QdrantAdapter*, *LMStudioAdapter*, *ForeXAIHTTPClient*, *LDAPIdentityProvider*) e *bootstrap/* (container de injeção de dependências e fábrica da aplicação *FastAPI*). Esta arquitetura concretiza os princípios SOLID: o *Single Responsibility Principle* (SRP) é assegurado pela separação de cada *service* por caso de uso; o *Open/Closed Principle* (OCP) permite

¹⁸ (Clean Architecture: Simplified and In-Depth Guide | Medium, 2026)

adicionar novos adaptadores sem modificar o código existente; o *Liskov Substitution Principle* (LSP) garante que qualquer implementação de repositório ou adaptador pode substituir a interface sem alterar o comportamento do sistema; o *Interface Segregation Principle* (ISP) mantém as interfaces específicas por entidade e por serviço externo; e o *Dependency Inversion Principle* (DIP) é o pilar central - a camada *bootstrap/* usa o *Dependency Injector* para compor o grafo completo de dependências em tempo de arranque, injetando as implementações concretas nas interfaces abstratas da camada de domínio.

A Figura 18 apresenta a vista lógica do *backend*, ilustrando os cinco módulos estruturais e as relações de dependência entre eles. Como se pode observar, a camada *domain/* não possui dependências para fora de si própria, confirmando o respeito pela regra de dependência da *Clean Architecture*; a camada *api/* depende de *application/*, que por sua vez depende de *domain/*; e a camada *infrastructure/* implementa as interfaces definidas em *domain/*, completando o ciclo de inversão de dependências.

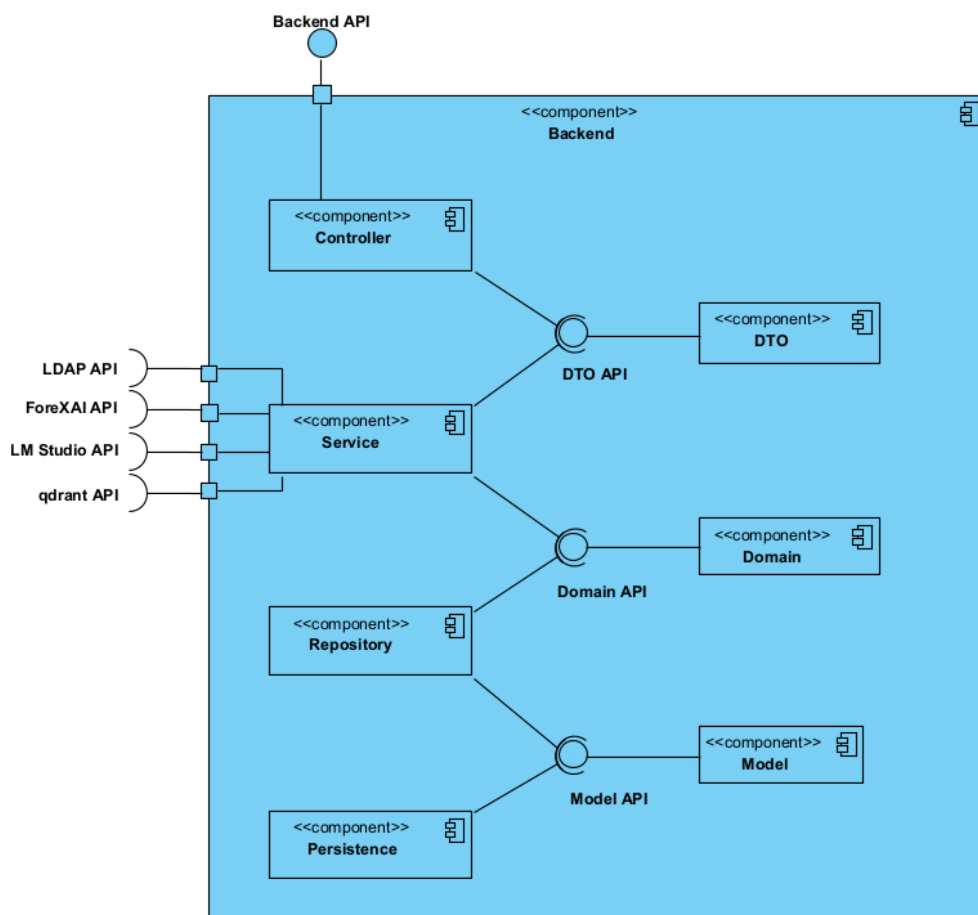


Figura 18: Vista lógica de nível 3

A Figura 19 apresenta a vista de implementação do *backend*, detalhando a estrutura de *packages* e as relações de dependência e realização entre eles. As setas de dependência

ligam as camadas superiores às inferiores, as setas de realização documentam a implementação das interfaces de domínio pelos adaptadores concretos, tornando explícito o padrão *Dependency Inversion* aplicado em toda a arquitetura.

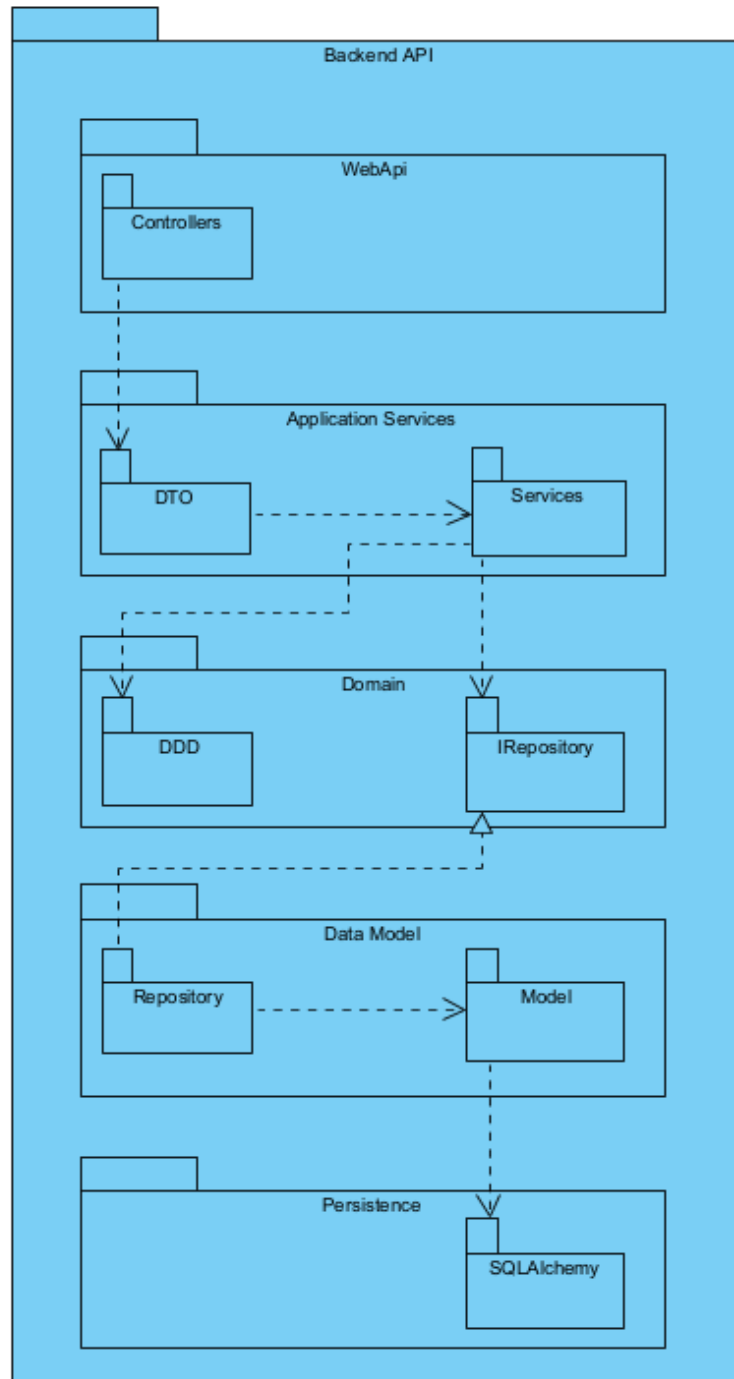


Figura 19: Vista de implementação de nível 3

O *System diagram* (SD) genérico de nível 3 (Figura 20) ilustra o fluxo padrão através de todas as camadas do *backend*: *Router*, *Controller*, *Service*, Entidade de Domínio, *Repository* e finalmente adaptador externo. Este padrão de delegação em cascata, onde cada camada

apenas conhece a interface da camada imediatamente inferior, é aplicado de forma consistente em todos os casos de uso.

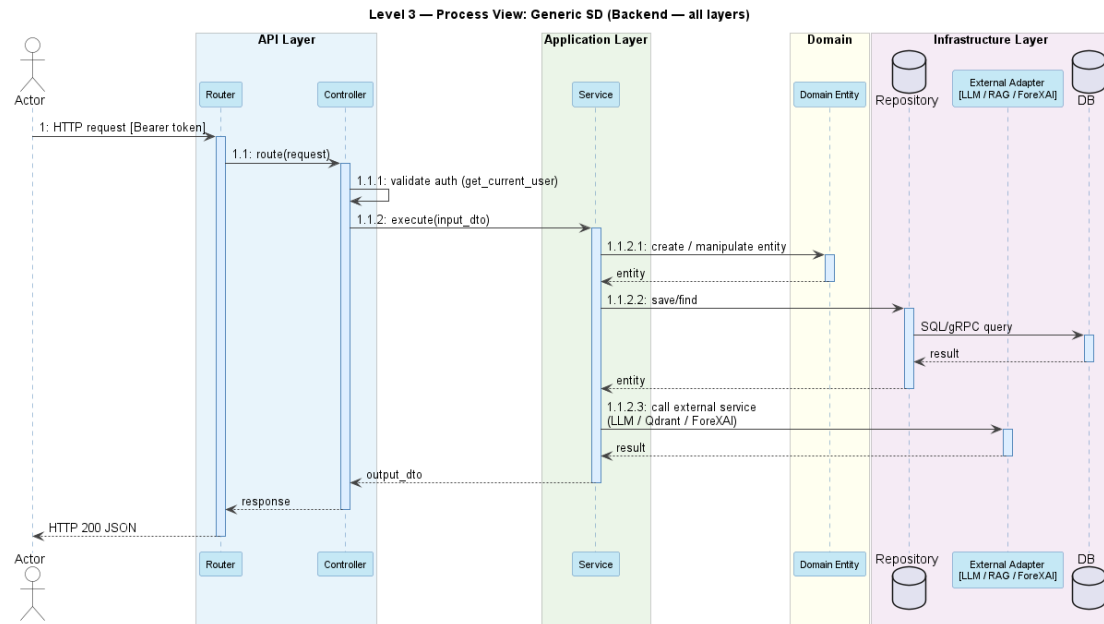


Figura 20: Vista de processo genérica de nível 3

A Figura 21 apresenta o SD do UC01 (Iniciar Conversa), que documenta a criação de uma nova sessão conversacional. O fluxo inclui a chamada do método de fábrica *Conversation.start_new()* na entidade de domínio, a verificação de colisão de *UUID* e a persistência via repositório, ilustrando a aplicação do padrão *Factory Method* neste caso de uso.

A Figura 22 apresenta o SD detalhado do UC04, documentando o fluxo completo de envio de mensagem. O *ChatController* delega no *SendMessageService*, que coordena o *SelectToolsService* (planeamento via LM Studio), o *ExpandToolWorkflowsService* (expansão de *workflows* predefinidos), o *GroundingPolicyService* (avaliação pré e pós execução das *tools*), o *ExecuteToolPlanService* (execução das *tools*, RAG via *Qdrant* ou *ForeXAI* via HTTP) e o *ResponseComposerAdapter* (geração da resposta final pelo LM Studio). O resultado é persistido no *PostgreSQL* através dos repositórios *SQLAlchemy*.

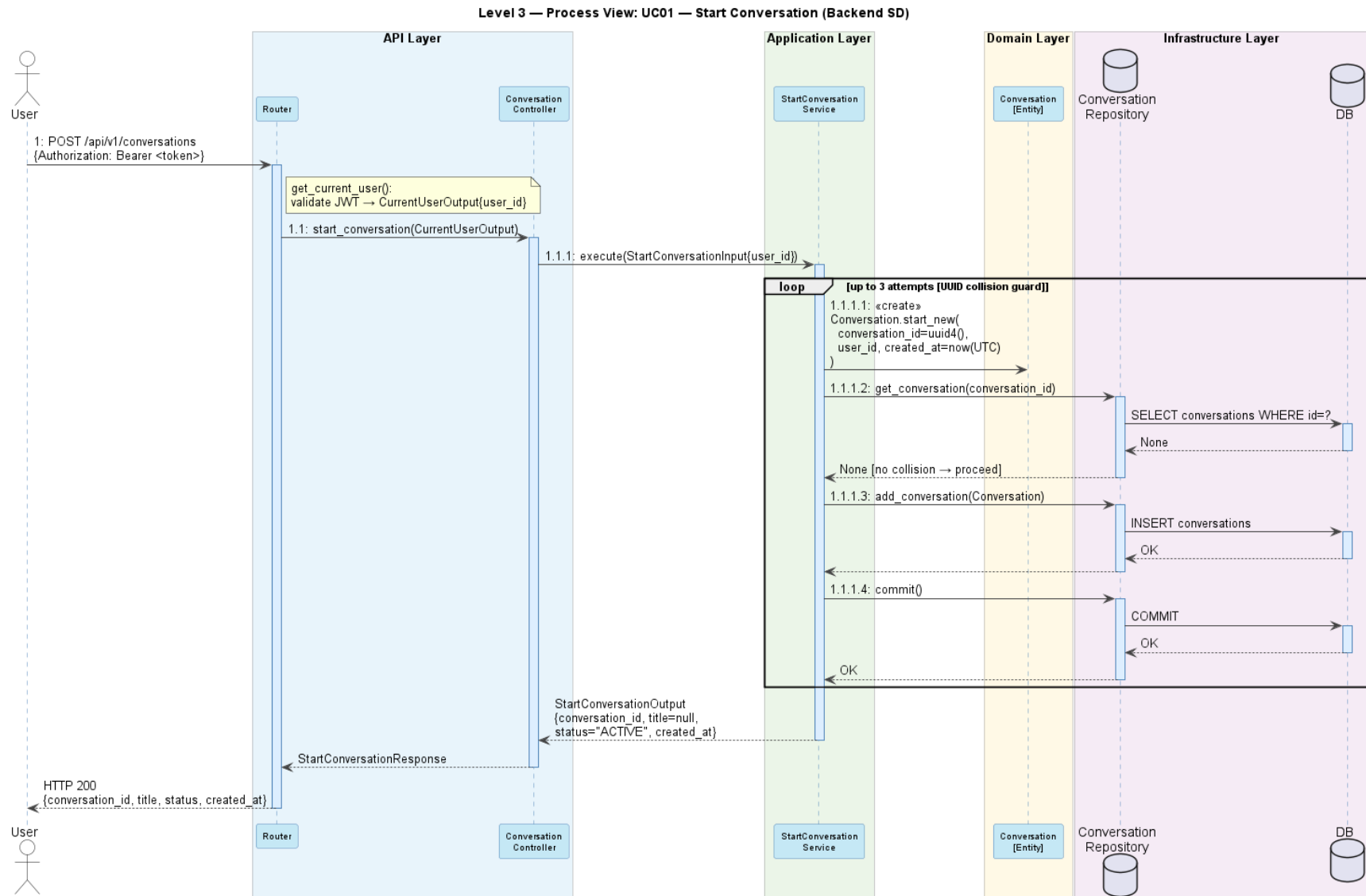


Figura 21: Vista de processo nível 3 de início de conversação

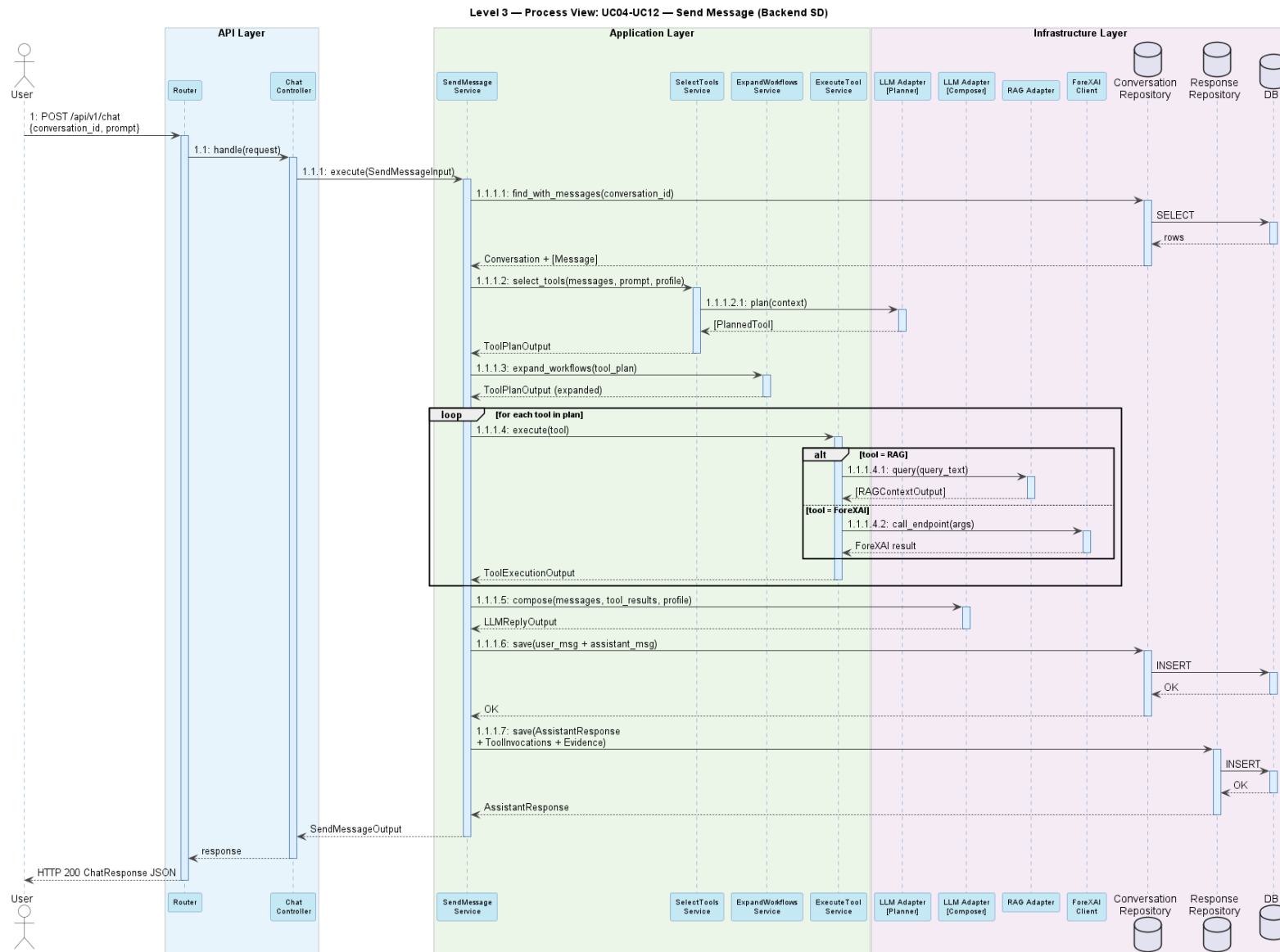


Figura 22: Vista de processo de nível 3 de envio de mensagem

A Figura 23 apresenta o SD da reindexação do *corpus* RAG (UC15), documentando o fluxo de gestão documental onde o *AdminRAGController* delega no *RunRAGIngestionService*, que coordena a leitura de documentos, o *chunking* semântico em *Markdown*, a geração de *embeddings* via *LM Studio* e a indexação no *Qdrant*, com registo no catálogo RAG.

Por fim, e mesmo o *design* do *frontend* não sendo o foco deste relatório, de forma a ajudar à compreensão do funcionamento de todos os componentes do sistema em conjunto, apresentamos o SD do *frontend* para o envio de mensagem (Figura 24) onde documenta a interação entre a *ChatPage*, o *hook useChatSession*, o *chatApiService* e o *backend*, evidenciando a separação entre a lógica de estado (*hook*) e comunicação HTTP (serviço de API).

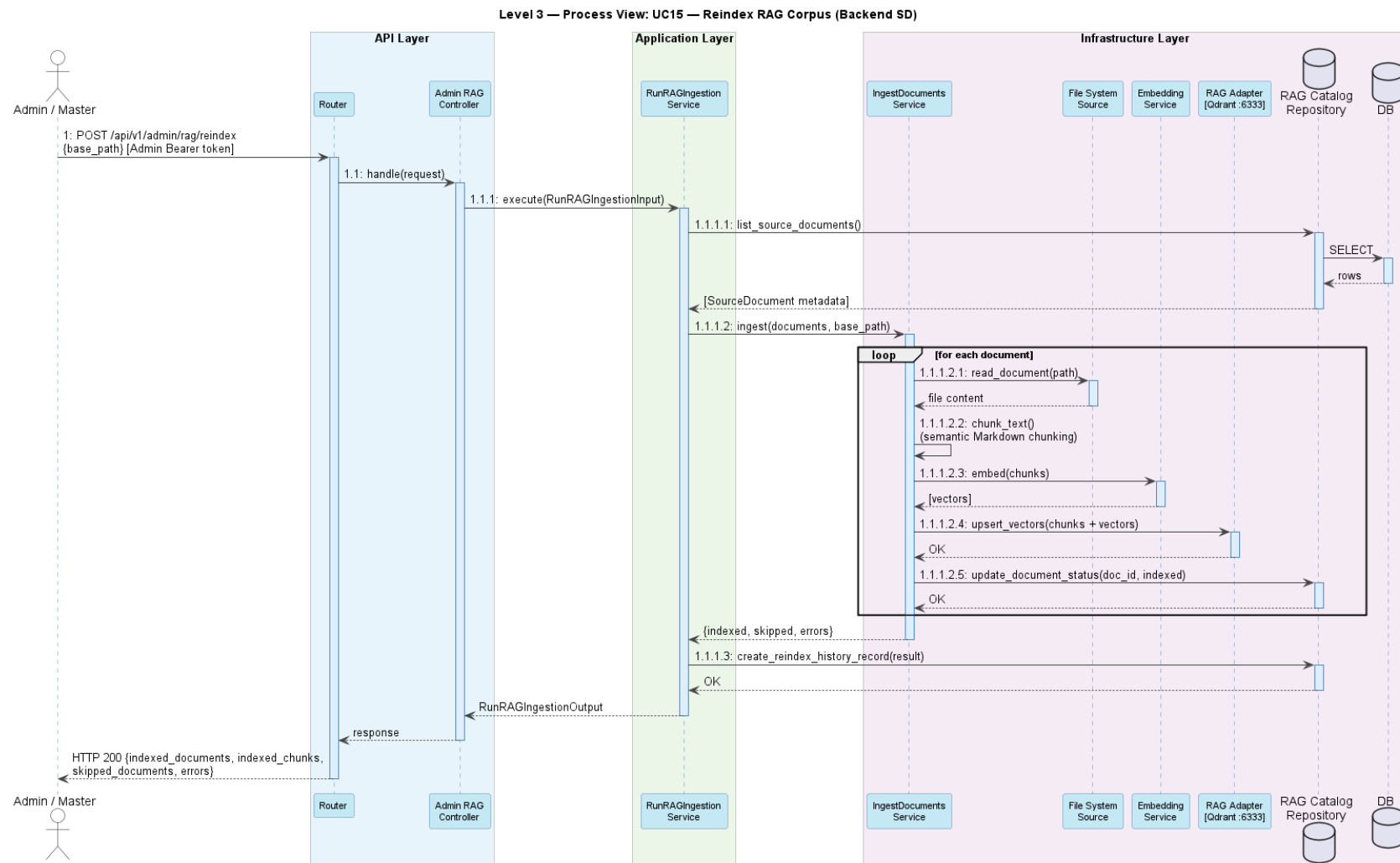


Figura 23: Vista de processo nível 3 de reindexação do corpus do RAG

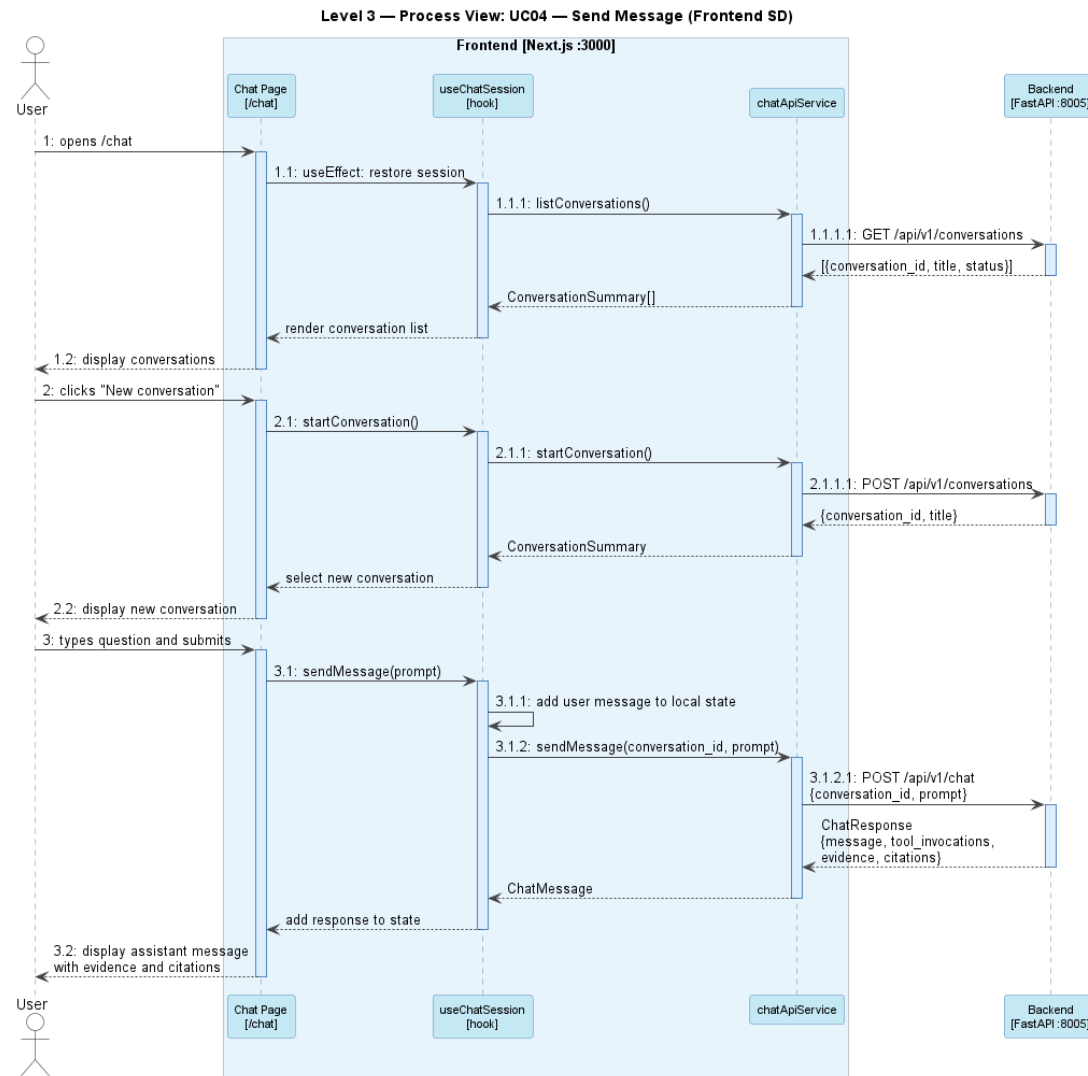


Figura 24: Vista de processo nível 3 do envio de mensagem no frontend

3.5.4 Design alternativo

Durante a fase de desenho foi considerada uma alternativa arquitetural baseada numa decomposição em serviços independentes. Nesta proposta, as principais capacidades do sistema seriam expostas como serviços autónomos, um serviço de autenticação, um serviço RAG, um serviço de orquestração LLM/*tools* e um proxy ForeXAI, coordenados por um API Gateway comum. Cada serviço seria implementado em contentor próprio.

Esta arquitetura constituiria a opção mais indicada num cenário de expansão de capacidade de negócio, permitiria escalar independentemente cada componente conforme a carga, por exemplo, replicar o serviço de ingestão RAG durante picos de indexação documental sem aumentar as instâncias de orquestração, evoluir cada serviço com tecnologias ou modelos distintos, e distribuir o desenvolvimento por equipas autónomas. A separação de responsabilidades ao nível de implementação traduz-se também numa maior resiliência operacional, dado que a falha de um serviço não compromete necessariamente a disponibilidade dos restantes.

Contudo, o pipeline de orquestração síncrono *planner, validator, executor, responder* exige coordenação estreita entre componentes no âmbito de um único pedido conversacional. Distribuído por microserviços introduziria latência de comunicação inter-serviço acumulada num fluxo que já envolve múltiplas chamadas ao LLM. Adicionalmente, o servidor de inferência LM Studio é externo e não contentorizado e representa por isso o gargalo real de desempenho, que nenhuma das arquiteturas elimina por si só.

Tendo em conta o objetivo do projeto e o nível de utilização esperado, sem requisitos de escalabilidade horizontal independente por componente, foi decidido abdicar do design alternativo e implementar o sistema de acordo com a *Clean Architecture* apresentada num *backend* monolítico, focado na injeção de dependências. Esta escolha garante a testabilidade e substituíbilidade dos adaptadores sem a sobrecarga operacional de uma arquitetura distribuída, sendo o design mais adequado ao âmbito e dimensão atuais do projeto. Caso os requisitos de escala ou de expansão de capacidade de negócio evoluam no futuro, a decomposição em serviços independentes constituiria a evolução arquitetural mais natural a partir da base implementada.

4 Implementação da Solução

Esta secção detalha as decisões de implementação do *backend*, do *frontend* e da infraestrutura de contentores do *XAI Energy Chatbot*. A exposição está organizada em quatro momentos: a fundamentação da escolha da tecnologia face à análise comparativa realizada no capítulo 2, a descrição das principais componentes de implementação, a caracterização da estratégia de testes adotada e a avaliação da solução face aos requisitos funcionais e não funcionais definidos na secção anterior.

4.1 Escolha tecnológica

As escolhas tecnológicas assentam em três prioridades fundamentais, derivadas diretamente dos critérios de comparação analisados no Capítulo 2:

1. Privacidade e controlo institucional - LM Studio com modelo *open-source* executado localmente e infraestrutura containerizada própria, em vez de APIs comerciais que implicariam o envio de dados para fornecedores externos.
2. Coerência arquitetural - *Python* e *FastAPI* como camada de orquestração central, *React* como camada de apresentação pura, e *tool calling* como mecanismo de integração com o ForeXAI e o RAG, assegurando que o *frontend* nunca acede diretamente ao LLM nem a serviços externos.
3. Modularidade e substituíbilidade - a injeção de dependências e o padrão adaptador permitem substituir qualquer componente (LLM, *vector store* ou serviço XAI) sem alterar a lógica de negócio do sistema.

4.2 Descrição da implementação

A implementação concretiza a arquitetura descrita na secção anterior em cinco domínios complementares: configuração declarativa do sistema, orquestração de *tools* com política de *grounding*, integração dos serviços externos (RAG, ForeXAI e autenticação), interface gráfica e infraestrutura de contentores.

4.2.1 Configuração do sistema

O comportamento do sistema é governado por **quatro ficheiros de configuração** declarativos localizados em *backend/configs/*, que permitem alterar o repertório de ferramentas, os perfis de utilizador e os *prompts* de sistema sem modificar código.

O ficheiro *tools.json* define o conjunto de *tools* disponíveis ao *planner* LLM. Estão definidas 19 *tools* atómicas de integração e 6 *workflows* predefinidos, num total de 25 entradas, podendo ser consultado um excerto do código no ANEXO C. As *tools* atómicas organizam-se em três categorias:

- rag (1 tool) - pesquisa vetorial no *Qdrant* a partir de uma *query* textual
- Ferramentas ForeXAI individuais (15 *tools*) - permitem consultar metadados de modelos (*forexai_get_model*, *forexai_get_model_info*, *forexai_list_models*), conjuntos de dados (*forexai_get_dataset*), métricas e erros de avaliação (*forexai_get_model_metrics*, *forexai_get_model_errors*), previsões históricas (*forexai_get_model_forecasts*) e seis tipos de explicabilidade XAI, SHAP global, SHAP local, LIME, Permutation Importance, PDP, ALE, Anchors e Counterfactuals;
- Ferramentas de agregação (3 *tools*) - *forexai_compare_models_by_metric* e *forexai_get_all_models_metrics* (comparação de métricas entre todos os modelos) e *forexai_get_best_historical_forecast* (identificação do melhor modelo para um ponto histórico específico).

Os 6 *workflows* predefinidos na Tabela 10 compõem sequências de *tools* atómicas para responder a perguntas recorrentes de forma coordenada:

Tabela 10: Workflows predefinidos

Workflow	Tools encadeadas	Finalidade
workflow_model_overview	1.get_model, 2.get_model_info, 3.get_model_metrics	Resumo completo de um modelo
workflow_model_quality	1.get_model_metrics, 2.get_model_forecasts, 3.get_model_errors	Avaliação de qualidade do modelo
workflow_global_xai	1.get_model, 2.get_model_metrics, 3.xai_shap_global	Explicabilidade global com SHAP
workflow_local_xai	1.get_model, 2.xai_shap_local	Explicabilidade local de uma instância
workflow_best_historical_forecast	1.get_dataset, 2.get_best_historical_forecast	Melhor previsão histórica num ponto
workflow_best_historical_forecast_with_xai	1.get_dataset, 2.get_best_historical_forecast, 3.xai_shap_local	Melhor previsão histórica + XAI local

O *ExpandToolWorkflowsService* substitui automaticamente qualquer *tool* de tipo *workflow* pelos seus passos atómicos antes da execução, tornando os *workflows* transparentes ao *executor*.

O ficheiro *profiles.json* define 4 perfis de utilizador que condicionam o estilo da resposta gerada pelo LLM, e pode ser consultado um excerto no ANEXO D. O perfil por defeito é **COMMON_USER**. Cada perfil especifica diretrizes de resposta (*response_guidance*) e uma mensagem de evidência insuficiente:

Tabela 11: Perfis de utilizador e orientações de resposta

Perfil	Orientação de resposta
COMMON_USER	Linguagem simples, definição prévia de termos técnicos, analogias, sem detalhe de implementação
ENERGY_SPECIALIST	Terminologia energética, dados quantitativos (métricas, intervalos de previsão), incertezas operacionais
ML_SPECIALIST	Precisão técnica, nomes exatos de métricas e métodos, distinção XAI global vs. local, sem simplificação de <i>trade-offs</i>
ADMINISTRATOR	Rastreabilidade explícita, limites de evidência, lacunas de dados, estrutura orientada para auditoria

O ficheiro *chat_prompts.json* contém as instruções de sistema do LLM, incluindo a instrução de língua por defeito e as diretrizes de comportamento geral do assistente, conforme se pode verificar no ANEXO E. Este ficheiro é o único ponto de configuração dos *prompts* de sistema, nenhum *prompt* está embebido no código da aplicação.

O ficheiro *rag_retrieval.json* configura os parâmetros do algoritmo de *overlap* lexical utilizado pelo *GroundingPolicyService* na filtragem pós-execução: define a lista de *stopwords* a ignorar no cálculo (preposições, artigos e conjunções em português e inglês) e um dicionário de sinónimos e variantes terminológicas (*term_aliases*) que permite equiparar termos tecnicamente equivalentes, por exemplo *shap* e *shapley*, durante a comparação lexical com a *query* do utilizador.

4.2.2 Orquestração de ferramentas

O fluxo de processamento de uma mensagem segue o pipeline *planner*, *validator*, *executor* e *responder*, que se traduz em planeamento, validação, execução e composição implementado no *SendMessageService* e nos serviços que este coordena.

Na fase de planeamento o *SelectToolsService* constrói um *prompt* de planeamento com as instruções do sistema (definidas em *chat_prompts.json*), a descrição de todas as *tools* (com as *tools* e *workflows*), e as últimas seis mensagens do historial, conforme apresentado no ANEXO G. Este *prompt* é submetido ao LM Studio via *ToolPlannerAdapter*. O LLM devolve um plano JSON num de dois modos, *no_tool* (nenhuma ferramenta necessária) ou *use_tools* (lista de *tools* com os respectivos argumentos).

Na validação o *ExecuteToolPlanService* valida cada *tool* do plano antes de qualquer execução. Para cada entrada verifica se a *tool* está registada, verifica a presença dos argumentos obrigatórios, alternativos ou condicionais e normaliza os valores de todos os argumentos (remoção de espaços). As *tools* que não passam a validação são excluídas do plano de execução e geram registos de erro estruturados (*ToolExecutionOutput*), que fazem parte do contexto final, mas não abortam o *pipeline*, as ferramentas válidas restantes são executadas.

A Política de *grounding* pré-execução, *GroundingPolicyService.enforce_tool_usage()*, atua exclusivamente quando o *planner* devolveu o modo *no_tool*, ou seja, quando o LLM decidiu que nenhuma ferramenta era necessária. Nesse caso, o serviço substitui o plano vazio por um único passo *rag* com a *query* do utilizador, garantindo que existe sempre pelo menos uma consulta documental antes de gerar a resposta. Se o *planner* já tiver devolvido o modo *use_tools*, ainda que sem *rag*, o plano transita sem alterações para a fase seguinte como se pode observar no ANEXO H.

A Expansão de workflows, *ExpandToolWorkflowsService*, percorre cada entrada do plano validado transitando as *tools* atômicas sem alterações e substituindo as entradas do tipo *workflow* pelos seus passos concretos. Em cada passo, os argumentos necessários predefinidos são preenchidos dinamicamente a partir dos argumentos fornecidos pelo *planner*. Além disso, cada passo expandido recebe a chave adicional *workflow_name* com o nome do *workflow* de origem, preservando a rastreabilidade nas evidências e nos registos de execução.

A execução processa-se no *ExecuteToolPlanService* que executa cada *tool* do plano expandido em sequência conforme apresentado no ANEXO I. A *tool rag* é tratada pelo *RAGToolHandler*, que delega na interface *RAGInterface* (implementada pelo *QdrantAdapter*) para realizar a pesquisa vetorial. As *tools forexai* são encaminhadas ao *handler* correspondente, que invoca o *ForeXAIHTTPClient* via HTTP REST. Os *workflows* que referenciam resultados de passos anteriores (dependências dinâmicas entre passos) são executados de forma sequencial pelo *execute_dynamic_workflow*. Cada execução produz um *ToolExecutionOutput* com os contextos RAG, as evidências e os eventuais erros.

Na Política de *grounding* pós-execução o *GroundingPolicyService.filter_tool_execution_results()*, apresentado no ANEXO J, filtra os *chunks* RAG de cada resultado de execução segundo uma lógica de três limiares:

- 1) se o *score* final for inferior a 0,55 (*min_final_score*), o *chunk* é sempre rejeitado;

- 2) se o *score* final for igual ou superior a 0,55 e o *score* de *overlap* lexical for igual ou superior a 0,15 (*min_lexical_overlap_score*), o *chunk* é aceite, pois garante-se assim que existe correlação terminológica com a *query* do utilizador;
- 3) se o *score* final for igual ou superior a 0,80 (*high_confidence_final_score*), o *chunk* é aceite independentemente do *overlap* lexical, assegurando que contextos de elevada similaridade vetorial não são descartados por ausência de sobreposição lexical exata.

As evidências ForeXAI transitam esta fase sem filtragem. Após a filtragem, *GroundingPolicyService.evaluate()* verifica se subsistem contextos RAG fundamentados ou evidências de *tool*, se não existir nenhuma devolve *can_answer=False*.

Na fase de composição se *can_answer=False*, o *SendMessageService* devolve diretamente a resposta de evidência insuficiente configurada para o perfil ativo em *profiles.json*, conforme apresentado no ANEXO K. Caso contrário, a função *build_llm_prompt()* constrói o *prompt* de resposta com: as instruções de sistema do *chat_prompts.json*, a orientação específica do perfil (*profiles.json*), a questão do utilizador e todos os blocos de evidência (contextos RAG e evidências ForeXAI).

4.2.3 Integração RAG

O módulo RAG é responsável pela ingestão incremental, indexação vetorial e pesquisa híbrida de documentação validada sobre modelos de energia. A ingestão é desencadeada pelo *endpoint POST /api/v1/admin/rag/reindex* e coordenada pelo *RunRAGIngestionService*.

Inicia pela leitura de documentos percorrendo recursivamente a pasta configurada em *RAG_SOURCE_DOCS_PATH*, suportando os formatos *.md*, *.txt*, *.rst* e *.pdf*. É efetuada a deteção de alterações para cada documento calculando um *hash* que combina o *source_id*, o conteúdo, o tipo de conteúdo e as chaves de configuração do *chunker* e do *vector store*, documentos cujo *hash* não mudou em relação ao catálogo são ignorados, tornando a reindexação incremental e eficiente. Depois segue-se a segmentação semântica em que o *TextChunker* usa estratégias distintas por tipo de conteúdo, por exemplo para *Markdown*, deteta a hierarquia de cabeçalhos (*#*, *##*, *###*) e preserva-a no conteúdo de cada *chunk* (ex.: *Nível1 > Nível2 > Nível3*), trata blocos de código, tabelas e listas como unidades atómicas, e agrupa blocos até *RAG_CHUNK_SIZE=1000* caracteres com sobreposição de *RAG_CHUNK_OVERLAP=150*. Seguidamente é feita a indexação vetorial onde os *embeddings* de cada *chunk* são gerados pelo *LMStudioEmbeddingAdapter* com o modelo configurado em *RAG_EMBEDDING_MODEL* e carregados no *Qdrant* com um *id* próprio, garantindo

reprodutibilidade entre reindexações. Também é feito o registo no catálogo que consiste na persistência dos metadados de cada documento. Este fluxo pode ser consultado no ANEXO L.

Durante a inferência, o *QdrantRAGQueryService* executa um pipeline de recuperação híbrido em quatro fases que pode ser consultado no ANEXO M. Na primeira fase, a *query* é convertida em *embedding* e são recuperados os $RAG_MAX_DOCUMENTS \times 4$ candidatos com *score* vetorial mínimo de RAG_MIN_SCORE . Na segunda fase, os *chunks* duplicados (mesmo par *source_id* + excerto) são eliminados e os excertos são truncados a $RAG_MAX_EXCERPT_CHARS$ caracteres. Na terceira fase, é calculado um *score* de *overlap* lexical entre os termos da *query* (com três ou mais caracteres, excluindo *stopwords* e expandidos via *term_aliases*) e o conteúdo de cada *chunk*, a pontuação combinada é dada por $combined_score = vector_score + lexical_overlap \times 0,08$. Na quarta fase, é aplicada uma filtragem relativa, se $score \geq top_score \times 0,75$ e $top_score - score \leq 0,25$, antes de serem devolvidos os $RAG_MAX_DOCUMENTS$ melhores resultados. O *GroundingPolicyService* aplica os limiares de qualidade (descritos na secção 4.1.2) sobre estes resultados antes de os incluir no contexto de composição.

4.2.4 Integração ForeXAI

O *ForeXAIHTTPClient* implementa um cliente HTTP REST consultivo sobre a API do ForeXAI, acessível na URL configurada em $FOREXAI_BASE_URL$. A integração é exclusivamente de leitura, a aplicação consulta modelos, métricas, previsões e artefactos XAI. Pressupõem-se que o modelo carregado no LM Studio suporte capacidades de visão pois modelos puramente textuais ignorarão as imagens e gerarão respostas sem interpretação visual dos artefactos XAI.

Os *endpoints* de consulta existentes permitem listagem de modelos, metadados e informação do modelo, metadados do dataset, métricas de avaliação e análise de erros, previsões históricas e melhor previsão histórica para um ponto. O grupo de explicabilidade XAI suporta atualmente oito métodos: *shap global*, *shap local*, *LIME*, *permutation*, *PDP*, *ALE*, *anchors* e *counterfactuals*. A escolha do método XAI em cada interação é responsabilidade do *planner* LLM, que seleciona as *tools* adequadas em função da pergunta do utilizador. Cada resposta da API é mapeada para um *ForeXAIResultOutput*.

4.2.5 Autenticação

A autenticação implementada (Figura 25) seleciona o fornecedor de identidade por uma lógica de prioridade local, se o utilizador existir na base de dados principal com *password hash* armazenado, a validação é delegada no *LocalIdentityProvider* independentemente de $AUTH_PROVIDER$, caso contrário, a delegação passa ao *LDAPIdentityProvider*. Esta lógica

permite que utilizadores criados em *bootstrap* (configurados em `BOOTSTRAP_LOCAL_USERS`) coexistam com uma organização LDAP sem conflitos de fornecedor.

Na primeira autenticação bem-sucedida de um utilizador LDAP, é criado automaticamente um registo local na base de dados com perfil `COMMON_USER` e *role* `USER`. Nas autenticações seguintes, o registo local é reutilizado, o que permite gerir ativamente o perfil e o *role* sem dependência contínua do diretório para consultas de autorização.

Após a autenticação, o *LocalTokenService* emite um *token HMAC-SHA256* com os campos *sub* (identificador do utilizador) e *exp* (timestamp de expiração), assinado com o segredo configurado em `AUTH_SECRET_KEY`. O *token* é incluído em todas as chamadas do *frontend* ao *backend* no cabeçalho *Authorization: Bearer*.

Para desenvolvimento e testes, `LDAP_MODE=test` ativa um contentor Samba AD DC pré-configurado com os utilizadores definidos em `BOOTSTRAP_LDAP_TEST_USERS`, replicando o comportamento esperado sem a dependência de infraestrutura real LDAP.

LEI-PROJ 2025/2026

GECAD

Sistema Conversacional baseado em LLM para Transparência e Explicabilidade em Modelos de Energia
Inicia sessão para aceder ao assistente conversacional.

Entrar no sistema

Inicia sessão com as tuas credenciais para aceder ao sistema com o teu perfil e permissões.

Username

Password

A sessão anterior expirou. Volta a autenticar-te para continuar.

Entrar

Contas de demonstração

- common_demo / CommonDemo123!
- energy_demo / EnergyDemo123!
- ml_demo / MLDemo123!
- admin_demo / AdminDemo123!
- master / Master123!

Figura 25: Interface gráfica da autenticação

4.2.6 Interface gráfica

O *frontend*, que dispõe de um conjunto alargado de imagens no ANEXO N para consulta, é uma aplicação *Next.js* organizada por *features* (*src/features*). As rotas principais são `/chat` (interface conversacional, acessível a todos os utilizadores autenticados) e `/admin` (painel de administração, restrito a utilizadores com *role* `ADMIN` ou `MASTER`).

A *feature chat* (Figura 26) assenta no *hook useChatSession*, que centraliza o estado conversacional: lista de conversas ordenadas por data de criação, identificador da conversa ativa, lista de mensagens com as respetivas *tool invocations* e evidências, *draft* da mensagem em edição, estado de carregamento e mensagem de estado visível ao utilizador. As mensagens do utilizador são adicionadas otimisticamente ao estado antes de a resposta do *backend* chegar e são removidas em caso de erro, garantindo uma experiência de resposta imediata. O *chatApiService* encapsula as sete chamadas HTTP ao *backend* utilizadas pelo *hook*: *listConversations*, *startConversation*, *getConversationHistory*, *sendMessage*, *updateConversationTitle*, *archiveConversation* e *deleteConversation*, todas autenticadas com cabeçalho *Authorization: Bearer {token}*.

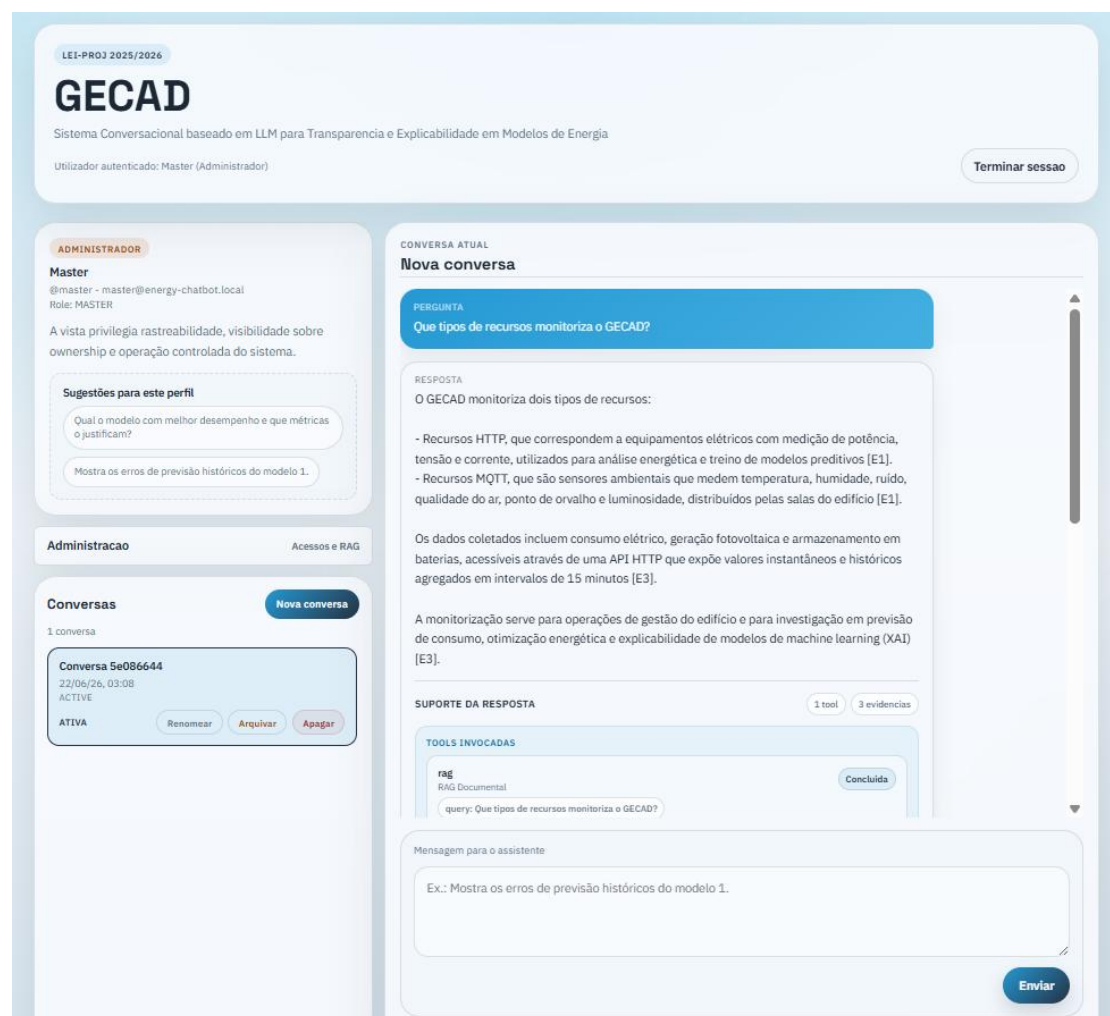


Figura 26: Vista do Sistema conversacional

A *feature admin* apresenta cinco separadores (Figura 27). O separador *Utilizadores* suporta a gestão de contas (ativação/desativação), atribuição de perfis e, para utilizadores com *role* MASTER, alteração de *roles*. O separador RAG apresenta o estado do *corpus* (número

de documentos indexados, coleção *Qdrant*, modelo de *embedding*) e o histórico de reindexações com detalhe por documento. O separador *ForeXAI* apresenta o catálogo de modelos disponíveis com métricas de avaliação. O separador *Ligações* expõe três cartões de monitorização, *LM Studio*, *ForeXAI* e *Qdrant*, com botões de teste individuais e um botão "Testar tudo" que executa as três verificações de saúde em paralelo. Cada cartão apresenta um indicador de estado (Ligado/Desativado/Inacessível) e os detalhes relevantes quando o serviço está acessível. O separador *Comparação* de perfis permite ao administrador submeter a mesma pergunta a dois perfis em simultâneo e comparar as respostas lado a lado. O *CompareProfilesService* executa o *pipeline* de *tools* uma única vez, planeamento, validação, expansão, execução e *grounding* são partilhados, mas invoca o *ResponseComposerAdapter* separadamente para cada perfil, injetando a orientação específica de *profiles.json* em cada composição. O resultado são duas respostas geradas sobre exatamente as mesmas evidências, tornando o efeito do perfil sobre o estilo e a profundidade da resposta diretamente observável.

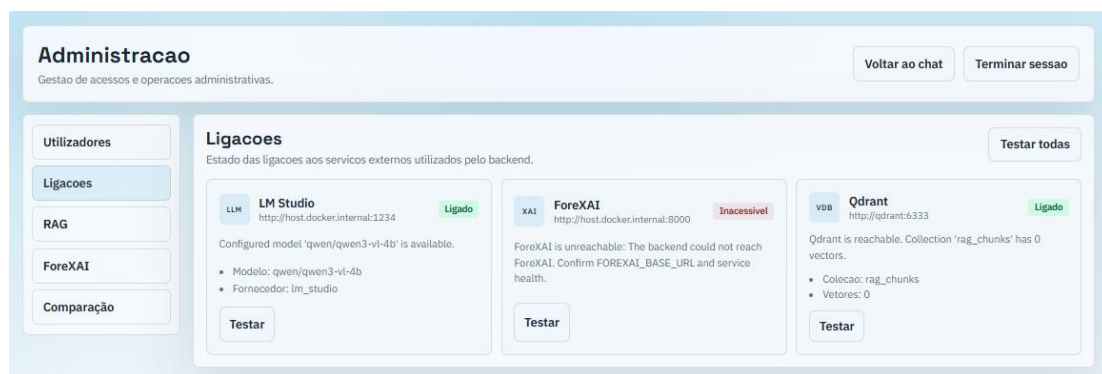


Figura 27: Painel de administração

4.2.7 Implementação

A solução é implementada via *Docker Compose*, que orquestra seis serviços num único *host*. A Tabela 12 descreve cada serviço.

Tabela 12: Serviços docker compose

Serviço	Tecnologia	Porta(s)	Papel
ldap_test	Samba AD DC	1389, 1636	Servidor LDAP de teste (LDAPS)
postgres	PostgreSQL 17 Alpine	5432	Base de dados principal (utilizadores, conversas, mensagens, respostas, evidências)

rag_catalog_postgres	PostgreSQL 17 Alpine	5435	Catálogo RAG (metadados de documentos e histórico de reindexações)
qdrant	Qdrant v1.13.4	6333, 6334	Vector store para <i>embeddings</i> RAG
backend	Python 3.12 / FastAPI	8005	Orquestrador central (LLM, RAG, ForeXAI, auth)
frontend	Node.js / Next.js	3000	Interface conversacional e painel de administração

O LM Studio corre como processo nativo no *host* (não containerizado) na porta 1234, expondo uma API compatível com a API *OpenAI*. Esta opção mantém o servidor de inferência fora do ciclo de vida do *Compose*, permitindo trocar e gerir modelos localmente sem reconstruir a *stack*.

Toda a configuração da *stack*, credenciais de base de dados, chaves de segurança, endereços de serviços, modelos LLM e parâmetros RAG está centralizada no ficheiro *.env* na raiz do repositório, que o *Compose* lê automaticamente. O ficheiro *.env.example* fornece um modelo documentado com todos os valores por defeito.

4.3 Testes

A estratégia de testes cobre as cinco camadas da *Clean Architecture* com testes unitários, valida o comportamento integrado do sistema com testes de integração e valida os fluxos completos de utilizador com testes *end-to-end* (E2E) executados num *browser* real.

4.3.1 Testes unitários

Os testes unitários estão organizados em *backend/tests/unit/* e cobrem as camadas de domínio, infraestrutura, API e aplicação. A Tabela 13 apresenta os grupos de testes, os ficheiros correspondentes e os principais aspetos verificados.

Tabela 13: Grupos de testes unitários por camada

Camada	Ficheiros de teste	Aspetos verificados
Domínio	test_conversation_entities.py, test_user_entities.py, test_assistant_response_entities.py	Invariantes das entidades, métodos de fábrica (<i>start_new()</i> , <i>create_user_messa</i>

		<i>ge()</i> , <i>create_response()</i> , validação de <i>value objects</i>
Infraestrutura LLM/RAG	test_lm_studio_adapter.py, test_lm_studio_embedding_service.py, test_rag_pipeline.py, test_rag_query_service.py	Contratos dos adaptadores LLM e RAG, <i>chunking</i> semântico, pesquisa vetorial
Infraestrutura ForeXAI	test_forexai_http_client_contract.py	Contrato HTTP do cliente ForeXAI, mapeamento de respostas para evidências
Infraestrutura Auth	test_composite_identity_provider.py, test_ldap_identity_provider.py, test_local_identity_provider.py	Delegação entre fornecedores, validação de credenciais LDAP e local
Infraestrutura Config	test_static_config_loader.py, test_settings.py	Carregamento de <i>tools.json</i> , <i>profiles.json</i> , variáveis de ambiente
API	test_health_controller.py, test_conversation_controller.py, test_chat_controller.py, test_admin_rag_controller.py, test_forexai_controller.py, test_auth_controller.py	Esquemas de pedido/resposta HTTP, delegação para <i>services</i> , códigos de estado
Aplicação	test_tool_selection_services.py, test_admin_rag_services.py, test_forexai_tool_handlers.py, test_auth_services.py, test_conversation_use_cases.py	Lógica dos <i>services</i> , orquestração de <i>tools</i> , mapeamento de respostas ForeXAI, fluxos de gestão RAG e autenticação

Bootstrap	test_user_bootstrap_service.py, test_ldap_test_users_bootstrap_service.py	Criação de utilizadores iniciais, configuração de utilizadores LDAP de teste
------------------	---	--

Na Figura 28 podemos verificar a execução dos testes unitários.

```
(.venv) PS C:\Users\ruigr\Documents\GitHub\PROJ_2025_2026_1230420> docker compose exec backend python -m pytest tests/unit -q
..... [ 54%]
..... [100%]

===== warnings summary =====
../usr/local/lib/python3.12/site-packages/ldap3/utils/asn1.py:50
/usr/local/lib/python3.12/site-packages/ldap3/utils/asn1.py:50: DeprecationWarning: tagMap is deprecated. Please use TAG_MAP instead.
  from pyasn1.codec.ber.encoder import tagMap, typeMap, AbstractItemEncoder

../usr/local/lib/python3.12/site-packages/ldap3/utils/asn1.py:50
/usr/local/lib/python3.12/site-packages/ldap3/utils/asn1.py:50: DeprecationWarning: typeMap is deprecated. Please use TYPE_MAP instead.
  from pyasn1.codec.ber.encoder import tagMap, typeMap, AbstractItemEncoder

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
196 passed, 2 warnings in 4.11s
(.venv) PS C:\Users\ruigr\Documents\GitHub\PROJ_2025_2026_1230420> 
```

Figura 28: Execução dos testes unitários

4.3.2 Testes de integração

Os testes de integração estão localizados em *backend/tests/integration/* e incluem dois ficheiros: *test_api.py* e *conftest.py*. O *conftest.py* configura as *fixtures* de base de dados e o cliente HTTP de teste (*TestClient* do *FastAPI*), assegurando que cada teste arranca com um estado de base de dados limpo. Os testes de integração verificam o comportamento dos *endpoints* principais contra uma base de dados de teste real, sem *mocks* de persistência, validando que a composição de serviços configurada pelo *container* de injeção de dependências funciona corretamente em conjunto. A Tabela 14 apresenta os principais cenários cobertos pela *suite* de integração.

Tabela 14: Testes de integração

Cenário	Aspetos verificados
Autenticação	<i>POST /auth/login</i> , emissão de token <i>bearer</i> e resolução do utilizador actual em <i>/auth/me</i>
Estado da aplicação	<i>Health checks</i> da aplicação e do adaptador LLM configurado para teste
Conversas	Criação, listagem, consulta de histórico vazio e renomeação de conversas

Autorização	Obrigatoriedade de autenticação para consultar o histórico de uma conversa
Ciclo de vida de conversas	Arquivo, remoção e resposta 404 após eliminação
Administração RAG	Reindexação, consulta do estado do <i>corpus</i> , histórico de reindexações, reindexação seletiva e gestão documental
Administração de utilizadores	Listagem administrativa e proteção do utilizador com <i>role</i> MASTER
Chat	Persistência da mensagem do utilizador, resposta do assistente, invocação da <i>tool</i> RAG e registo de evidências

Na Figura 29 podemos verificar a execução dos testes de integração.

```
(.venv) PS C:\Users\ruigr\Documents\GitHub\PROJ_2025_2026_1230420> docker compose exec backend python -m pytest te
sts/integration -q
..... [100%]
===== warnings summary =====
../usr/local/lib/python3.12/site-packages/fastapi/testclient.py:1
/usr/local/lib/python3.12/site-packages/fastapi/testclient.py:1: StarletteDeprecationWarning: Using `httpx` with
`starlette.testclient` is deprecated; install `httpx2` instead.
  from starlette.testclient import TestClient as TestClient # noqa
../usr/local/lib/python3.12/site-packages/ldap3/utils/asn1.py:50
/usr/local/lib/python3.12/site-packages/ldap3/utils/asn1.py:50: DeprecationWarning: tagMap is deprecated. Please
use TAG_MAP instead.
  from pyasn1.codec.ber.encoder import tagMap, typeMap, AbstractItemEncoder
../usr/local/lib/python3.12/site-packages/ldap3/utils/asn1.py:50
/usr/local/lib/python3.12/site-packages/ldap3/utils/asn1.py:50: DeprecationWarning: typeMap is deprecated. Pleas
e use TYPE_MAP instead.
  from pyasn1.codec.ber.encoder import tagMap, typeMap, AbstractItemEncoder
-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
9 passed, 3 warnings in 3.47s
(.venv) PS C:\Users\ruigr\Documents\GitHub\PROJ_2025_2026_1230420>
```

Figura 29: Execução dos testes de integração

4.3.3 Testes end-to-end

Os testes E2E estão localizados em *frontend/e2e/app.spec.ts* e exercitam a aplicação completa através de um *browser* real *Chromium* controlado pelo *Playwright*. Ao contrário dos testes de integração, que interagem diretamente com o *backend* via *TestClient*, os testes E2E simulam o comportamento efetivo do utilizador navegando para rotas, preenchendo formulários, clicando em botões e verificando elementos visíveis no ecrã. A Tabela 15 apresenta os oito cenários cobertos, organizados por papel do utilizador e funcionalidade exercitada.

Tabela 15: Cenários de teste E2E

#	Cenário	Utilizador	Aspetos verificados
1	RAG reindex indexa o documento de demonstração	<i>master</i>	Navegação <i>/chat</i> seguido de <i>/admin</i> , separador RAG, <i>trigger</i> de reindexação, confirmação do documento <i>e2e-demo.md</i> no estado do corpus
2	Painel de utilizadores mostra utilizadores criados em <i>bootstrap</i>	<i>master</i>	Painel de utilizadores admin, presença do utilizador Master com <i>role</i> MASTER e indicação de proteção
3	Utilizador envia mensagem com fundamentação e vê evidências	<i>viewer</i>	Envio de mensagem, resposta fundamentada pelo <i>mock</i> , expansão do painel de suporte com citação <i>e2e-demo.md</i> e <i>tool rag</i>
4	<i>Login</i> falhado mostra erro e não redireciona	-	Credenciais inválidas, visibilidade da mensagem de erro, permanência no painel de <i>login</i>
5	<i>Role</i> USER vê <i>chat</i> mas não vê o link de administração	<i>viewer</i>	Ausência do <i>admin-link</i> , disponibilidade do <i>chat-input</i> , visibilidade da lista de conversas
6	<i>Role</i> USER é bloqueado no painel de administração	<i>viewer</i>	Navegação directa para <i>/admin</i> , mensagem "Acesso administrativo indisponível", indicação da <i>role</i> USER
7	Gestão de conversas: criação e arquivo	<i>master</i>	Criação de nova conversa, arquivo com confirmação visual do estado
8	<i>Logout</i> limpa a sessão e mostra o painel de <i>login</i>	<i>master</i>	Clique em "Terminar sessão", reaparecimento do painel de <i>login</i>

Na Figura 30 podemos visualizar o relatório de execução dos testes E2E.

Q

All 8Passed 8Failed 0Flaky 0Skipped 0

Project: chromium22/06/2026, 05:43:18 Total time: 24.3s

▼ app.spec.ts

✓ admin: RAG reindex indexes the demo document4.4s
app.spec.ts:26 ▶

✓ admin: users panel shows bootstrapped users1.9s
app.spec.ts:40 ▶

✓ user can send a grounded chat message and see evidence2.0s
app.spec.ts:52 ▶

✓ login failure shows error and does not redirect1.6s
app.spec.ts:73 ▶

✓ USER role sees chat but not admin link1.5s
app.spec.ts:86 ▶

✓ USER role is blocked from admin panel2.1s
app.spec.ts:94 ▶

✓ conversation management: create and archive1.7s
app.spec.ts:103 ▶

✓ logout clears session and shows login panel1.6s
app.spec.ts:123 ▶

Figura 30: Relatório dos testes E2E

4.4 Avaliação da solução

A avaliação da solução confronta os casos de uso e os requisitos não funcionais definidos na secção 3 com o estado de implementação efetivo, identificando os objetivos concretizados e as funcionalidades ainda em curso.

4.4.1 Requisitos funcionais

A Tabela 16 apresenta o estado de implementação de cada caso de uso.

Tabela 16: Estado de implementação dos requisitos funcionais

Caso de uso	Título	Estado
UC-Auth	Autenticação (login, sessão)	Implementado
UC01	Iniciar conversa	Implementado
UC02	Consultar histórico de conversação	Implementado
UC03	Encerrar/arquivar conversa	Implementado
UC04	Enviar mensagem e obter resposta	Implementado
UC05	Adaptar resposta ao perfil do utilizador	Implementado

Rui Margarido64

UC06	Consultar previsões energéticas (ForeXAI)	Implementado
UC07	Consultar explicações XAI (ForeXAI)	Implementado
UC08	Consultar métricas e análise de erros	Implementado
UC09	Consultar documentação validada (RAG)	Implementado
UC10	Orquestrar plano de invocação de <i>tools</i>	Implementado
UC11	Registar evidências e citações	Implementado
UC12	Identificar estado de conclusão da resposta	Implementado
UC13	Gerir utilizadores	Implementado
UC14	Atribuir/alterar perfil e <i>role</i> de utilizador	Implementado
UC15	Gerir <i>corpus</i> RAG	Implementado
UC16	Consultar estado do <i>corpus</i> e histórico de reindexações	Implementado
UC17	Consultar catálogo de modelos ForeXAI	Implementado

4.4.2 Requisitos não funcionais

A Tabela 17 apresenta a avaliação sumária dos principais requisitos não funcionais definidos na secção 3.3.

Tabela 17: Avaliação dos requisitos não funcionais

NFR	Descrição	Estado	Observação
NFR01	<i>Backend</i> como único orquestrador do LLM, RAG e ForeXAI	Verificado	<i>Frontend</i> nunca contacta LM Studio, Qdrant nem ForeXAI diretamente
NFR02	Arquitetura que facilita integração com serviços externos	Verificado	<i>Clean Architecture</i> com adaptadores isolados para ForeXAI, RAG e LDAP
NFR03	Execução local do LLM via LM Studio	Verificado	<i>LMStudioAdapter</i> e <i>LMStudioEmbeddingAdapter</i> com <i>API OpenAI-compatible</i>

NFR04	Registo das <i>tools</i> invocadas e evidências em cada resposta	Verificado	Invocações de <i>tools</i> e evidências persistidas atomicamente no PostgreSQL
NFR05	Histórico de conversações preservado de forma consistente	Verificado	Entidades <i>Conversation</i> , <i>Message</i> e <i>AssistantResponse</i> persistidas transaccionalmente
NFR06	Evolução de <i>tools</i> e fontes de informação sem reestruturação	Verificado	Configuração declarativa em <i>tools.json</i> e <i>rag_retrieval.json</i> ; novos adaptadores sem alteração do domínio
NFR07	Contentorização e implementação via <i>Docker Compose</i>	Verificado	6 serviços <i>Docker Compose</i> ; migrações automáticas em arranque
NFR08	Tempos de resposta adequados para utilização interativa	Parcialmente verificado	Pipeline síncrono com múltiplas chamadas ao LLM; latência dependente do LM Studio
NFR09	Autenticação de utilizadores e controlo por <i>role</i>	Verificado	<i>CompositIdentityProvider</i> (LDAP + local); JWT; autorização por <i>role</i> (USER/ADMIN/MAS-TER)
NFR10	Sistema e documentação técnica em inglês onde necessário	Verificado	Código e classes em inglês

4.4.3 Validação qualitativa

A adequação dos perfis de resposta foi avaliada qualitativamente com recurso ao painel de Comparação de Perfis descrito na secção 4.1.6. Para cada um dos quatro perfis, foram submetidas perguntas representativas dos cenários de uso previstos, consulta de previsões energéticas, interpretação de métricas de avaliação e análise de explicações XAI, e as

respostas foram avaliadas quanto à coerência com as diretrizes de cada perfil definidas em *profiles.json*. A partilha de *tool invocations* e evidências entre perfis permitiu isolar o efeito da orientação de perfil, confirmando que o mesmo contexto factual é apresentado com níveis de detalhe técnico e vocabulário distintos conforme o destinatário. Não foram realizados estudos de usabilidade formais nem testes com utilizadores reais das categorias previstas, o que constitui uma limitação identificada na secção 5.2.

5 Conclusões

Esta secção apresenta uma síntese crítica do trabalho realizado, começando pela verificação dos objetivos concretizados e dos principais resultados obtidos, seguida da identificação das limitações atuais da solução e das oportunidades de melhoria prioritárias para trabalho futuro. Para concluir faremos uma apreciação final sobre o processo de desenvolvimento e o posicionamento da solução no contexto da transição energética.

5.1 Objetivos concretizados

O *XAI Energy Chatbot* foi implementado com sucesso como plataforma conversacional baseada em LLM para transparência e explicabilidade de modelos de energia, cumprindo os objetivos principais definidos no início do projeto.

Do ponto de vista arquitetural, a solução adota uma *Clean Architecture* de cinco camadas, assegurando que o domínio de negócio é completamente independente de *frameworks*, bases de dados e serviços externos. A regra de dependência é garantida por um *container* de injeção de dependências, que compõe o esquema completo de dependências em tempo de arranque e torna os adaptadores concretos substituíveis sem alterações na lógica de negócio.

Do ponto de vista funcional, foram concretizados os 18 casos de uso previstos, cobrindo o ciclo completo de interação conversacional, a administração de utilizadores e *corpus* RAG, e a consulta de modelos e artefactos de explicabilidade XAI via ForeXAI. Em particular, os fluxos mais complexos foram todos implementados:

- A orquestração de *tools* com o pipeline *planner*, *validator*, *executor*, *responder*.
- A política de *grounding* que garante que as respostas são sempre fundamentadas em evidências documentais verificáveis;

- O RAG com *chunking* semântico, indexação no *Qdrant* e registo no catálogo RAG;
- A adaptação dinâmica do estilo de resposta a quatro perfis de utilizador;
- A autenticação LDAP e local (apenas para o *Master*);
- A interface conversacional (*frontend Next.js*) e o painel de administração com gestão de utilizadores, *corpus* RAG e catálogo ForeXAI;
- A infraestrutura totalmente contentorizada em *Docker Compose*.

Foram também implementados testes unitários e testes de integração que cobrem as cinco camadas da arquitetura. A cobertura estende-se ainda a testes E2E com *Playwright*, que exercitam a experiência completa do utilizador num *browser* real através de toda a *stack* contentorizada.

5.2 Limitações e trabalho futuro

Em termos de limitações, o servidor de inferência LM Studio não está contentorizado, o que introduz uma dependência de configuração manual no *host*, ou seja, o utilizador deve instalar e iniciar o LM Studio separadamente, carregar o modelo adequado e expor a API na porta 1234 antes de arrancar a *stack Docker*. Esta dependência externa aumenta a fricção no processo de configuração inicial. Substituição do LM Studio por um servidor contentorizado como o *Ollama* com serviço *Docker* eliminaria a dependência de configuração manual no *host* e simplificaria o processo de implementação.

Os três limiares de *grounding* (0,55 para rejeição, 0,15 para *overlap* lexical e 0,80 para aceitação por alta confiança) foram definidos de forma empírica com base em poucos testes. Não foi realizada uma avaliação da sua adequação a diferentes domínios documentais ou tipos de pergunta, o que pode conduzir a filtragens excessivamente restritivas ou permissivas em contextos não testados. Uma avaliação sistemática com conjuntos de perguntas representativos permitiria calibrar os limiares de similaridade e *overlap* para diferentes tipos de *corpus* documental.

Por fim, a implementação de *streaming* de resposta para envio progressivo da resposta ao *frontend* melhoraria significativamente a experiência do utilizador em respostas longas.

5.3 Apreciação final

O desenvolvimento do *XAI Energy Chatbot* representou um desafio de engenharia multidimensional: integrar um LLM local com RAG vetorial, uma API de explicabilidade XAI externa e um sistema de autenticação LDAP, mantendo ao longo de toda a solução os princípios da *Clean Architecture* e a separação estrita de responsabilidades entre camadas.

A decisão de adotar a *Clean Architecture* revelou-se particularmente valiosa. A independência do domínio face a *frameworks* e serviços externos permitiu testar os conceitos de negócio de forma isolada, substituir adaptadores como o fornecedor de identidade sem alteração da lógica de aplicação, e integrar a ForeXAI de forma incremental sem risco de regressão nos fluxos principais. Esta flexibilidade, que se manifesta na facilidade de configuração e na cobertura de testes, dificilmente seria alcançável numa arquitetura menos estruturada.

O tema do projeto adquire uma relevância crescente num contexto de transição energética e de intensificação do escrutínio regulatório sobre sistemas de IA. Plataformas que permitam a utilizadores com diferentes perfis interrogar modelos de energia em linguagem natural e obter respostas fundamentadas e rastreáveis constituem uma ponte necessária entre a sofisticação técnica dos modelos e a sua utilização responsável em contextos reais.

O projeto demonstrou que é possível construir, com recursos de inferência locais e tecnologias *open-source*, um sistema conversacional com garantias formais de *grounding* e rastreabilidade representando um passo concreto na direção de IA explicável e auditável ao serviço da transição energética.

Referências

- Ali, S., Abuhmed, T., El-Sappagh, S., Muhammad, K., Alonso-Moral, J. M., Confalonieri, R., Guidotti, R., Del Ser, J., Díaz-Rodríguez, N., & Herrera, F. (2023). Explainable Artificial Intelligence (XAI): What we know and what is left to attain Trustworthy Artificial Intelligence. *Information Fusion, Studies in Computational Intelligence*, 99, 101805. <https://doi.org/10.1016/j.inffus.2023.101805>
- Ali, S., Akhlaq, F., Imran, A. S., Kastrati, Z., Daudpota, S. M., & Moosa, M. (2023). The enlightening role of explainable artificial intelligence in medical & healthcare domains: A systematic literature review. *Computers in Biology and Medicine*, 166, 107555. <https://doi.org/10.1016/j.compbiomed.2023.107555>
- Amazon. (2026). *Alexa*. <https://alexa.amazon.com/about>
- Apple. (2026). *Apple Intelligence and Siri - Apple*. 2026. <https://www.apple.com/apple-intelligence/>
- C4 model. (2026). <https://c4model.com/>
- Casheekar, A., Lahiri, A., Rath, K., Prabhakar, K. S., & Srinivasan, K. (2024). A contemporary review on chatbots, AI-powered virtual conversational agents, ChatGPT: Applications, open challenges and future research directions. *Computer Science Review*, 52, 100632. <https://doi.org/10.1016/J.COSREV.2024.100632>
- Chitty-Venkata, K. T., Mittal, S., Emani, M., Vishwanath, V., & Somani, A. K. (2023). A survey of techniques for optimizing transformer inference. *Journal of Systems Architecture*, 144, 102990. <https://doi.org/10.1016/j.sysarc.2023.102990>
- Chroma Docs. (2026). <https://docs.trychroma.com/docs/overview/introduction>
- Clean Architecture: Simplified and In-Depth Guide | Medium. (2026). <https://medium.com/@DrunknCode/clean-architecture-simplified-and-in-depth-guide-026333c54454>
- Clement, T., Truong, H., Nguyen, T., Kemmerzell, N., Abdelaal, M., & Stjelja, D. (2024). *Beyond explaining: XAI-based Adaptive Learning with SHAP Clustering for Energy Consumption Prediction*. <https://arxiv.org/pdf/2402.04982v1>
- Dinis Castelhana, I. (2026). *Explainable AI: Enhancing Machine Learning Model Interpretability with Generative AI*. <http://hdl.handle.net/10362/190584>

- Django*. (2026). <https://www.djangoproject.com/>
- Docker*. (2026). <https://docs.docker.com/>
- Documentation - Claude API Docs*. (2026). <https://platform.claude.com/docs/en/home>
- Documentation - Qdrant*. (2026). <https://qdrant.tech/documentation/>
- Falcon. (2026). *Introducing the Technology Innovation Institute's Falcon Perception Making Advanced AI accessible and Available to Everyone, Everywhere*. <https://falconllm.tii.ae/>
- FastAPI*. (2026). <https://fastapi.tiangolo.com/>
- Flask Documentation*. (2026). <https://flask.palletsprojects.com/en/stable/>
- Fowler, Martin. (2003). *Patterns of enterprise application architecture*. Addison-Wesley.
- Gamma, Erich., Helm, Richard., Johnson, R. E. ., & Vlissides, John. (2016). *Design patterns : elements of reusable object-oriented software*. Addison-Wesley.
- GECAD. (2026). *gecad – Research Group on Intelligent Engineering and Computing for Advanced Innovation and Development*. <https://www.gecad.isep.ipp.pt/>
- Ghani, S., Håkansson, A., Pasichnyi, O., & Shahrokni, H. (2025). *Conversational Agents for Building Energy Efficiency -- Advising Housing Cooperatives in Stockholm on Reducing Energy Consumption*. <http://arxiv.org/abs/2511.08587>
- Google. (2026). *Google Assistant, your own personal Google*. <https://assistant.google.com/>
- Gradio*. (2026). <https://gradio.app/>
- Huang, L., Yu, W., Ma, W., Zhong, W., Feng, Z., Wang, H., Chen, Q., Peng, W., Feng, X., Qin, B., & Liu, T. (2025a). A Survey on Hallucination in Large Language Models: Principles, Taxonomy, Challenges, and Open Questions. *ACM Transactions on Information Systems*, 43(2). <https://doi.org/10.1145/3703155>
- Huang, L., Yu, W., Ma, W., Zhong, W., Feng, Z., Wang, H., Chen, Q., Peng, W., Feng, X., Qin, B., & Liu, T. (2025b). A Survey on Hallucination in Large Language Models: Principles, Taxonomy, Challenges, and Open Questions. *ACM Transactions on Information Systems*, 43(2). <https://doi.org/10.1145/3703155>
- Ji, Z., Lee, N., Frieske, R., Yu, T., Su, D., Xu, Y., Ishii, E., Bang, Y., Chen, D., Dai, W., Chan, H. S., Madotto, A., & Fung, P. (2024a). Survey of Hallucination in Natural Language Generation. *ACM Computing Surveys*, 55(12). <https://doi.org/10.1145/3571730>

- Ji, Z., Lee, N., Frieske, R., Yu, T., Su, D., Xu, Y., Ishii, E., Bang, Y., Chen, D., Dai, W., Chan, H. S., Madotto, A., & Fung, P. (2024b). Survey of Hallucination in Natural Language Generation. *ACM Computing Surveys*, 55(12). <https://doi.org/10.1145/3571730>
- Ji, Z., Lee, N., Frieske, R., Yu, T., Su, D., Xu, Y., Ishii, E., Bang, Y., Chen, D., Dai, W., Chan, H. S., Madotto, A., & Fung, P. (2024c). Survey of Hallucination in Natural Language Generation. *ACM Computing Surveys*, 55(12). <https://doi.org/10.1145/3571730>
- Kaiyrbekov, K., Dobbins, N. J., & Mooney, S. D. (2025). Automated Survey Collection with LLM-based Conversational Agents. *ArXiv*, arXiv:2504.02891v1. <https://pmc.ncbi.nlm.nih.gov/articles/PMC12306824/>
- Kovari, A. (2025a). Explainable AI chatbots towards XAI ChatGPT: A review. *Heliyon*, 11(2), e42077. <https://doi.org/10.1016/j.heliyon.2025.e42077>
- Kovari, A. (2025b). Explainable AI chatbots towards XAI ChatGPT: A review. *Heliyon*, 11(2), e42077. <https://doi.org/10.1016/j.heliyon.2025.e42077>
- LangChain*. (2026). <https://docs.langchain.com/>
- Larman, Craig. (2004). *Applying UML and Patterns: An Introduction to Object-Oriented Analysis and Design and Iterative Development, Third Edition*. Pearson.
- Learn Java - Dev.java*. (2026). <https://dev.java/learn/>
- Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W., Rocktäschel, T., Riedel, S., & Kiela, D. (2021a). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *Advances in Neural Information Processing Systems, 2020-December*. <http://arxiv.org/abs/2005.11401>
- Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W., Rocktäschel, T., Riedel, S., & Kiela, D. (2021b). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *Advances in Neural Information Processing Systems, 2020-December*. <http://arxiv.org/abs/2005.11401>
- Lin, Y.-T., & Chen, Y.-N. (2023). *LLM-EVAL: Unified Multi-Dimensional Automatic Evaluation for Open-Domain Conversations with Large Language Models*.
- Liu, M., Yang, D., Beyette, F. R., Wang, S., & Zhao, L. (2023). *Recent Progress in Understanding Transformer and Developing Its Surpasser: A Survey*.
- Llama*. (2026). <https://developer.meta.com/ai/>

- LlamaIndex Developer Documentation.* (2026).
<https://developers.llamaindex.ai/python/framework/>
- LM Studio.* (2026). <https://lmstudio.ai/docs/app>
- Lundberg, S. M., Allen, P. G., & Lee, S.-I. (2026). *A Unified Approach to Interpreting Model Predictions.* <https://github.com/slundberg/shap>
- Martin, R. C. . (2018). *Clean architecture : a craftsman's guide to software structure and design.* Prentice Hall.
- Medium. (2026). *Clean Architecture: Simplified and In-Depth Guide | Medium.*
<https://medium.com/@DrunknCode/clean-architecture-simplified-and-in-depth-guide-026333c54454>
- Meta Intelligence. (2026). *LLM Function Calling: OpenAI Tools API, Multi-Step Tool Chains & Error Handling.* <https://www.meta-intelligence.tech/en/insight-function-calling>
- Michelon, F., Zhou, Y., & Morstyn, T. (2025). Large Language Model Interface for Home Energy Management Systems. *E-ENERGY 2025 - Proceedings of the 2025 16th ACM International Conference on Future and Sustainable Energy Systems, 1*, 590–602.
<https://doi.org/10.1145/3679240.3734586>
- Minaee, S., Mikolov, T., Nikzad, N., Chenaghlu, M., Socher, R., Amatriain, X., & Gao, J. (2025a). *Large Language Models: A Survey.* <https://arxiv.org/pdf/2402.06196v3>
- Minaee, S., Mikolov, T., Nikzad, N., Chenaghlu, M., Socher, R., Amatriain, X., & Gao, J. (2025b). *Large Language Models: A Survey.* <https://arxiv.org/pdf/2402.06196v3>
- Mistral AI.* (2026). <https://mistral.ai/>
- Nguyen, V. B., Schlötterer, J., & Seifert, C. (2024). *XAgent: A Conversational XAI Agent Harnessing the Power of Large Language Models.* <https://github.com/aix-group/XAGENT/>
- Node.js v26.3.1 Documentation.* (2026). <https://nodejs.org/docs/latest/api/>
- OpenAI. (2026a). *ChatGPT.* <https://chatgpt.com/>
- OpenAI. (2026b). *Function calling | OpenAI API.*
<https://developers.openai.com/api/docs/guides/function-calling>
- OpenAI Developers.* (2026). <https://developers.openai.com/>

- Parlamento Europeu e Conselho da UE. (2024). *Regulation - EU - 2024/1689 - EN - EUR-Lex*.
<https://eur-lex.europa.eu/eli/reg/2024/1689/oj/eng>
- Pinecone documentation - Pinecone Docs*. (2026). <https://docs.pinecone.io/guides/get-started/overview>
- PostgreSQL Global Development Group*. (2026). <https://www.postgresql.org/docs/>
- Prabhu, H., Valdi, J., & Arjunan, P. (2023). Explainable AI for Energy Prediction and Anomaly Detection in Smart Energy Buildings. *The 10th ACM International Conference on Systems for Energy-Efficient Buildings, Cities, and Transportation (BuildSys '23), November 15-16, 2023, Istanbul, Turkey*, 1. <https://doi.org/10.1145/3600100.3626638>
- Python Documentation*. (2026). <https://www.python.org/doc/>
- React*. (2026). <https://react.dev/reference/react>
- Retrieval Augmented Generation | Weaviate*. (2026). <https://weaviate.io/rag>
- Ribeiro, M. T., Singh, S., & Guestrin, C. (2016). “Why should i trust you?” Explaining the predictions of any classifier. *Proceedings of the ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 13-17-August-2016*, 1135–1144. <https://doi.org/10.1145/2939672.2939778>
- Ronanki, K., Arvidsson, S., & Axell, J. (2025). *Prompt Engineering Guidelines for Using Large Language Models in Requirements Engineering*.
- Santos, G., Pinto, T., Ramos, C., & Corchado, J. M. (2023). Editorial: Explainability in knowledge-based systems and machine learning models for smart grids. *Frontiers in Energy Research*, 11, 1269397. <https://doi.org/10.3389/fenrg.2023.1269397>
- Saranya, A., & Subhashini, R. (2023). A systematic review of Explainable Artificial Intelligence models and applications: Recent developments and future trends. *Decision Analytics Journal*, 7(5), 100230. <https://doi.org/10.1016/j.dajour.2023.100230>
- Singh, S. U., & Namin, A. S. (2025). A survey on chatbots and large language models: Testing and evaluation techniques. *Natural Language Processing Journal*, 10. <https://doi.org/10.1016/j.nlp.2025.100128>
- Streamlit documentation*. (2026). <https://docs.streamlit.io/>
- Wang, L., Ma, C., Feng, X., Zhang, Z., Yang, H., Zhang, J., Chen, Z., Tang, J., Chen, X., Lin, Y., Zhao, W. X., Wei, Z., & Wen, J.-R. (2025). A Survey on Large Language Model based

Autonomous Agents. *Frontiers of Computer Science*, 18(6), 1–42.
<https://doi.org/10.1007/s11704-024-40231-1>

Zhang, L., & Chen, Z. (2024). Large language model-based interpretable machine learning control in building energy systems. *Energy and Buildings*, 313(1), 114278.
<https://doi.org/10.1016/j.enbuild.2024.114278>

Zhao, W. X., Zhou, K., Li, J., Tang, T., Wang, X., Hou, Y., Min, Y., Zhang, B., Zhang, J., Dong, Z., Du, Y., Yang, C., Chen, Y., Chen, Z., Jiang, J., Ren, R., Li, Y., Tang, X., Liu, Z., ... Wen, J.-R. (2023). A Survey of Large Language Models. *ArXiv Preprint ArXiv:2303.18223*.
<https://arxiv.org/pdf/2303.18223>

Zhao, W. X., Zhou, K., Li, J., Tang, T., Wang, X., Hou, Y., Min, Y., Zhang, B., Zhang, J., Dong, Z., Du, Y., Yang, C., Chen, Y., Chen, Z., Jiang, J., Ren, R., Li, Y., Tang, X., Liu, Z., ... Wen, J.-R. (2026). A Survey of Large Language Models. *ArXiv Preprint ArXiv:2303.18223*.
<http://arxiv.org/abs/2303.18223>

Anexo A Cronograma

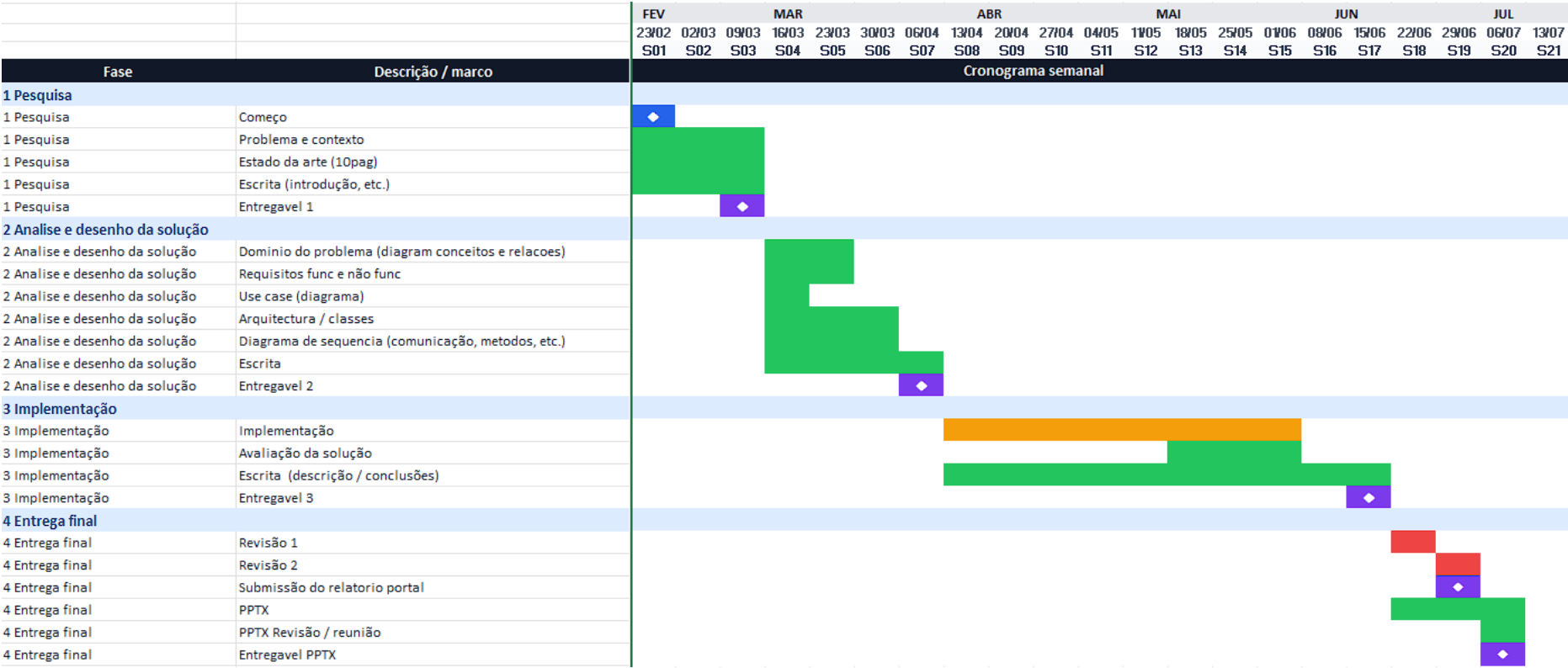


Figura 31: Diagrama de Gantt do planeamento do trabalho

Anexo B SHAP - LIME

<i>Característica</i>	<i>SHAP (Shapley Additive Explanations)</i>	<i>LIME (Local Interpretable Model-Agnostic Explanations)</i>
<i>Tipo de explicação</i>	Local e global	Principalmente local
<i>Base teórica</i>	Teoria dos jogos cooperativos (valores de Shapley)	Aproximação local do modelo através de regressão interpretável
<i>Objetivo</i>	Quantificar a contribuição de cada variável para a previsão	Identificar quais variáveis influenciam uma previsão específica
<i>Funcionamento</i>	Calcula o impacto marginal de cada variável considerando diferentes combinações possíveis	Gera perturbações nos dados de entrada e ajusta um modelo simples para aproximar o comportamento local
<i>Interpretabilidade</i>	Elevada consistência teórica	Fácil interpretação local
<i>Vantagens</i>	Explicações consistentes e comparáveis entre diferentes modelos	Método simples e aplicável a qualquer modelo
<i>Limitações</i>	Elevado custo computacional em modelos complexos	Resultados podem variar consoante as amostras geradas
<i>Tipo de modelo</i>	Model-agnostic (existem implementações específicas para alguns modelos)	Model-agnostic
<i>Aplicações típicas</i>	Análise de importância de variáveis, explicações locais e globais	Interpretação de decisões individuais

Anexo C tools.json

```
backend > configs > {} tools.json > ...
1  {
2    "tools": [
3      {
4        "name": "rag",
5        "endpoint": "internal",
6        "description": [
7          "Use when the user asks for validated documentation, grounded explanations, definitions,",
8          "or evidence that should come from the local document base."
9        ],
10       "required_arguments": ["query"]
11     },
12     {
13       "name": "forexai_list_models",
14       "endpoint": "GET /api/v1/models/",
15       "description": [
16         "Use when the user asks what ForeXAI models are available or needs to choose a model."
17       ],
18       "required_arguments": []
19     },
20     {
21       "name": "forexai_get_model",
22       "endpoint": "GET /api/v1/models/{model_id}",
23       "description": [
24         "Use to get base status and metadata for a ForeXAI model. Requires model_id unless",
25         "the backend can resolve it from conversation context or demo configuration."
26       ],
27       "required_arguments": []
28     },
29     {
30       "name": "forexai_get_model_info",
31       "endpoint": "GET /api/v1/models/{model_id}/info",
32       "description": [
33         "Use to get model introspection, feature names, train/test row counts and target information.",
34         "Requires model_id unless the backend can resolve it from context."
35       ],
36       "required_arguments": []
37     },
38   ],
39 }
```

Figura 32: Excerto do ficheiro tools.json

Anexo D profiles.json

```
> configs > {} profiles.json > {} profiles > {} ADMINISTRATOR > {} insufficient_evidence_response > abc 1
{
  "default_profile_type": "COMMON_USER",
  "profiles": {
    "COMMON_USER": {
      "response_guidance": [
        "Use plain, accessible language suitable for a non-technical audience.",
        "Briefly define any technical term before using it.",
        "Use analogies or examples when they help clarify a concept.",
        "Keep responses concise and well-structured.",
        "Do not include API endpoint paths, URL patterns, parameter names, or any technical implementation details."
      ],
      "insufficient_evidence_response": [
        "It was not possible to find sufficient information to answer your question with confidence.",
        "The answer can only be based on the evidence retrieved from the available sources."
      ]
    },
    "ENERGY_SPECIALIST": {
      "response_guidance": [
        "Use energy-domain terminology when supported by the evidence.",
        "Preserve relevant assumptions, constraints, and operational implications.",
        "Include quantitative details such as error metrics and forecast ranges when the evidence supports them.",
        "Surface uncertainties or limitations in the evidence when they affect operational conclusions."
      ],
      "insufficient_evidence_response": [
        "Insufficient evidence was found in the available tools to answer with confidence.",
        "To maintain a well-grounded response, additional domain-specific evidence would be required."
      ]
    },
    "ML_SPECIALIST": {
      "response_guidance": [
        "Use technically precise language and preserve exact metric names, method names, and parameter names from the evidence.",
        "When supported by the evidence, surface model limitations, confidence cues, and implementation nuances.",
        "Distinguish between global and local explanations when discussing XAI results.",
        "Avoid simplifying technical trade-offs or omitting caveats present in the evidence."
      ],
      "insufficient_evidence_response": [
        "Insufficient evidence was found in the available tools to answer with confidence.",
        "To maintain a well-grounded response, no extrapolations will be made beyond the retrieved evidence."
      ]
    }
  }
}
```

Figura 33: Excerto do ficheiro profiles.json

Anexo E chat_prompts.json

```
"response": {
  "system_instructions": [
    "You are composing a grounded response for an energy model explainability chatbot.",
    "Answer only with facts explicitly supported by the evidence blocks provided.",
    "Do not use outside knowledge, unstated assumptions, or invented details.",
    "If the evidence does not fully support the answer, state clearly what is supported and what is not.",
    "Prefer clear synthesis over copying evidence verbatim.",
    "When the question asks about procedures, endpoints, parameters, metrics, or model features, preserve the exact technical names from the evidence.",
    "Cite the evidence inline using evidence labels, for example [E1] or [E2].",
    "Do not cite a label unless that evidence block directly supports the claim."
  ],
  "requirements": [
    "Response requirements:",
    "- Answer always in European Portuguese (Portugal), unless the user explicitly asks for a different language.",
    "- Start with the direct answer.",
    "- Use short paragraphs or bullets when the answer has multiple points.",
    "- Include inline evidence labels such as [E1] next to each supported claim.",
    "- Preserve exact endpoint paths, parameter names, metric names, feature names, and method names from the evidence.",
    "- If the user asks for a future prediction and the evidence has no predict endpoint, state that the current ForeXAI interface does not expose future prediction.",
    "- When selecting a best model, state the metric criterion used.",
    "- If the evidence only answers part of the question, clearly separate the supported part from the missing part.",
    "- Do not mention vector scores, lexical scores, chunk numbers, or internal retrieval mechanics.",
    "- When evidence contains artifact.type image, state that the visual result (chart or plot) is displayed in the conversation. Reference the model ID, method, and row index"
  ]
}
```

Figura 34: Excerto do ficheiro chat_prompts.json

Anexo F rag_retrieval.json

```
{
  "stopwords": [
    "a", "ao", "aos", "as", "com", "como",
    "da", "das", "de", "do", "dos",
    "e", "em",
    "for", "how", "is",
    "na", "nas", "no", "nos",
    "o", "of", "or", "os",
    "para", "por",
    "que",
    "the", "to",
    "um", "uma"
  ],
  "term_aliases": {
    "documentation": ["documentacao", "document", "docs", "documentos", "documentation"],
    "documentacao": ["documentation", "document", "docs", "documentos", "documentacao"],
    "validated": ["validada", "validado", "validadas", "validados", "validated"],
    "validada": ["validated", "validado", "validadas", "validados", "validada"],
    "transparency": ["transparencia", "transparent", "transparency"],
    "transparencia": ["transparency", "transparent", "transparencia"],
    "energy": ["energia", "energetica", "energetico", "energy", "energetic"],
    "energia": ["energy", "energetic", "energetica", "energetico", "energia"],
    "energetica": ["energy", "energetic", "energia", "energetico", "energetica"],
    "energetico": ["energy", "energetic", "energia", "energetica", "energetico"],
    "evidence": ["evidencia", "evidencias", "evidence", "evidences"],
    "evidencia": ["evidence", "evidences", "evidencia", "evidencias"],
    "explainability": ["explicabilidade", "explainability"],
    "explicabilidade": ["explainability", "explicabilidade"],
    "ale": ["accumulated", "local", "effects", "pdp", "correlacionadas"],
    "anchors": ["anchor", "regras", "rules", "if", "then"],
    "counterfactual": ["counterfactuals", "contrafactual", "contrafactuais", "dice"],
    "counterfactuals": ["counterfactual", "contrafactual", "contrafactuais", "dice"],
    "cv": ["cross", "validation", "validacao", "timeseries", "kfold"],
    "data": ["dados", "dataset"],
    "dataset": ["data", "dados", "csv"],
    "endpoint": ["rota", "route", "api", "rest"],
    "forexai": ["forecast", "xai", "mlps"],
    "forecast": ["previsao", "previsoes", "forexai"],
    "fotovoltaica": ["pv", "solar", "geracao"],
    "leakage": ["vazamento", "fuga", "lookahead"],
    "lime": ["local", "interpretable", "model", "agnostic", "explanations"],
    "pdp": ["partial", "dependence", "plot", "ale"],
    "preprocess": ["preprocessamento", "pre", "processamento"],
    "preprocessamento": ["preprocess", "pre", "processamento"],
    "p_kw": ["potencia", "power", "kw", "kilowatt", "consumo", "carga"],
    "pv": ["fotovoltaica", "solar", "geracao"],
    "shap": ["shapley", "global", "local", "beeswarm", "waterfall"],
    "soc": ["state", "charge", "carga", "bateria"],
    "soh": ["state", "health", "saude", "bateria"],
    "timeseries": ["serie", "temporal", "cv", "kfold"],
    "xai": ["explicabilidade", "explainability", "shap", "lime", "ale", "pdp"]
  }
}
```

Figura 35: Excerto do ficheiro rag_retrieval.json

Anexo G `_build_planner_prompt`

```
def _build_planner_prompt(self, *, input_dto: SelectToolsInput) -> str:
    """Builds the full prompt sent to the planner LLM.

    Assembles the system instructions, the tool and workflow catalogues (from the
    registries), the recent conversation history, the expected output format, and
    the current user question into a single prompt string.

    Args:
        input_dto (SelectToolsInput): DTO with the user prompt and conversation
            history used to populate the prompt.

    Returns:
        str: Fully rendered planner prompt ready to be sent to the LLM.
    """
    tool_descriptions = "\n".join(
        self._format_tool_description(tool)
        for tool in CHAT_TOOL_REGISTRY.values()
    )
    workflow_descriptions = "\n".join(
        self._format_workflow_description(workflow)
        for workflow in CHAT_TOOL_WORKFLOW_REGISTRY.values()
    )
    history_lines = [
        f"- {message.role}: {message.content}"
        for message in input_dto.recent_history[-self._MAX_HISTORY_MESSAGES :]
    ]
    rendered_history = "\n".join(history_lines) if history_lines else "- none"

    return (
        f"{self._prompts_config.planner_system_instructions}\n\n"
        "Available tools:\n"
        f"{tool_descriptions}\n\n"
        "Available workflows:\n"
        f"{workflow_descriptions or '- none'}\n\n"
        "Recent conversation history:\n"
        f"{rendered_history}\n\n"
        "Choose between these output shapes:\n"
        '{"mode": "no_tool", "selected_tools": []}\n'
        'or\n'
        '{"mode": "use_tools", "selected_tools": [{"tool": "<tool or workflow name>", '
        '"arguments": {"query": "...", "model_id": "...", "method": "...", "row_index": "..."}, '
        '"reason": "..."}]}\n\n'
        f"{self._prompts_config.planner_rules}\n\n"
        "User question:\n"
        f"{input_dto.user_prompt}"
    )
```

Figura 36: Construção do prompt para o planner

Anexo H `_enforce_tool_usage`

```
def enforce_tool_usage(
    self,
    *,
    user_prompt: str,
    tool_plan: ToolPlanOutput,
) -> ToolPlanOutput:
    """Ensures the plan includes at least one tool call (Validator phase).

    If the planner already selected tools, the plan is returned unchanged. If the
    planner produced a no_tool plan, a RAG tool call is injected using the user
    prompt as the query, enforcing the policy that all responses must be grounded
    in retrieved documents.

    An empty or whitespace-only prompt bypasses injection and returns a no_tool
    plan, as a RAG call with a blank query would return no useful results.

    Args:
        user_prompt (str): Original user message, used as the RAG query when
            injection is needed.
        tool_plan (ToolPlanOutput): Tool plan produced by the Planner phase.

    Returns:
        ToolPlanOutput: The original plan if it already uses tools; a new plan
            with a single RAG invocation otherwise (or no_tool for a blank prompt).
    """
    if tool_plan.mode == "use_tools":
        return tool_plan

    normalized_prompt = user_prompt.strip()
    if not normalized_prompt:
        return ToolPlanOutput.no_tool()

    return ToolPlanOutput.use_tools(
        [
            PlannedToolOutput(
                tool_name="rag",
                arguments={"query": normalized_prompt},
                reason="Policy requires tool-backed responses.",
            )
        ]
    )
```

Figura 37: Grounding de no_tool

Anexo I execute

```

async def execute(self, *, plan: ToolPlanOutput) -> list[ToolExecutionOutput]:
    """Executes all tools and workflows in a plan and returns their results.

    Separates the plan into dynamic workflows (step-output dependencies) and
    everything else (static tools and simple workflows). Dynamic workflows are
    executed sequentially step-by-step; the remaining tools are expanded by
    ExpandToolWorkflowsService and then executed.

    Validation errors from workflow argument checks are included in the returned
    list alongside successful results.

    Args:
        plan (ToolPlanOutput): Validated tool plan from SelectToolsService,
            possibly already enforced by GroundingPolicyService.

    Returns:
        list[ToolExecutionOutput]: Results from all executed tools, including
            TOOL_VALIDATION entries for tools that failed argument validation.
    """

    # Phase 1 – Validate workflow-level arguments.
    # Workflows that are missing required arguments are converted to TOOL_VALIDATION
    # error outputs and removed from the plan; the rest proceed as validated entries.
    workflow_checked_plan, workflow_validation_errors = self._validate_workflow_plan(plan)

    # Seed the result list with any validation errors so they are always returned,
    # even if no further tools are executed.
    executed_tools: list[ToolExecutionOutput] = list(workflow_validation_errors)

    # If validation removed every tool from the plan, there is nothing left to run.
    if workflow_checked_plan.mode != "use_tools":
        return executed_tools

    # Phase 2 – Route each planned entry to the correct execution path.
    # Tools and workflows that can be expanded statically are collected in
    # static_tools; dynamic workflows are executed immediately in this loop.
    static_tools: list[PlannedToolOutput] = []
    for planned_tool in workflow_checked_plan.selected_tools:

        # Check whether this entry refers to a known workflow definition.
        # Individual tool calls (e.g. "rag") are not in the workflow registry.
        workflow_definition = CHAT_TOOL_WORKFLOW_REGISTRY.get(planned_tool.tool_name)
        if workflow_definition is None:
            # Not a workflow – treat as a plain tool and defer to static execution.
            static_tools.append(planned_tool)
            continue

        if self._workflow_uses_step_outputs(workflow_definition):
            # Dynamic workflow: at least one step argument references a prior step's
            # output via "$steps.<tool>.<key>". These steps must run sequentially so
            # each step can consume the previous one's results.
            executed_tools.extend(
                await self._execute_dynamic_workflow(
                    workflow_name=workflow_definition.name,
                    workflow_arguments=planned_tool.arguments,
                )
            )
            continue

        # Simple workflow: no inter-step dependencies. ExpandToolWorkflowsService
        # can expand it into a flat list of tool calls, so defer to static execution.
        static_tools.append(planned_tool)

```

Figura 38: Execução das tools (parte 1/2)

```
# Phase 3 – Expand and execute all static entries.
# ExpandToolWorkflowsService replaces each simple workflow name with its
# constituent tool steps, producing a flat plan of individual tool calls.
expanded_plan = self._workflow_expansion_service.execute(
    plan=ToolPlanOutput.use_tools(static_tools),
)

# Validate each individual tool's arguments against CHAT_TOOL_REGISTRY and
# execute them via their registered handlers. Results are appended in order.
executed_tools.extend(await self._execute_expanded_plan(expanded_plan))

return executed_tools
```

Figura 39: Execução das tools (parte 2/2)

Anexo J `_filter_tool_execution_results`

```
def filter_tool_execution_results(
    self,
    tool_execution_results: list[ToolExecutionOutput],
) -> list[ToolExecutionOutput]:
    """Removes low-relevance RAG contexts from each tool's execution result.

    Applies ``_context_is_grounded`` to every RAG context in the results and
    discards those that fail the score thresholds. Non-RAG evidences (e.g. ForeXAI
    results stored in the evidences list) are passed through unchanged.

    Args:
        tool_execution_results (list[ToolExecutionOutput]): Raw results returned
            by ExecuteToolPlanService.

    Returns:
        list[ToolExecutionOutput]: Filtered results where each entry's
            ``rag_context`` contains only sufficiently grounded chunks.
    """
    filtered_results: list[ToolExecutionOutput] = []
    for tool_execution_result in tool_execution_results:
        filtered_results.append(
            ToolExecutionOutput(
                tool_name=tool_execution_result.tool_name,
                arguments=dict(tool_execution_result.arguments),
                # Apply the two-tier score check to each RAG chunk individually.
                # Chunks that fail are dropped; the rest are preserved in order.
                rag_context=[
                    context
                    for context in tool_execution_result.rag_context
                    if self._context_is_grounded(context)
                ],
                # ForeXAI evidences are not subject to the RAG grounding filter
                # and are always forwarded to the Responder unchanged.
                evidences=list(tool_execution_result.evidences),
                workflow_name=tool_execution_result.workflow_name,
            )
        )
    return filtered_results
```

Figura 40: Grounding pós execução

Anexo K `_compose_assistant_reply`

```

async def _compose_assistant_reply(
    self,
    *,
    user_prompt: str,
    user_profile_type: str,
    can_answer: bool,
    tool_execution_results: list[ToolExecutionOutput],
) -> tuple[str, str | None]:
    """Composes the assistant's text reply (Responder phase of the pipeline).

    If the grounding policy determines there is insufficient evidence to answer
    (can_answer=False), returns a pre-defined response adapted to the user profile
    without invoking the LLM.

    Otherwise, builds the response prompt with the numbered evidences and delegates
    text generation to the ResponseComposerInterface.

    Args:
        user_prompt (str): Original user message.
        user_profile_type (str): User profile (e.g. COMMON_USER, ENERGY_SPECIALIST),
            used to adapt the style and level of detail of the response.
        can_answer (bool): Whether there is sufficient evidence to answer.
            Determined by GroundingPolicyService.evaluate().
        tool_execution_results (list[ToolExecutionOutput]): Results from the executed
            tools (RAG and ForeXAI), which provide the factual context to the LLM.

    Returns:
        tuple[str, str | None]: Pair of (response content, LLM model name used).
        The model name is None when the response is pre-defined (can_answer=False).
    """

    # Short-circuit: no evidence means the LLM would have nothing factual to rely on.
    # Return a profile-adapted fallback message without making an LLM call.
    if not can_answer:
        return get_insufficient_evidence_response(user_profile_type), None

    # Build a structured prompt that embeds the numbered evidences alongside the
    # user question, then delegate generation to the LLM via the composer adapter.
    llm_reply = await self._response_composer.compose_response(
        prompt=build_llm_prompt(
            user_prompt=user_prompt,
            user_profile_type=user_profile_type,
            tool_execution_results=tool_execution_results,
        ),
        # Pass any base64-encoded images produced by ForeXAI tools (e.g. SHAP plots)
        # so the LLM can reference them in its multimodal response.
        image_data_uris=extract_image_data_uris(tool_execution_results),
    )
    return llm_reply.content, llm_reply.model_name

```

Figura 41: Construção da resposta do assistente

Anexo L `_ingest_selected_documents`

```
def _ingest_selected_documents(
    self,
    *,
    source_documents: list[SourceDocument],
    force_reindex: bool,
) -> IngestDocumentsResult:
    indexed_documents = 0
    indexed_chunks = 0
    skipped_documents = 0
    failed_documents = 0
    document_details: list[IngestDocumentDetail] = []

    for source_document in source_documents:
        try:
            # Compute a fingerprint of the document that covers its content AND
            # the current chunking and embedding configuration. This means that
            # changing the chunking strategy or the embedding model automatically
            # triggers re-indexing on the next run, even if the file is unchanged.
            content_hash = self._compute_content_hash(source_document)
            existing_document = self._document_catalog.get_document_by_source_id(source_document.source_id)

            # Skip documents whose content hash has not changed since the last
            # ingestion, unless the caller explicitly requested a full re-index.
            if (
                not force_reindex
                and existing_document is not None
                and existing_document.content_hash == content_hash
            ):
                skipped_documents += 1
                document_details.append(
                    IngestDocumentDetail(
                        source_id=source_document.source_id,
                        action="SKIPPED",
                        content_type=source_document.content_type,
                        chunk_count=existing_document.chunk_count,
                    )
                )
                continue

            # Derive a deterministic document_id from the source_id so that the
            # same document always gets the same UUID across ingestion runs.
            # This allows replace_document_chunks to overwrite previous vectors
            # in Qdrant without orphaning stale data.
            document_id = str(uuid5(NAMESPACE_URL, source_document.source_id))

            # Split the document into chunks according to the configured strategy
            # (fixed-size, semantic Markdown, etc.).
            chunks = self._text_chunker.split_document(
                text=source_document.content,
                content_type=source_document.content_type,
            )
```

Figura 42: Fluxo RAG de ingestão documental (parte 1/3)

```

# Atomically replace all vectors for this document in Qdrant.
# Using replace (delete-then-insert) prevents stale chunks from a
# previous version of the document from remaining searchable.
self._vector_store.replace_document_chunks(
    document_id=document_id,
    chunks=[
        VectorStoreChunk(
            document_id=document_id,
            source_id=source_document.source_id,
            chunk_index=chunk.index,
            content=chunk.content,
        )
        for chunk in chunks
    ],
)

# Update the catalog document record with the new hash and chunk count.
# Upsert handles both the initial insert and subsequent re-indexing.
self._document_catalog.upsert_document(
    document_id=document_id,
    source_id=source_document.source_id,
    content_hash=content_hash,
    content_type=source_document.content_type,
    chunk_count=len(chunks),
)

# Replace the catalog chunk records to mirror what was written to Qdrant.
# These records enable the admin UI to display chunk-level details and
# support future features such as per-chunk relevance auditing.
self._document_catalog.replace_chunks(
    document_id=document_id,
    chunks=[
        DocumentCatalogChunk(
            # Chunk IDs are also deterministic: same document + same
            # chunk index always produces the same UUID.
            id=str(uuid5(NAMESPACE_URL, f"{document_id}:{chunk.index}")),
            document_id=document_id,
            source_id=source_document.source_id,
            chunk_index=chunk.index,
            content=chunk.content,
        )
        for chunk in chunks
    ],
)

indexed_documents += 1
indexed_chunks += len(chunks)
document_details.append(
    IngestDocumentDetail(
        source_id=source_document.source_id,
        action="INDEXED",
        content_type=source_document.content_type,
        chunk_count=len(chunks),
    )
)

```

Figura 43: : Fluxo RAG de ingestão documental (parte 2/3)

```
return IngestDocumentsResult(  
    indexed_documents=indexed_documents,  
    indexed_chunks=indexed_chunks,  
    skipped_documents=skipped_documents,  
    removed_documents=0,  
    failed_documents=failed_documents,  
    document_details=document_details,  
)
```

Figura 44: Fluxo RAG de ingestão documental (parte 3/3)

Anexo M `_retrieve_context`

```

async def retrieve_context(self, *, query: str) -> list[RAGContextOutput]:
    # Over-fetch from Qdrant (4x the desired limit) to ensure enough candidates
    # survive the hard score filter, deduplication, and relative score filter
    # before the final top-N cap is applied.
    contexts = self._vector_store.search(query=query, limit=self._max_documents * 4)

    # Pre-compute query terms once so the same set is reused for every chunk.
    query_terms = self._extract_query_terms(query)

    unique_contexts: list[RAGContextOutput] = []
    seen_pairs: set[tuple[str, str]] = set()

    for context in contexts:
        # Hard floor: discard chunks whose vector similarity is too low to be
        # relevant, before spending time on deduplication or reranking.
        if context.score < self._min_score:
            continue

        # Truncate the excerpt to the configured character limit so the LLM
        # prompt stays within a predictable token budget.
        excerpt = context.excerpt[: self._max_excerpt_chars].strip()

        # Deduplicate by (source_id, excerpt) to prevent the same passage from
        # appearing multiple times when a document has overlapping chunks.
        key = (context.source_id, excerpt)
        if not excerpt or key in seen_pairs:
            continue

        seen_pairs.add(key)
        unique_contexts.append(
            RAGContextOutput(
                source_id=context.source_id,
                excerpt=excerpt,
                score=context.score,
                # Preserve the raw vector score separately so the grounding policy
                # can inspect it independently from the combined score assigned later.
                vector_score=context.vector_score if context.vector_score is not None else context.score,
                lexical_overlap_score=context.lexical_overlap_score,
                document_id=context.document_id,
                chunk_index=context.chunk_index,
            )
        )

    # Rerank: compute a combined score (vector + weighted lexical overlap) and
    # sort descending. Chunks that share keywords with the query are promoted.
    ranked_contexts = self._rerank_contexts(
        query_terms=query_terms,
        contexts=unique_contexts,
    )

    # Remove outliers: drop chunks that are too far below the top-ranked result.
    filtered_contexts = self._filter_by_top_score(ranked_contexts)

```

Figura 45: Fluxo RAG de recuperação (parte 1/2)


```
# Return at most max_documents results, now with the combined score exposed
# as the primary score field so downstream consumers (grounding policy, UI)
# see the final ranking signal rather than the raw vector score.
return [
    RAGContextOutput(
        source_id=ranked_context.context.source_id,
        excerpt=ranked_context.context.excerpt,
        score=ranked_context.combined_score,
        vector_score=(
            ranked_context.context.vector_score
            if ranked_context.context.vector_score is not None
            else ranked_context.context.score
        ),
        lexical_overlap_score=ranked_context.lexical_overlap,
        document_id=ranked_context.context.document_id,
        chunk_index=ranked_context.context.chunk_index,
    )
    for ranked_context in filtered_contexts[: self._max_documents]
]
```

Figura 46: Fluxo RAG de recuperação (parte 2/2)

Anexo N Interface

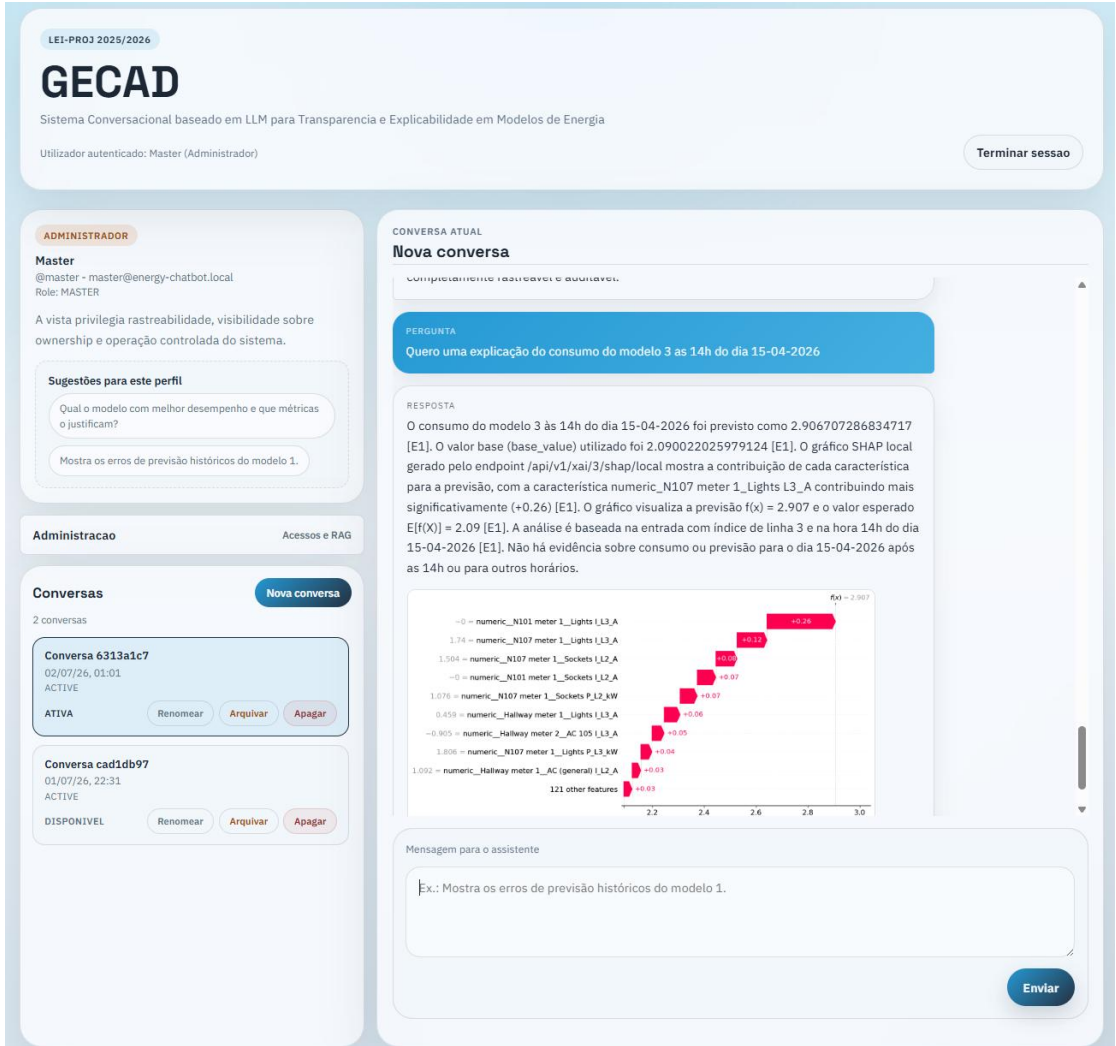


Figura 47: Vista geral do chat

CONVERSA ATUAL

Nova conversa

Completamente rastreável e auditável.

PERGUNTA

Quero uma explicação do consumo do modelo 3 as 14h do dia 15-04-2026

RESPOSTA

O consumo do modelo 3 às 14h do dia 15-04-2026 foi previsto como 2.906707286834717 [E1]. O valor base (base_value) utilizado foi 2.090022025979124 [E1]. O gráfico SHAP local gerado pelo endpoint /api/v1/xai/3/shap/local mostra a contribuição de cada característica para a previsão, com a característica numeric_N107 meter 1_Lights L3_A contribuindo mais significativamente (+0.26) [E1]. O gráfico visualiza a previsão $f(x) = 2.907$ e o valor esperado $E[f(X)] = 2.09$ [E1]. A análise é baseada na entrada com índice de linha 3 e na hora 14h do dia 15-04-2026 [E1]. Não há evidência sobre consumo ou previsão para o dia 15-04-2026 após as 14h ou para outros horários.

Característica	Contribuição
numeric_N101 meter 1_Lights L3_A	+0.26
numeric_N107 meter 1_Lights L3_A	+0.12
numeric_N107 meter 1_Sockets L2_A	+0.08
numeric_N101 meter 1_Sockets L2_A	+0.07
numeric_N107 meter 1_Sockets P_L2_kW	+0.07
numeric_Hallway meter 1_Lights L3_A	+0.06
numeric_Hallway meter 2_AC 105 L3_A	+0.05
numeric_N107 meter 1_Lights P_L3_kW	+0.04
numeric_Hallway meter 1_AC (general) L2_A	+0.03
121 other features	+0.03

Mensagem para o assistente

Ex.: Mostra os erros de previsão históricos do modelo 1.

Enviar

Figura 48: Vista específica do painel de conversação

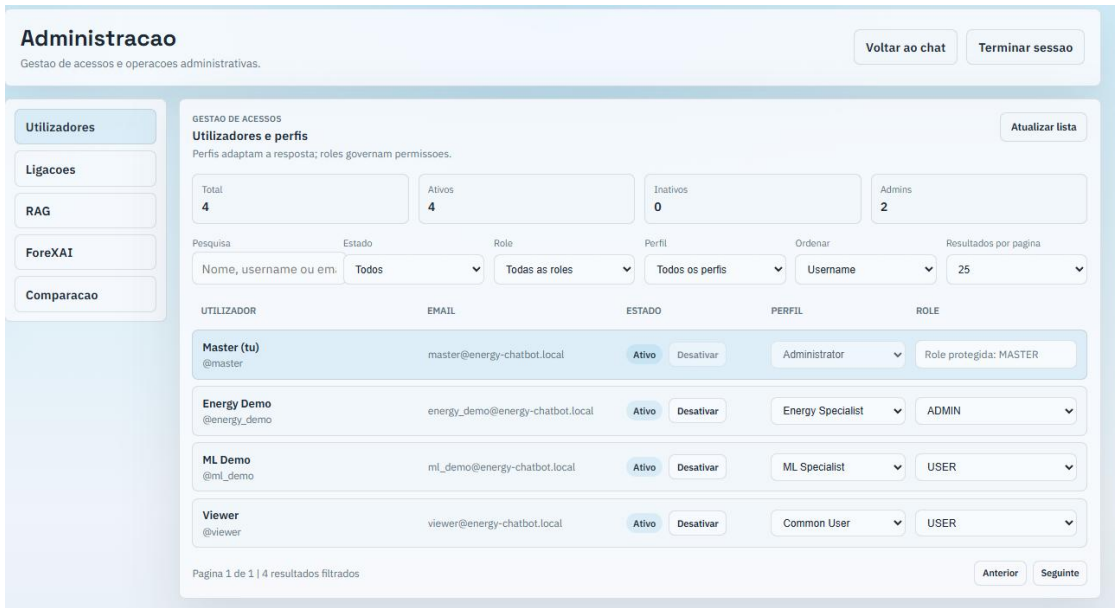


Figura 49: Painel de gestão de utilizadores

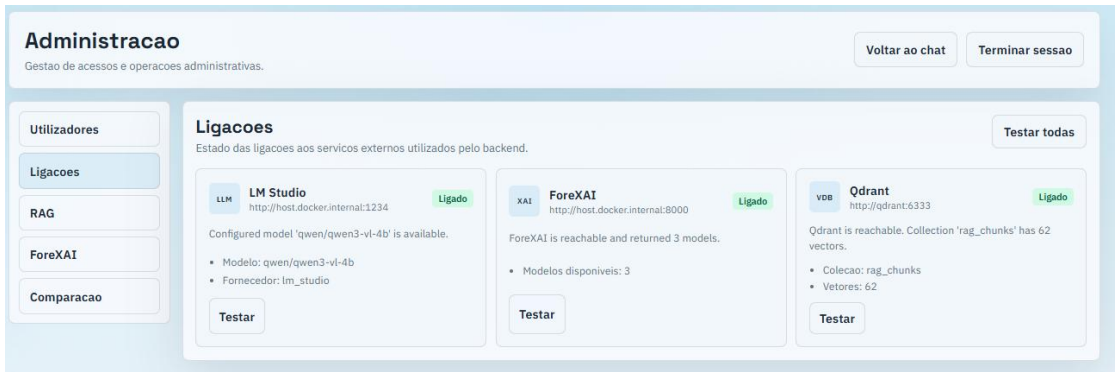


Figura 50: Painel de avaliação do estado das principais ligações

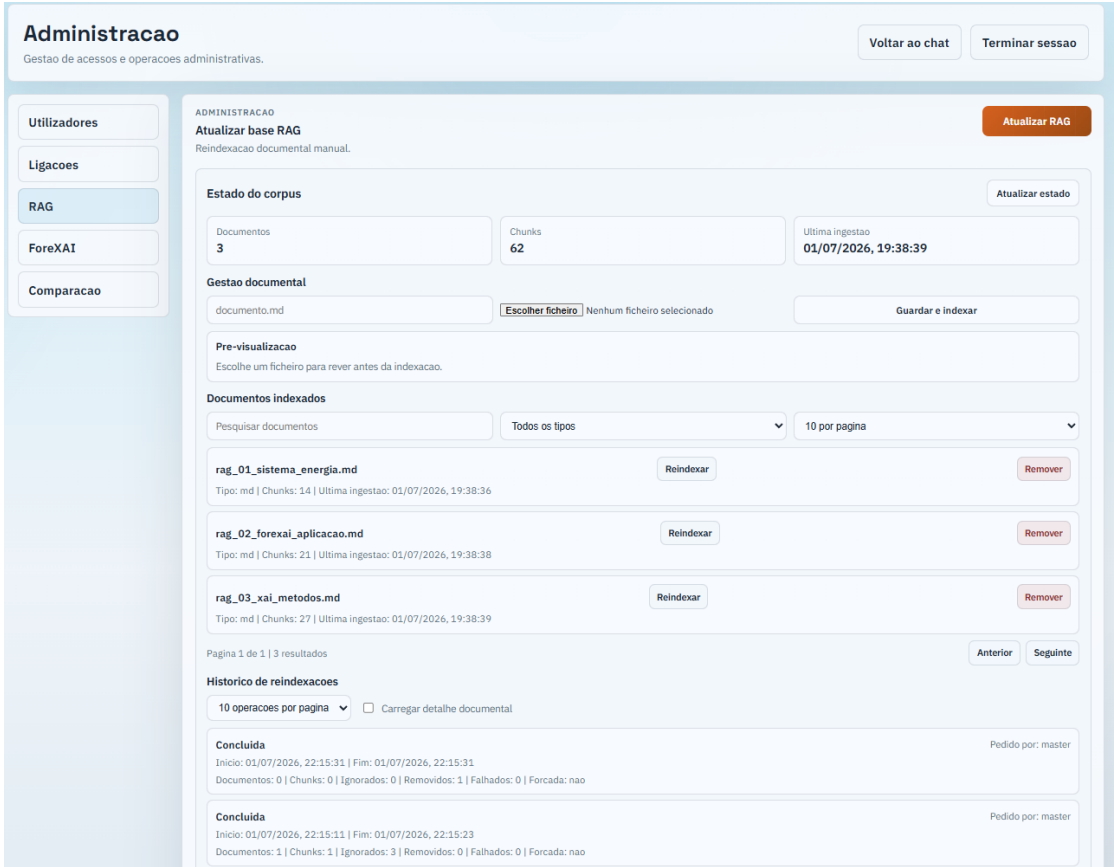


Figura 51: Painel de administração RAG

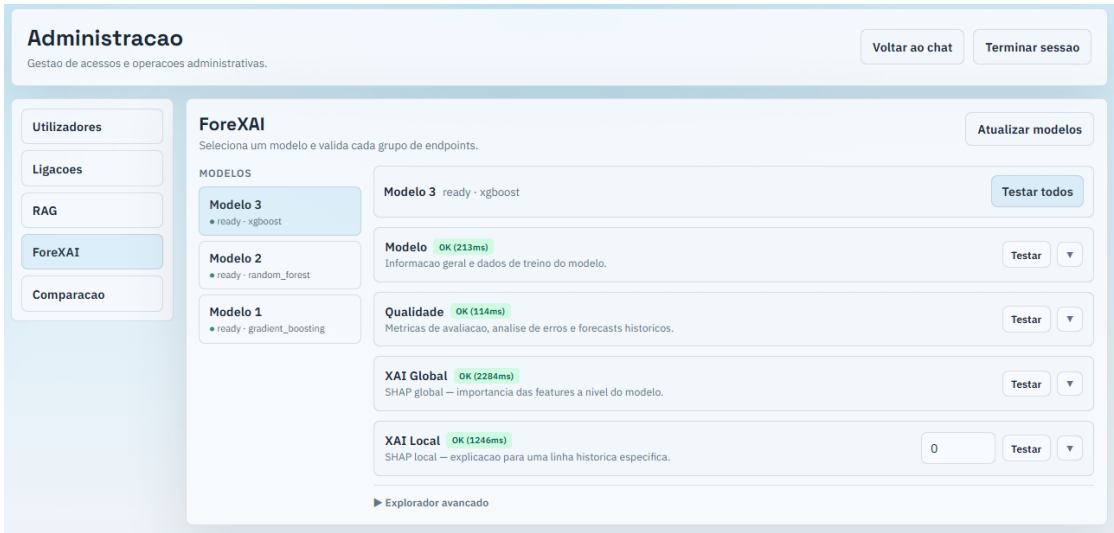


Figura 52: Painel de consulta de modelos ForeXAI

Administracao

Gestao de acessos e operacoes administrativas.

Voltar ao chat

Terminar sessao

Utilizadores

Ligacoes

RAG

ForeXAI

Comparacao

Configuracao da comparacao

PERFIL A

Utilizador Geral (COMMON_USER)

Utilizador Geral

PERFIL B

Especialista de Energia (ENERGY_SPECIALIST)

Especialista de Energia

PROMPTS STANDARD

O que é o SHAP e como se interpreta?

Qual o modelo com melhor desempenho segundo o MAE?

Explica o SHAP local do modelo 3 na linha 0.

Compara as métricas de todos os modelos disponíveis.

MESSAGEM

Escreve a mensagem ou escolhe um prompt standard acima...

Comparar

Figura 53: Painel de comparação da explicabilidade da resposta para diferentes perfis de utilizador